

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统



第三章：主存储器

第四章：数据的表示与运算

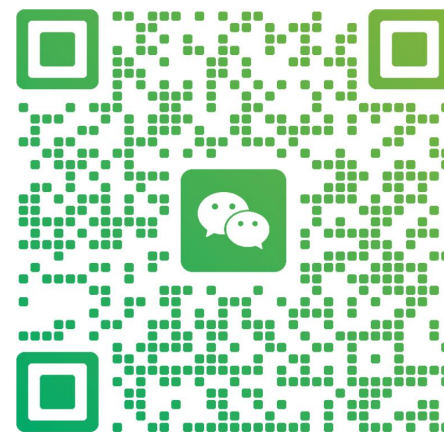
第五章：指令系统

第六章：中央处理器（CPU）

第七章：输入输出（I/O）系统

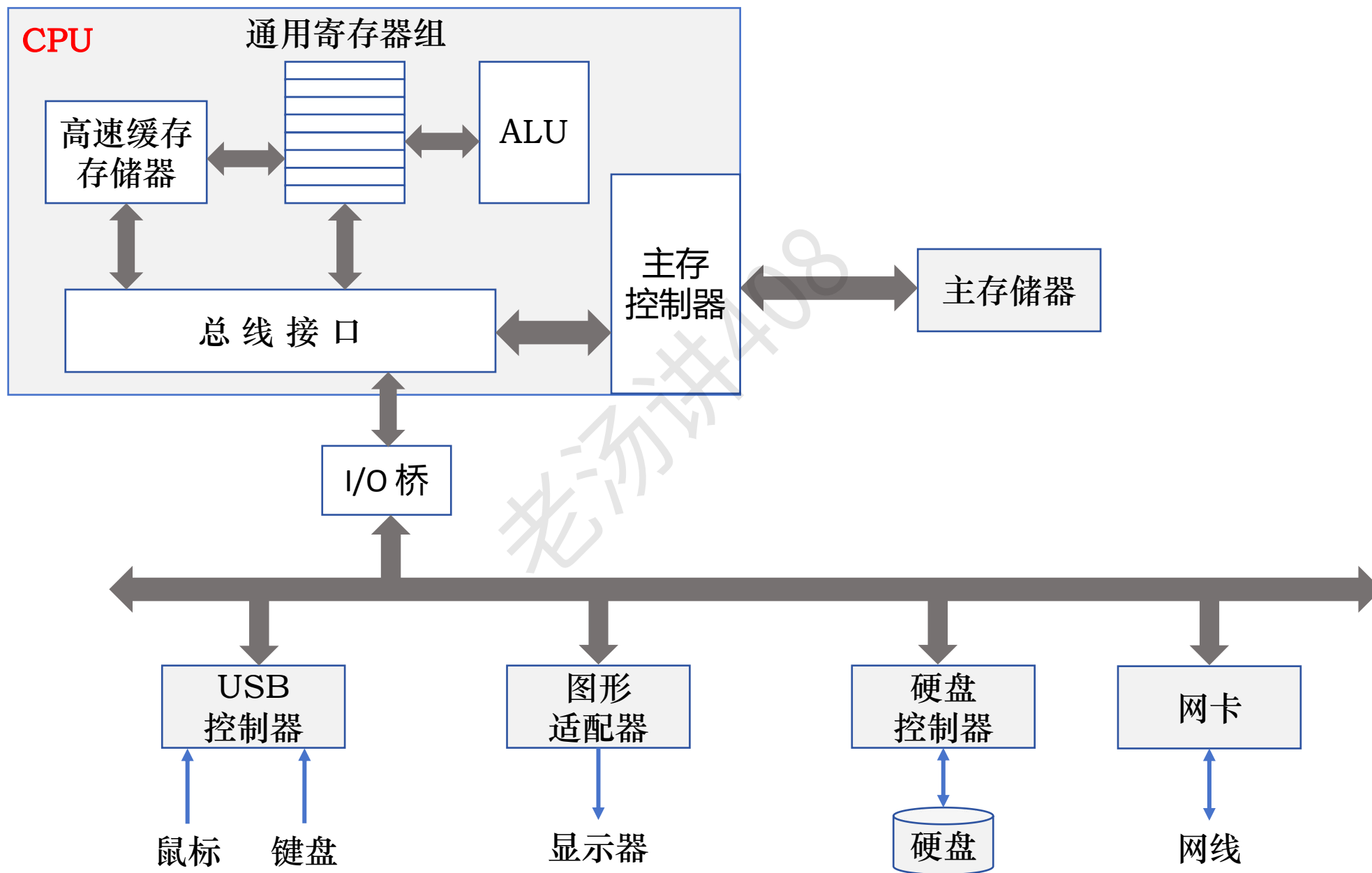


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

【第三章：主存储器】内容



计算机组成原理

讲师：老汤

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

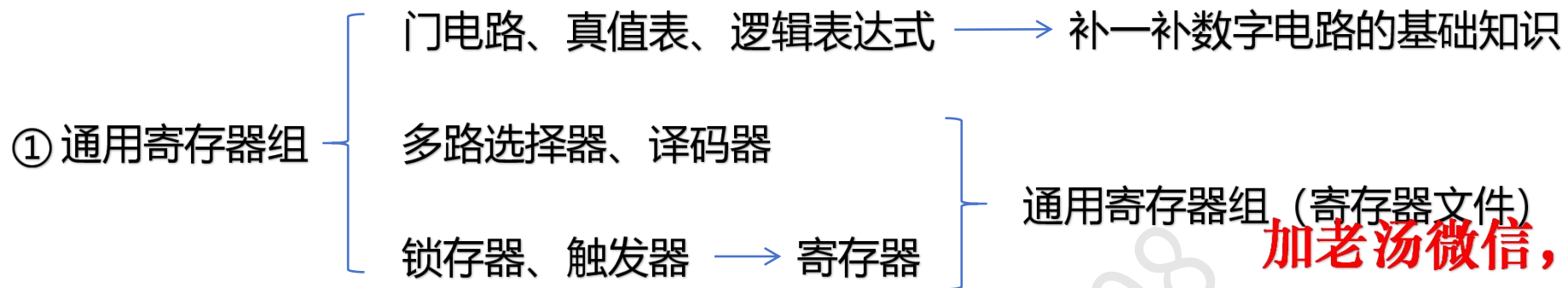
第五章：指令系统

第六章：中央处理器 (CPU)



第七章：输入输出 (I/O) 系统

【第三章：主存储器】内容



加老汤微信，帮你 408 冲 120+

② 主存储器



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

✓ 第四章：数据的表示与运算

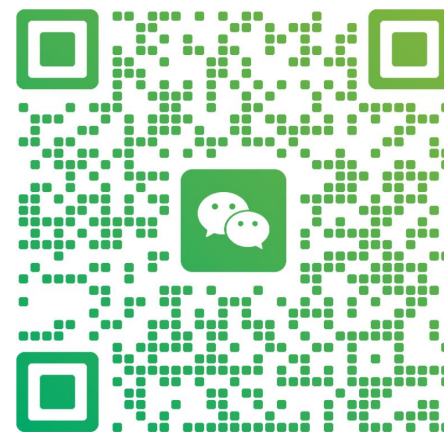
第五章：指令系统

第六章：中央处理器（CPU）

第七章：输入输出（I/O）系统

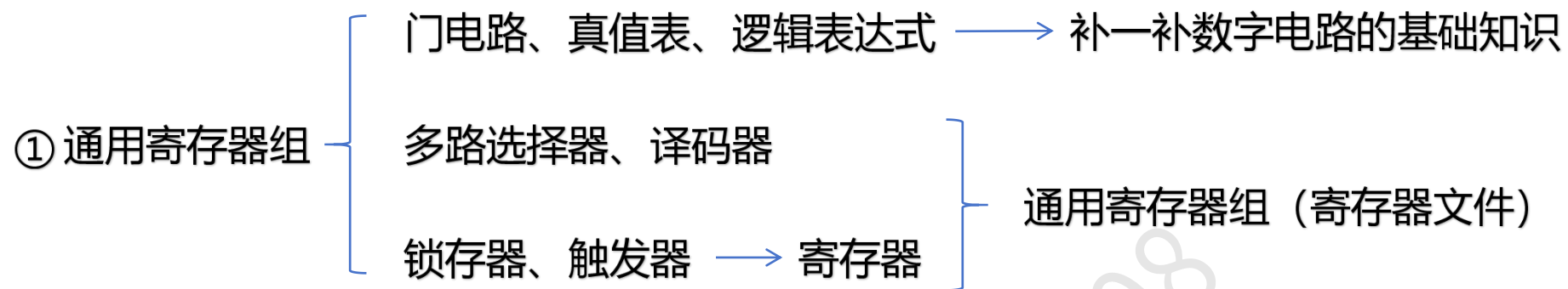


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

【第三章：主存储器】内容



② 主存储器

计算机组成原理

讲师：老汤

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器

第四章：数据的表示与运算

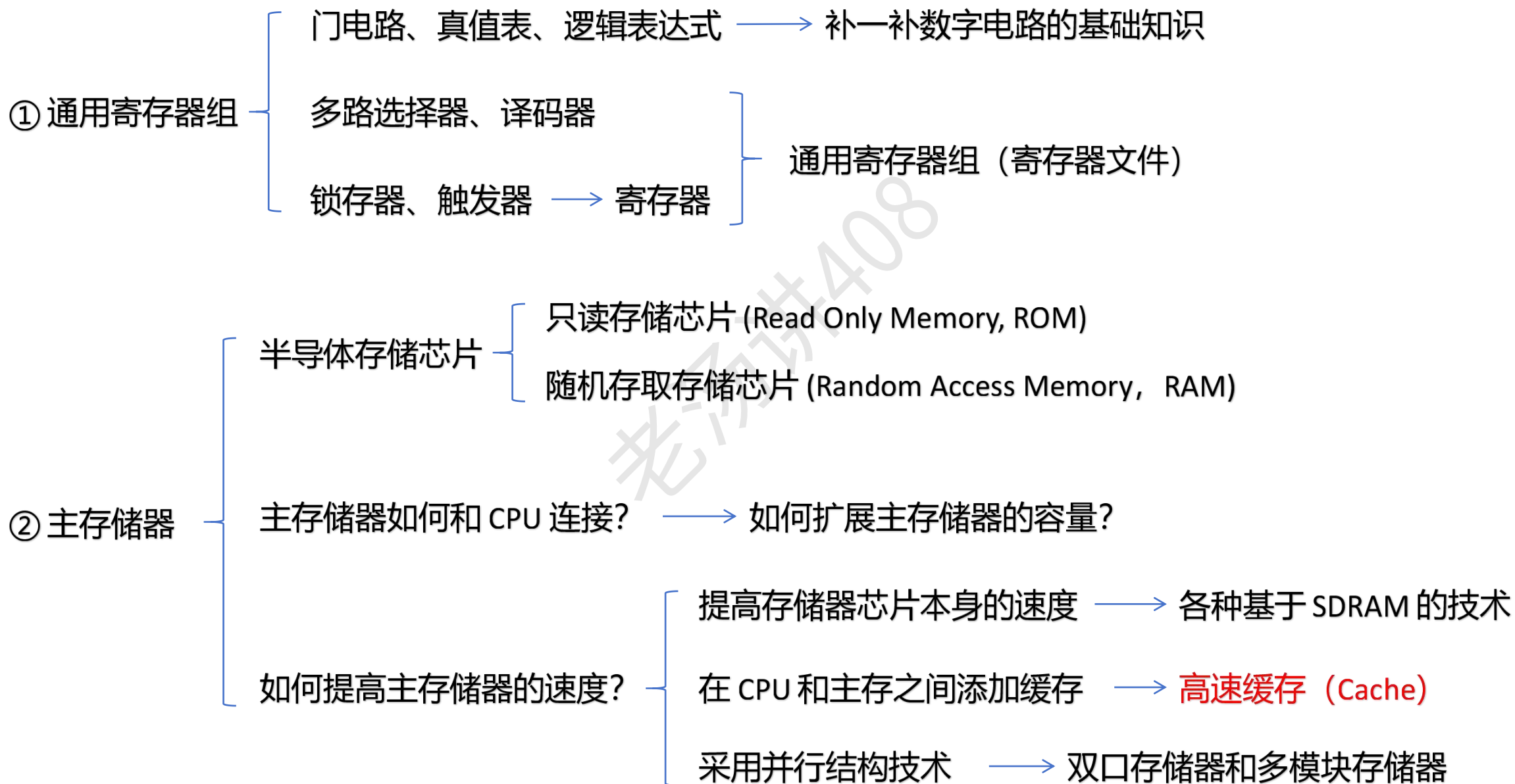
第五章：指令系统



第六章：中央处理器 (CPU)

第七章：输入输出 (I/O) 系统

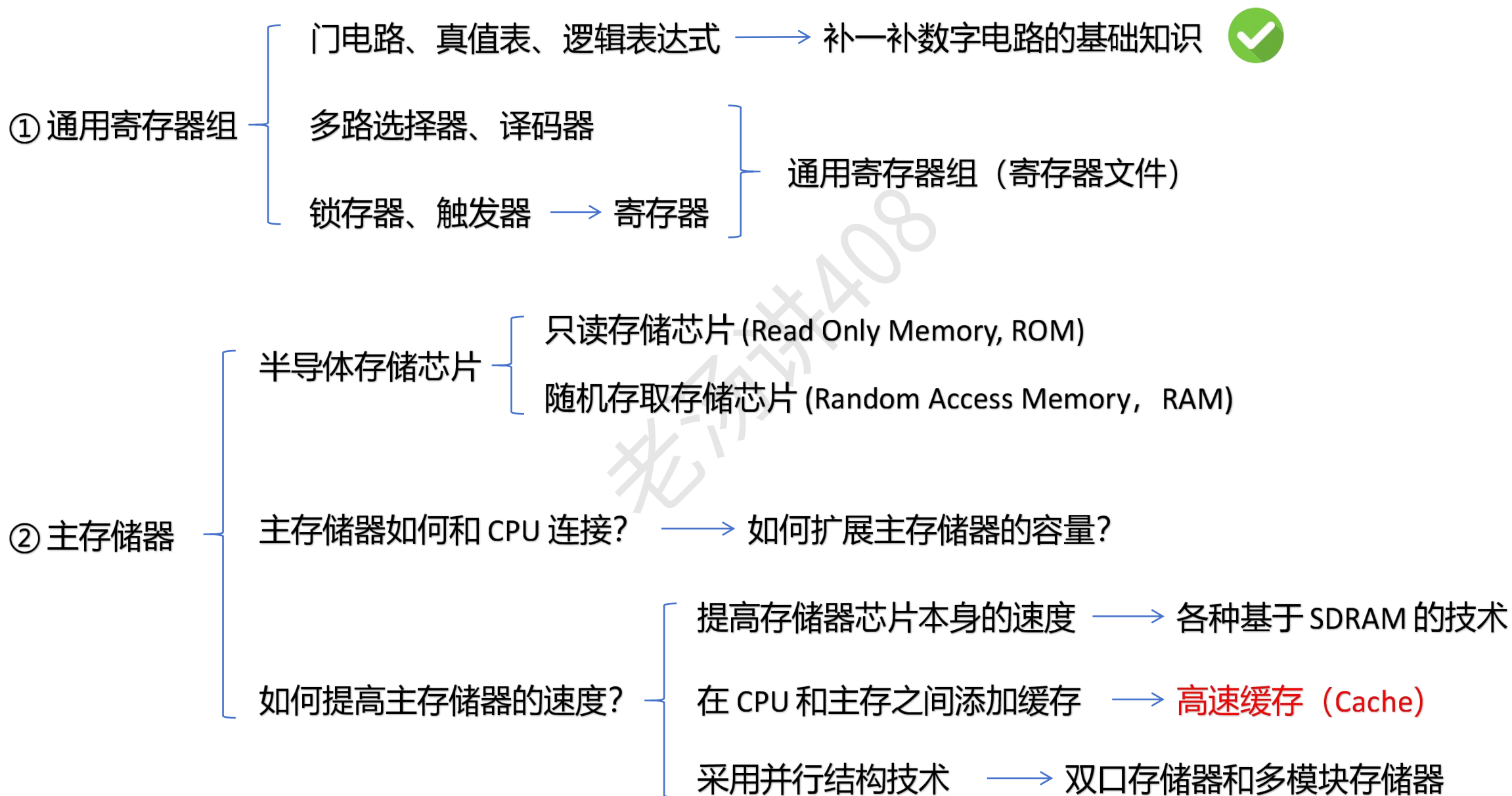
【第三章：主存储器】内容



408 如何考本章的内容?

老汤讲408

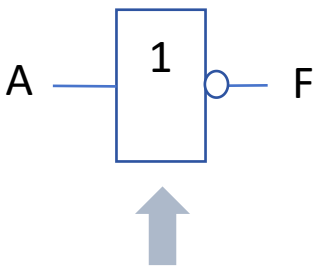
【第三章：主存储器】内容



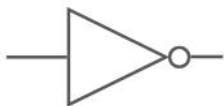
补一补数字电路基础：逻辑门电路

逻辑表达式：

$F = \overline{A}$

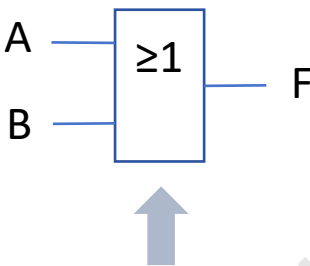


非门电路



逻辑和

$F = A + B$

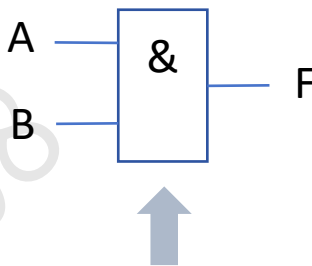


或门电路

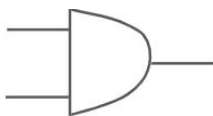


逻辑积

$F = A \cdot B$

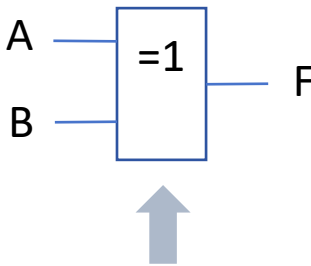


与门电路

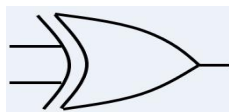


$F = A \wedge B$

$F = A \oplus B$

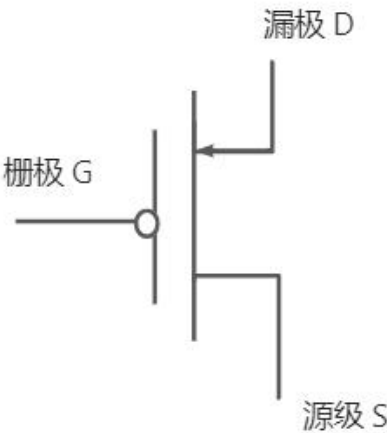


异或门电路



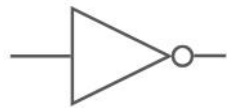
逻辑门电路

晶体管

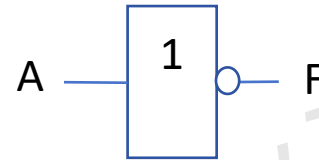


补一补数字电路基础：真值表

逻辑表达式: $F = \overline{A}$



非门电路



真值表



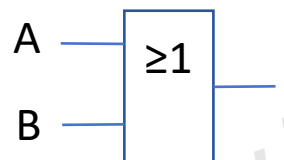
输入 (A)	输出 (F)
0	1
1	0

补一补数字电路基础：真值表



或门电路

逻辑表达式: $F = A + B$



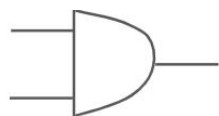
真值表



输入 (A)	输入 (B)	输出 (F)
0	0	0
0	1	1
1	0	1
1	1	1

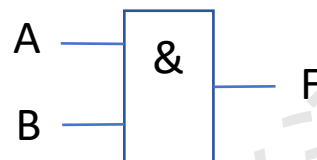
如果有一个输入是 1, 那么输出就是 1, 如果两个输入都是 0, 那么输出是 0

补一补数字电路基础：真值表



与门电路

逻辑表达式: $F = A \cdot B$



真值表



输入 (A)	输入 (B)	输出 (F)
0	0	0
0	1	0
1	0	0
1	1	1

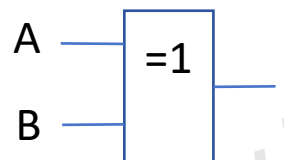
如果有一个输入是 0，那么输出就是 0，如果两个输入都是 1，那么输出是 1

补一补数字电路基础：真值表

逻辑表达式： $F = A \wedge B$



异或门电路



真值表

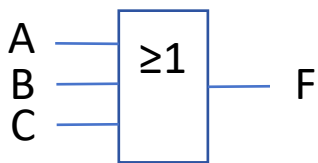


输入 (A)	输入 (B)	输出 (F)
0	0	0
0	1	1
1	0	1
1	1	0

如果两个输入相同，那么输出就是 0，如果两个输入不同，那么输出就是 1

补一补数字电路基础：真值表

逻辑表达式： $F = A + B + C$



或门电路

真值表

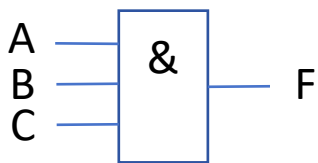


输入 (A)	输入 (B)	输入 (C)	输出 (F)
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

如果有一个输入是 1，那么输出就是 1，如果三个输入都是 0，那么输出是 0

补一补数字电路基础：真值表

逻辑表达式： $F = A \cdot B \cdot C$



与门电路

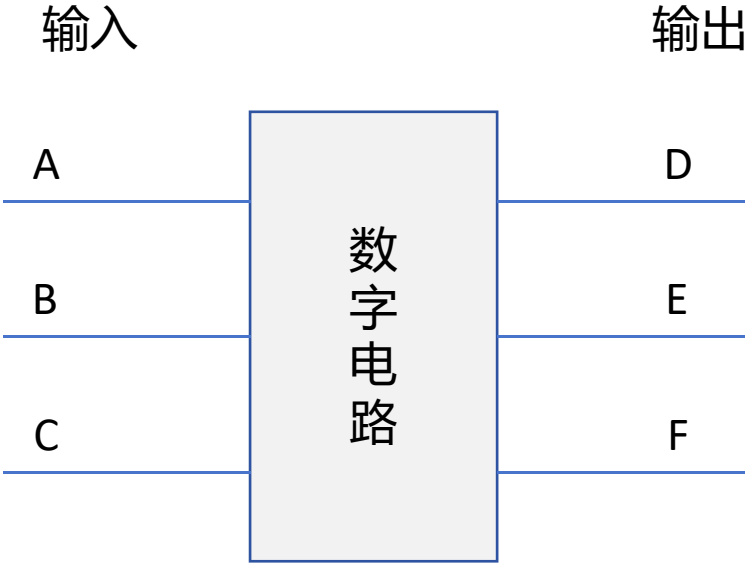
真值表



输入 (A)	输入 (B)	输入 (C)	输出 (F)
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

如果有一个输入是0，那么输出就是0，如果三个输入都是1，那么输出是1

补一补数字电路基础：真值表



真值表



输入			输出		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

数字电路实现的功能是：

- 如果有一个输入为 1，则输出 D 为 1；
- 如果只有两个输入为 1，则输出 E 为 1；
- 如果三个输入都是 1，则输出 F 为 1

补一补数字电路基础：输出的逻辑表达式

输入			输出		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

输出 $D = A + B + C$

输出 $F = A \cdot B \cdot C$

$$\begin{array}{ccccccc} \text{输出 } E = & A \bullet B \bullet \overline{C} & + & A \bullet C \bullet \overline{B} & + & B \bullet C \bullet \overline{A} & \\ & \text{乘积项} & & \text{乘积项} & & \text{乘积项} & \\ & \underbrace{\hspace{10em}} & & & & & \\ & & & & & & \text{乘积和} \end{array}$$

真值表中该输出为 1 的表项对应一个乘积项

乘积项为所有输入的乘积或输入取反后的乘积，是否取反取决于真值表中该变量对应的信号是 1 还是 0

补一补数字电路基础：输出的逻辑表达式

输入			输出
A	B	C	D
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$\text{输出 } D = \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot \bar{C} + A \cdot \bar{B} \cdot \bar{C} + A \cdot B \cdot C$$

同一定律： $A + 0 = A$, $A \cdot 1 = A$

0 和 1 定律： $A + 1 = 1$, $A \cdot 0 = 0$

互补定律： $A + \bar{A} = 1$, $A \cdot \bar{A} = 0$

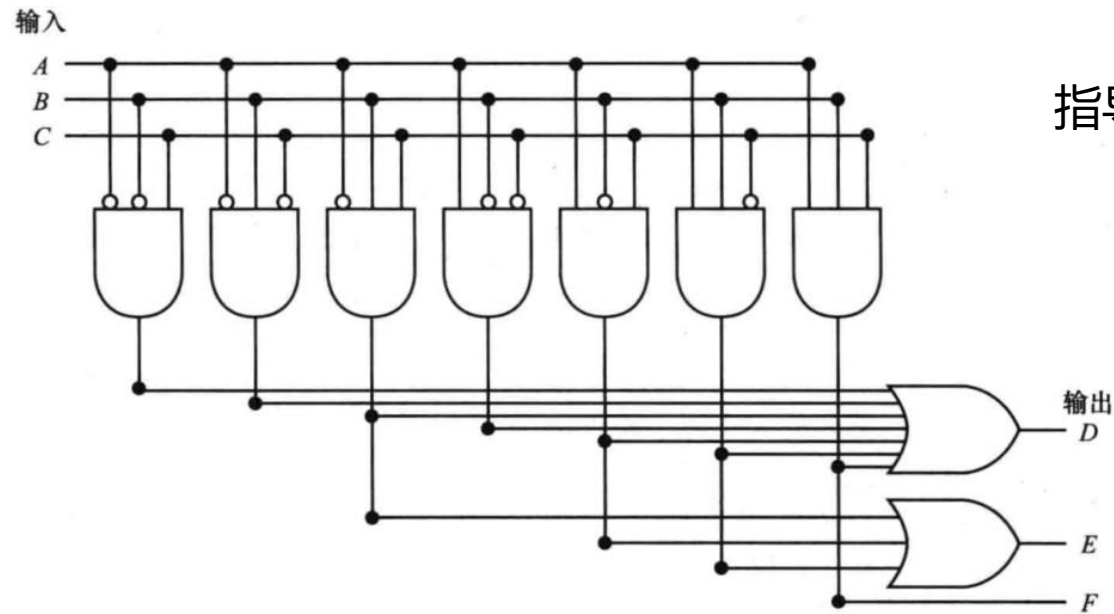
交换律： $A + B = B + A$, $A \cdot B = B \cdot A$

结合律： $A + (B + C) = (A + B) + C$, $A \cdot (B \cdot C) = (A \cdot B) \cdot C$

分配律： $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$, $A + (B \cdot C) = (A + B) \cdot (A + C)$

补一补数字电路基础：数字电路实现

可编程逻辑阵列 (PLA)



指导实现数字电路



输出 $D = A + B + C$

输出 $E = A \cdot B \cdot \bar{C} + A \cdot C \cdot \bar{B} + B \cdot C \cdot \bar{A}$

输出 $F = A \cdot B \cdot C$



输出逻辑表达式

数字电路实现的功能是：

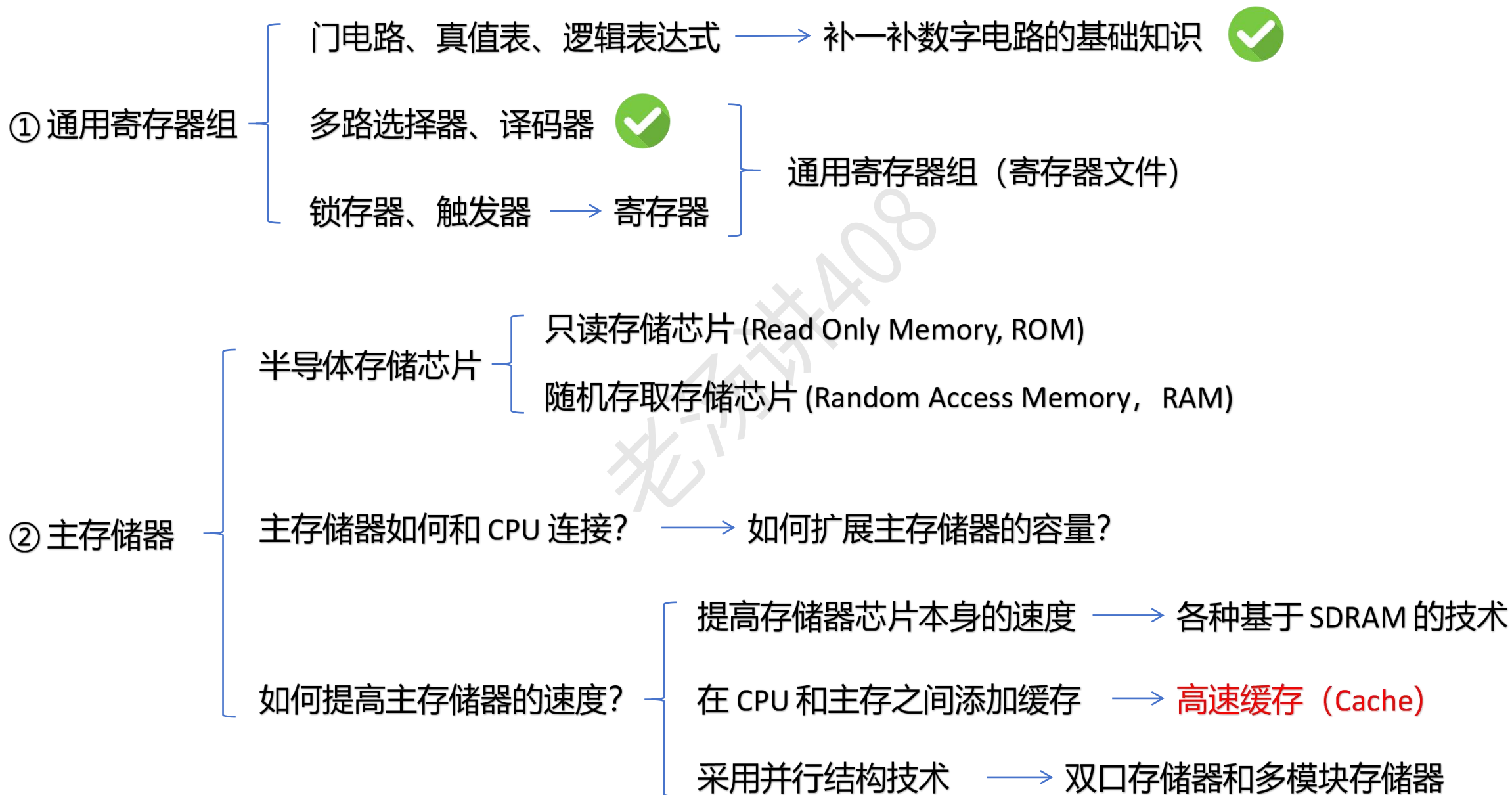
- 如果有一个输入为 1，则输出 D 为 1；
- 如果只有两个输入为 1，则输出 E 为 1；
- 如果三个输入都是 1，则输出 F 为 1

真值表

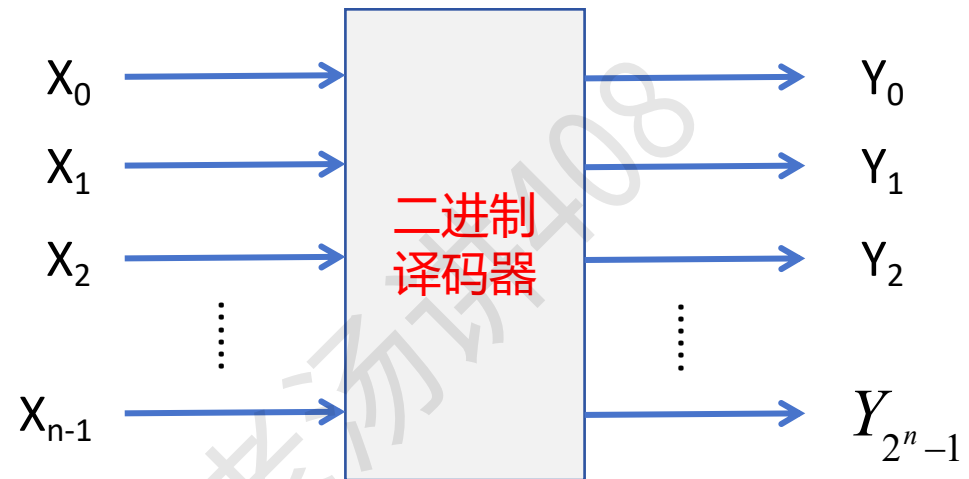


输入			输出		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

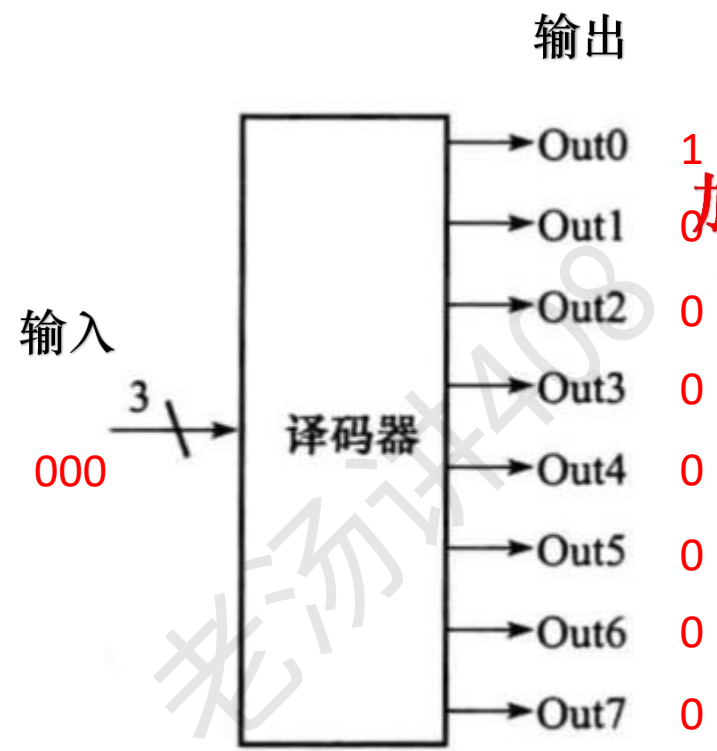
【第三章：主存储器】内容



补一补数字电路基础：译码器



补一补数字电路基础：译码器

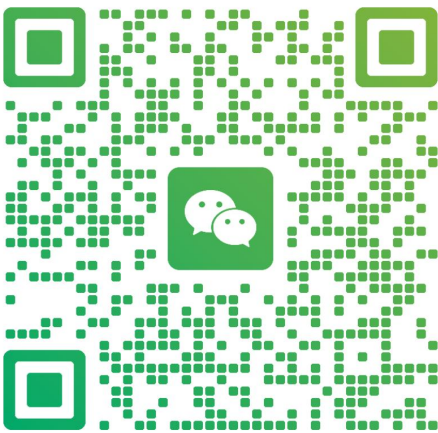


3-8 译码器

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

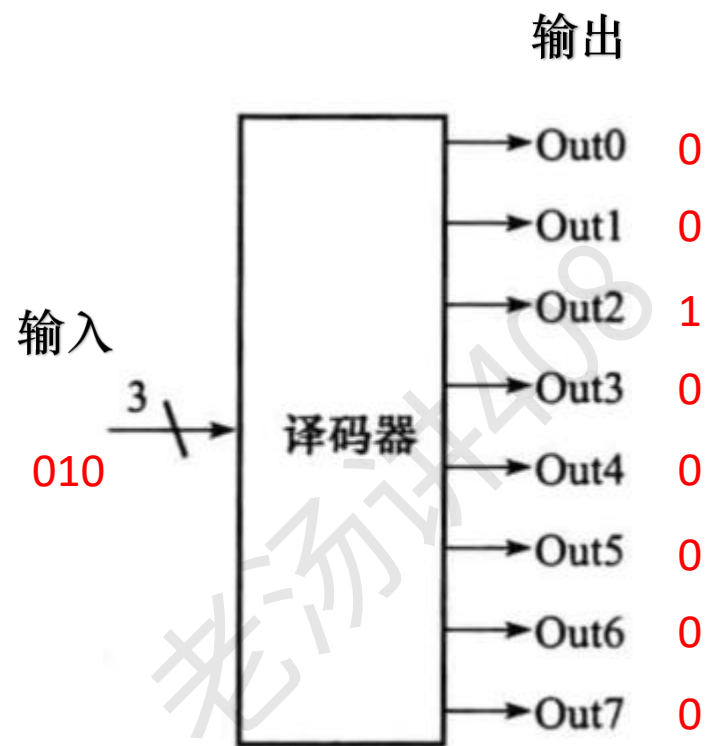


老汤
浙江 杭州



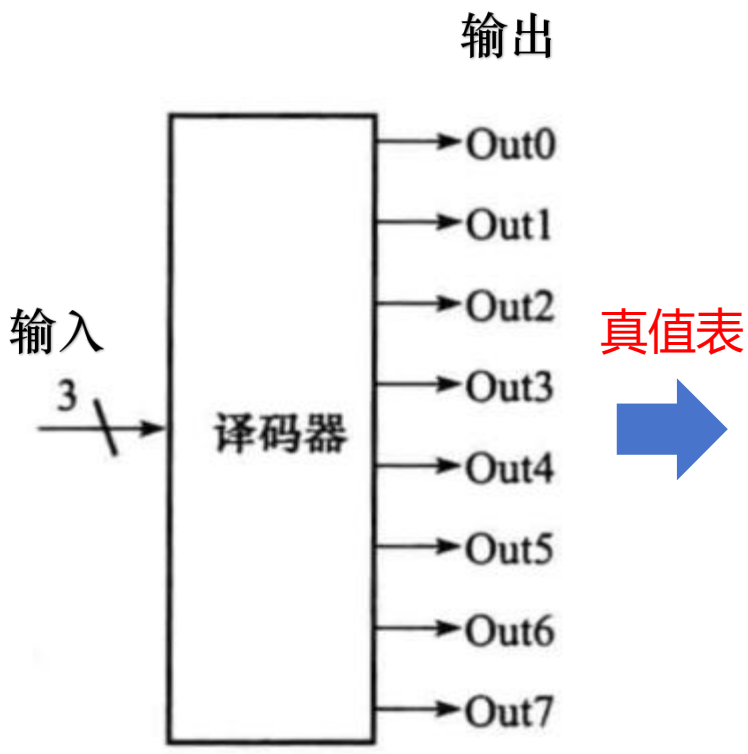
扫一扫上面的二维码图案，加我为朋友。

补一补数字电路基础：译码器



3-8 译码器

补一补数字电路基础：译码器



输入			输出							
12	11	10	Out7	Out6	Out5	Out4	Out3	Out2	Out1	Out0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

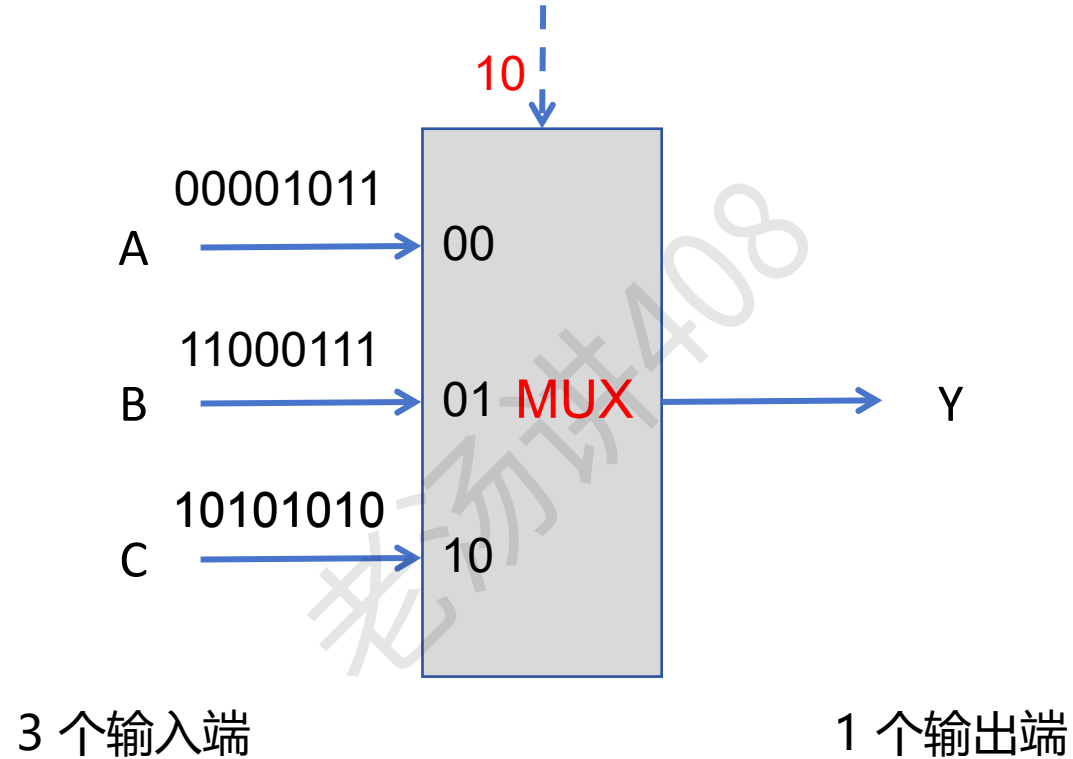
3-8 译码器

将输入转化成对应的十进制，然后将对应的输出设置为 1，其他的输出设置为 0

补一补数字电路基础：多路选择器

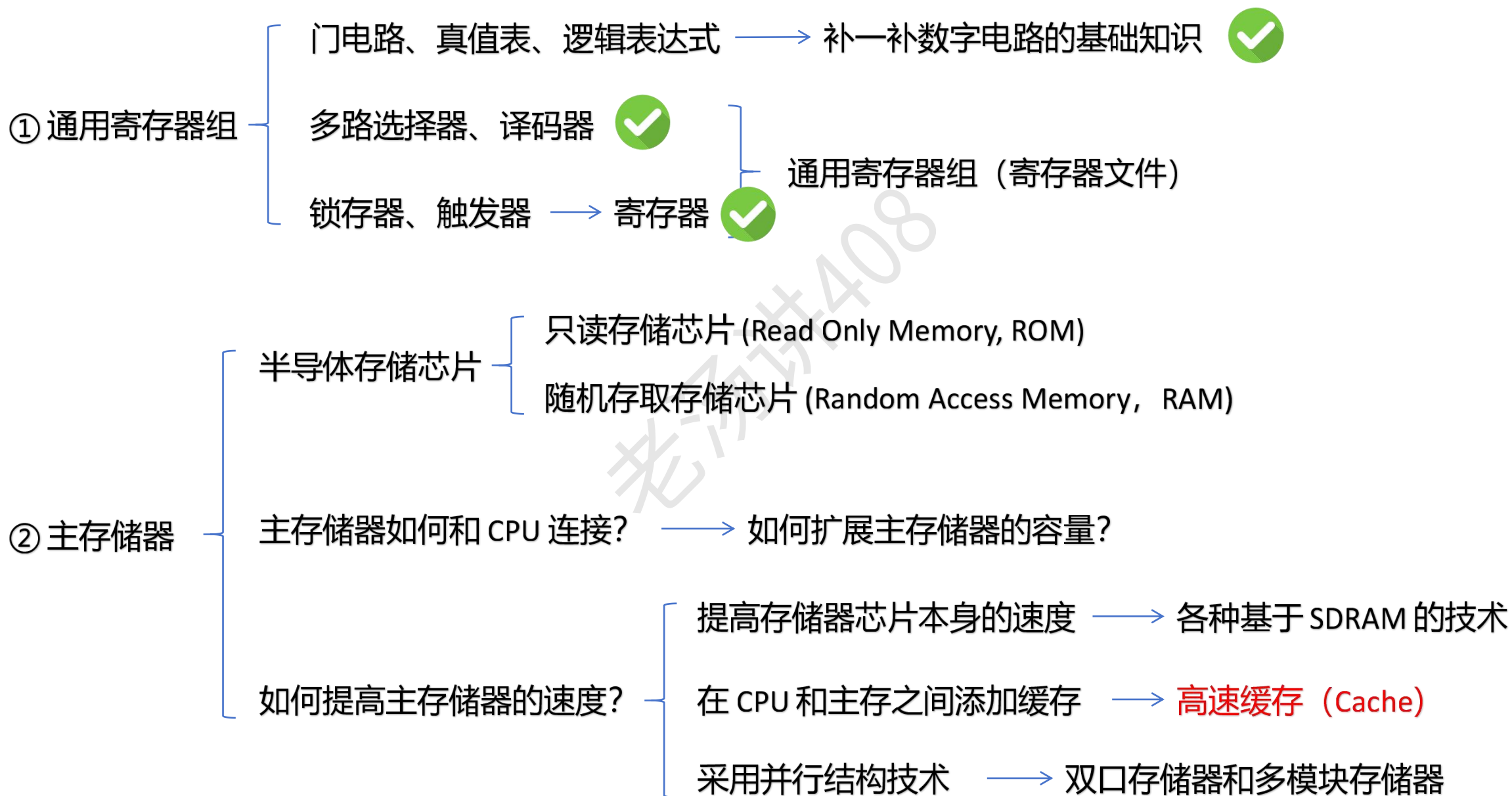
从多路输入数据中选择其中一路送至输出端

控制选择信号：Select



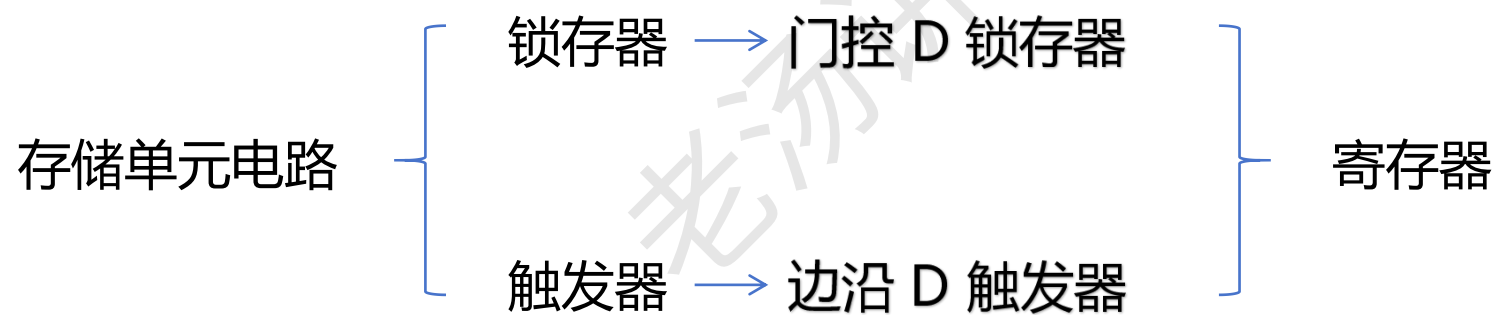
多路选择器 (multiplexer) : 简称 MUX

【第三章：主存储器】内容



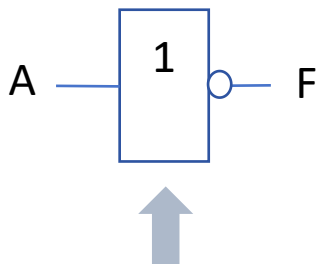
存储单元电路：锁存器和触发器

在数字电路中，可以存储一个二进制位的器件称为存储单元电路

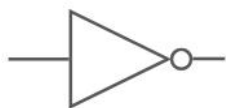


锁存器：逻辑门电路

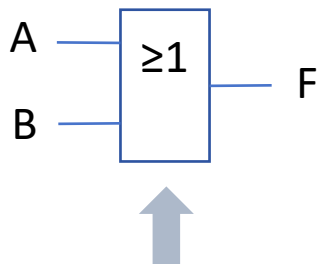
$$F = \overline{A}$$



非门电路



$$F = A + B$$

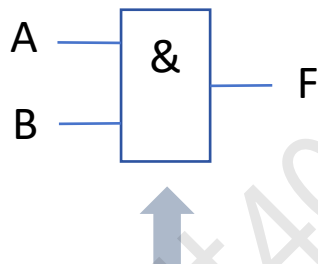


或门电路

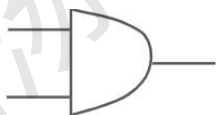


$$F = AB$$

$$F = A \cdot B$$

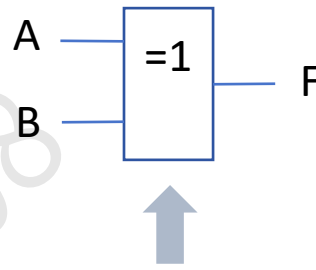


与门电路

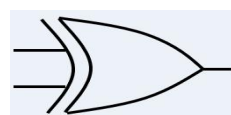


$$F = A \wedge B$$

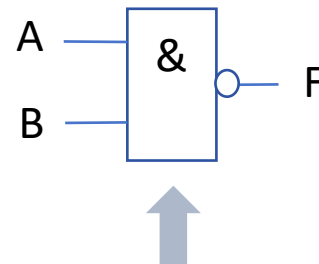
$$F = A \oplus B$$



异或门电路

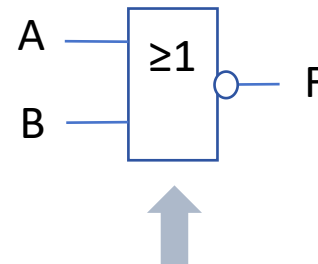


$$F = \overline{AB}$$



与非门电路

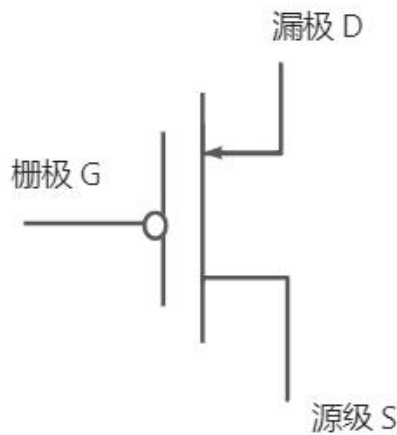
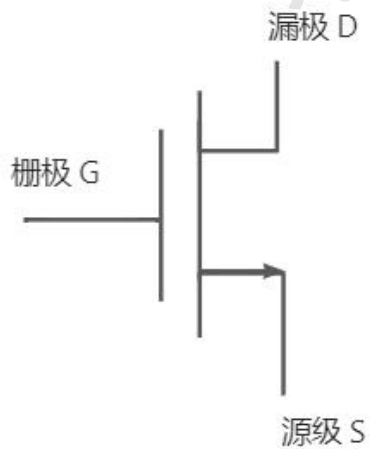
$$F = \overline{A + B}$$



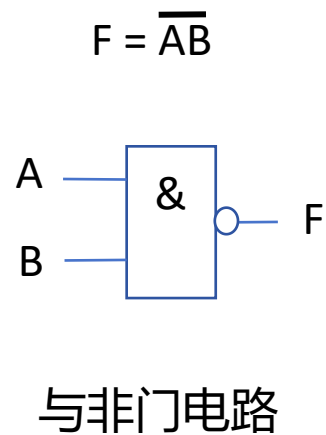
或非门电路

逻辑门电路

晶体管



锁存器：逻辑与非门电路

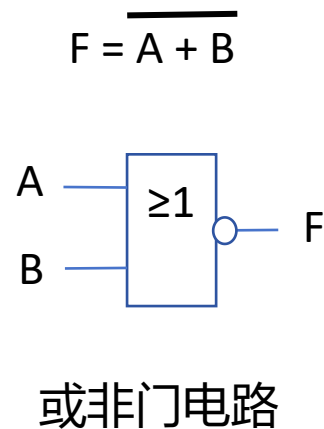


与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

如果 A 和 B 有一个为 0 的话，那么与非的结果必定是 1，只有当 A 和 B 都是 1 时，结果才是 0

锁存器：逻辑或非门电路



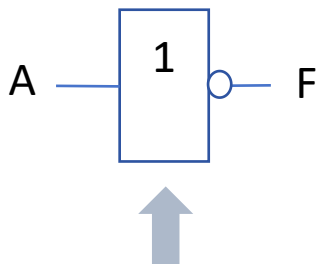
或非逻辑真值表

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

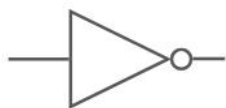
如果 A 和 B 有一个为 1 的话，那么或非的结果必定是 0，只有当 A 和 B 都是 0 时，结果才是 1

锁存器：逻辑门电路

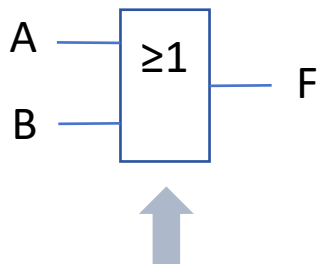
$$F = \overline{A}$$



非门电路



$$F = A + B$$

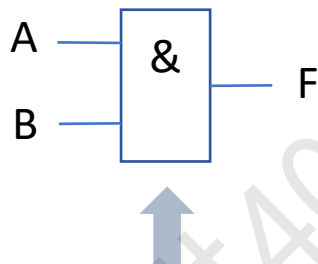


或门电路

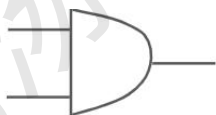


$$F = AB$$

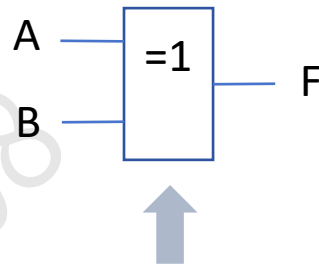
$$F = A \cdot B$$



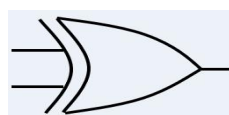
与门电路



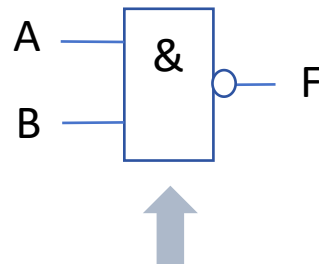
$$F = A \oplus B$$



异或门电路

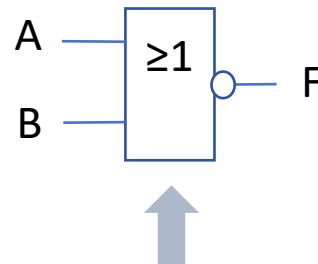


$$F = \overline{AB}$$



与非门电路

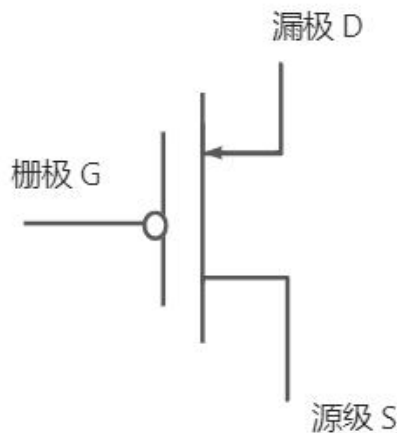
$$F = \overline{A + B}$$



或非门电路

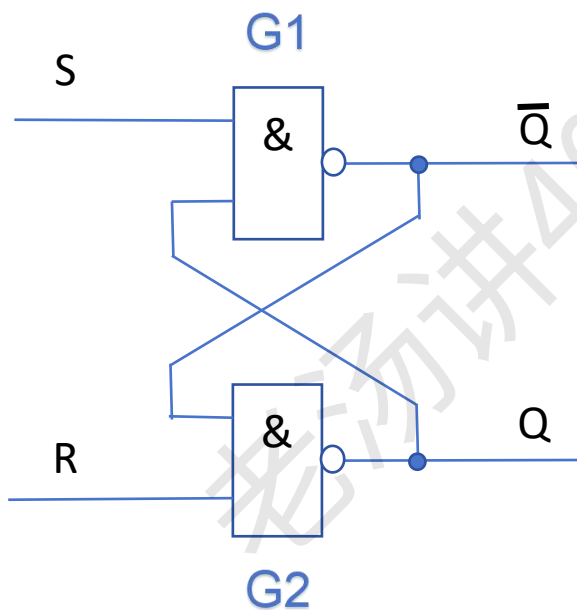
逻辑门电路

晶体管



RS 锁存器：与非门电路实现

Reset -Set 锁存器，复位 - 置位锁存器



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



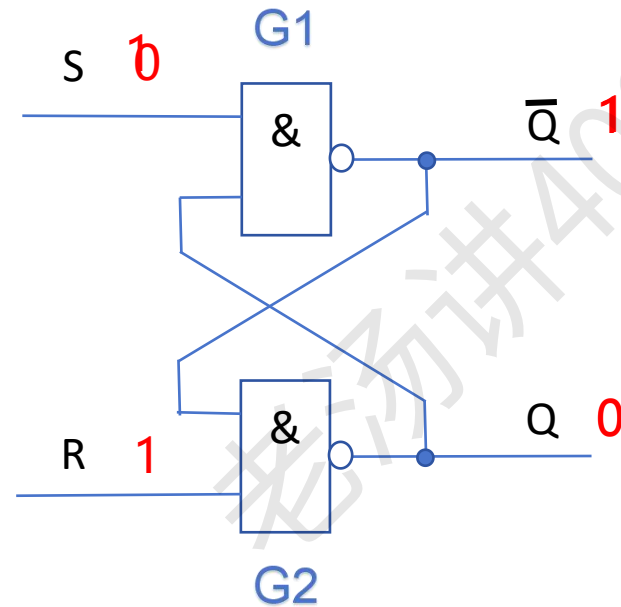
老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

RS 锁存器：与非门电路实现

Reset -Set 锁存器，复位 - 置位锁存器



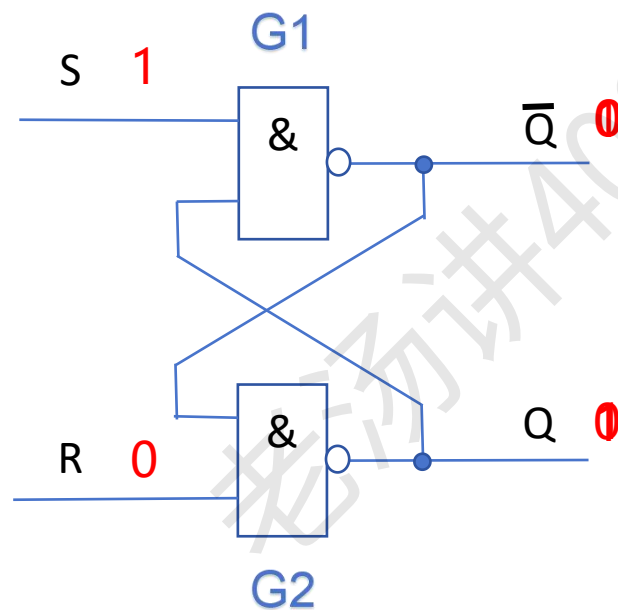
0 状态

与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

RS 锁存器：与非门电路实现

Reset -Set 锁存器，复位 - 置位锁存器



1 状态

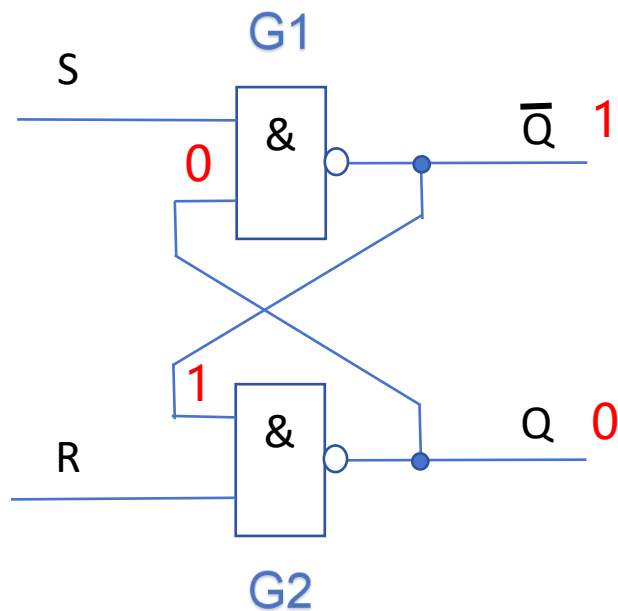
与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

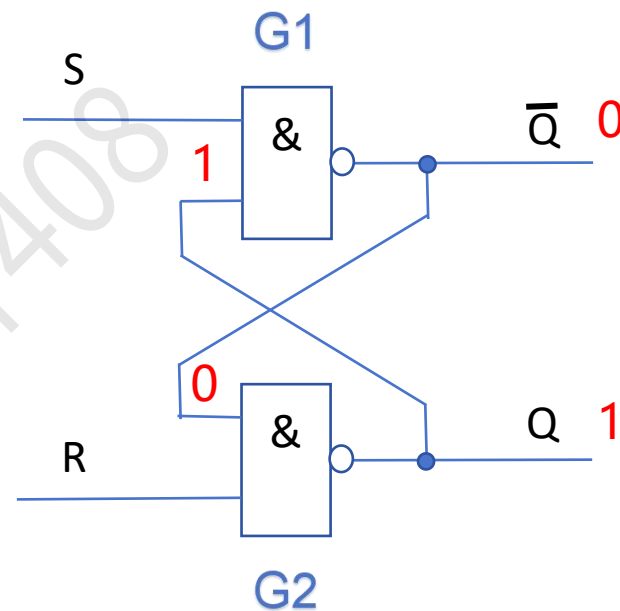
RS 锁存器：与非门电路实现

Reset -Set 锁存器，复位 - 置位锁存器

双稳态锁存器



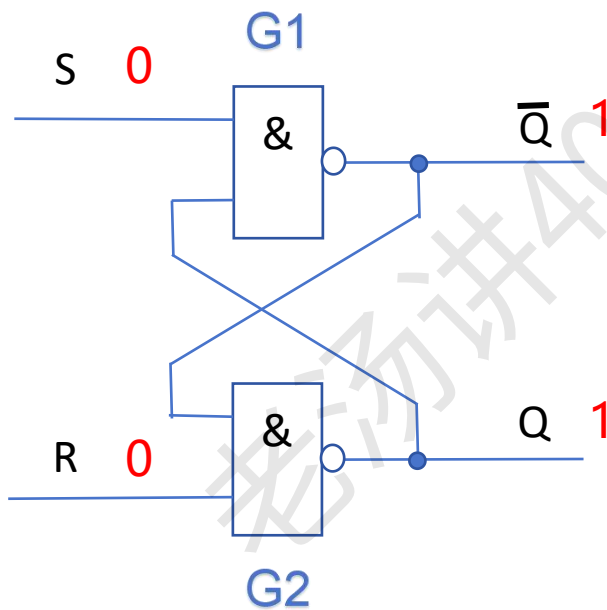
0 状态
↓
二进制 0



1 状态
↓
二进制 1

RS 锁存器：与非门电路实现

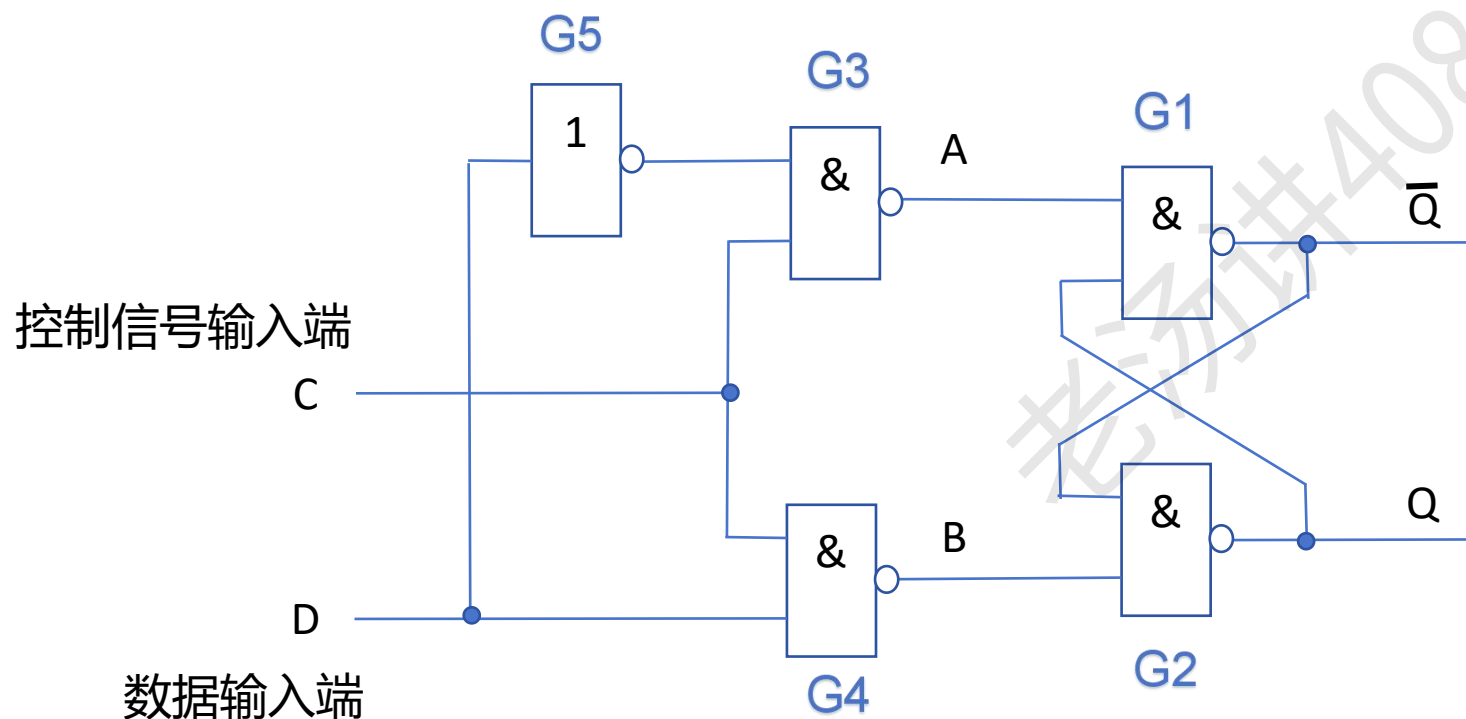
Reset -Set 锁存器，复位 - 置位锁存器



与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

门控 D 锁存器



Control: 控制

Data: 数据

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



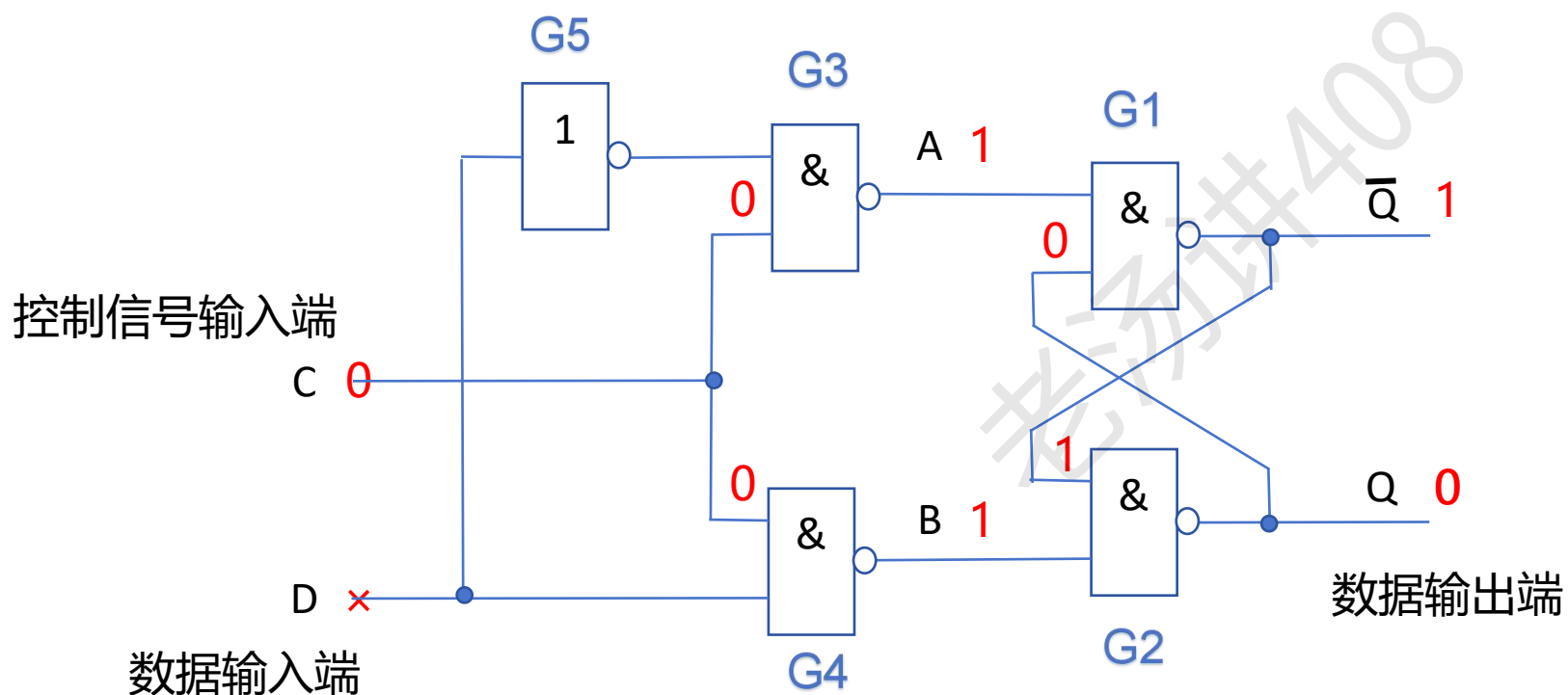
老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

门控 D 锁存器

① $C = 0$, $D = x$, 读取锁存器中保存的二进制 0



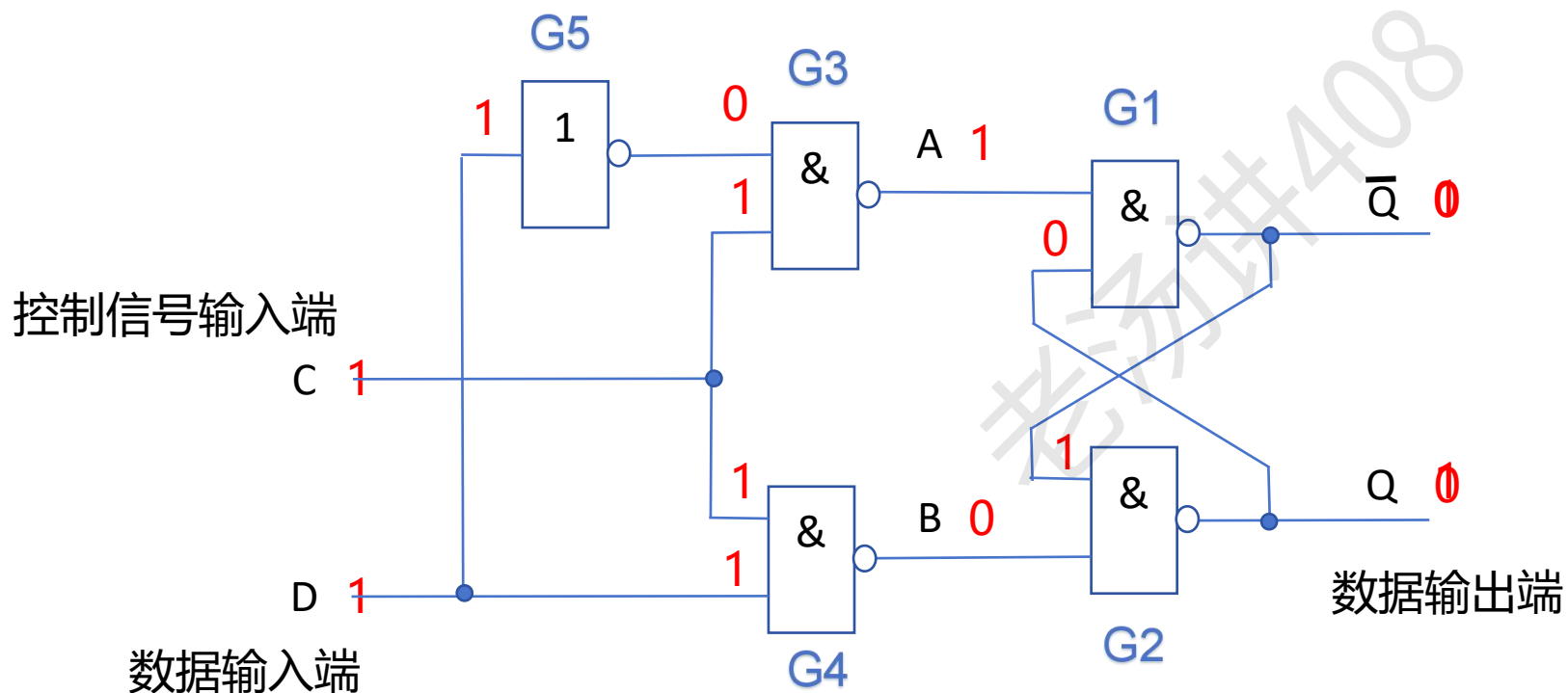
与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

门控 D 锁存器

① $C = 0$, $D = x$, 读取锁存器中保存的二进制 0

② $C = 1$, $D = 1$, 将二进制 1, 保存到锁存器



与非逻辑真值表

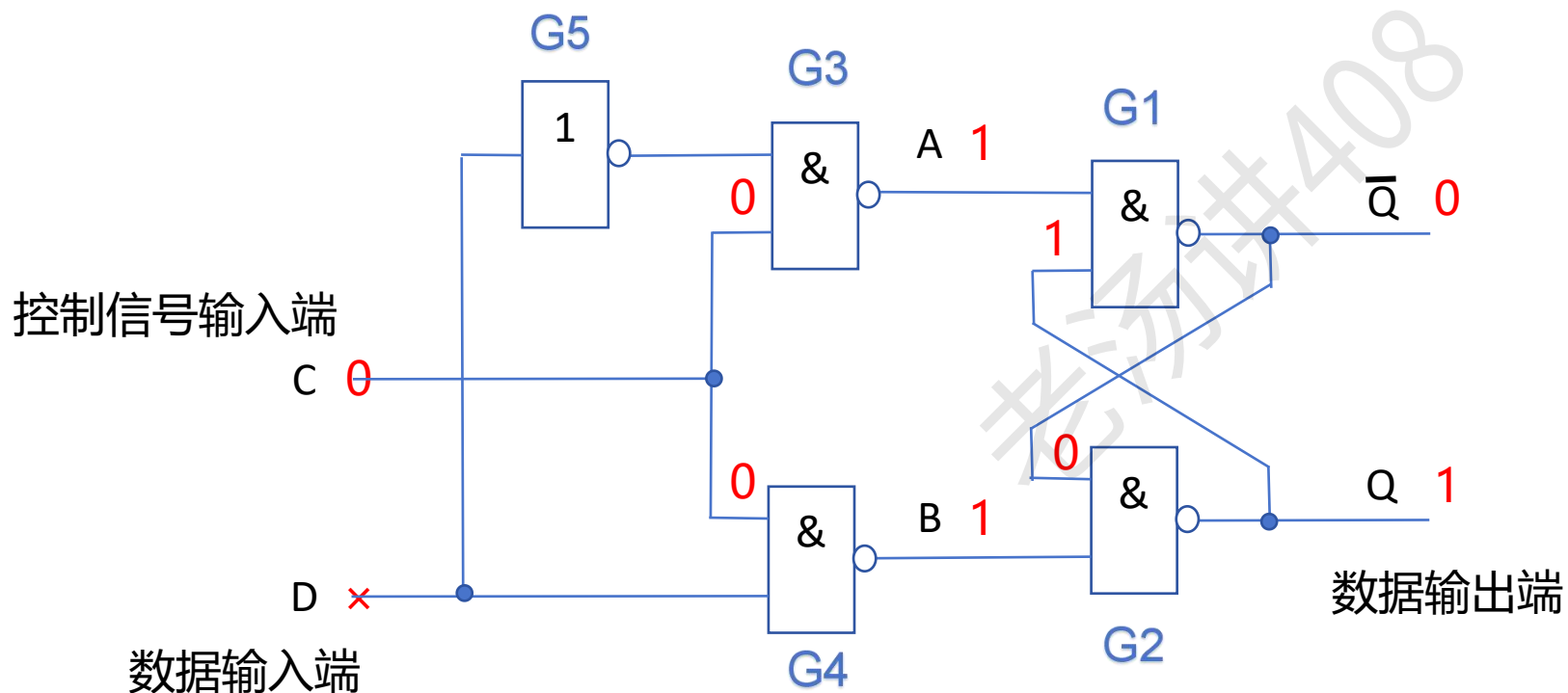
A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

门控 D 锁存器

① $C = 0$, $D = x$, 读取锁存器中保存的二进制 0

② $C = 1$, $D = 1$, 将二进制 1, 保存到锁存器

③ $C = 0$, $D = x$, 读取锁存器中保存的二进制 1



与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

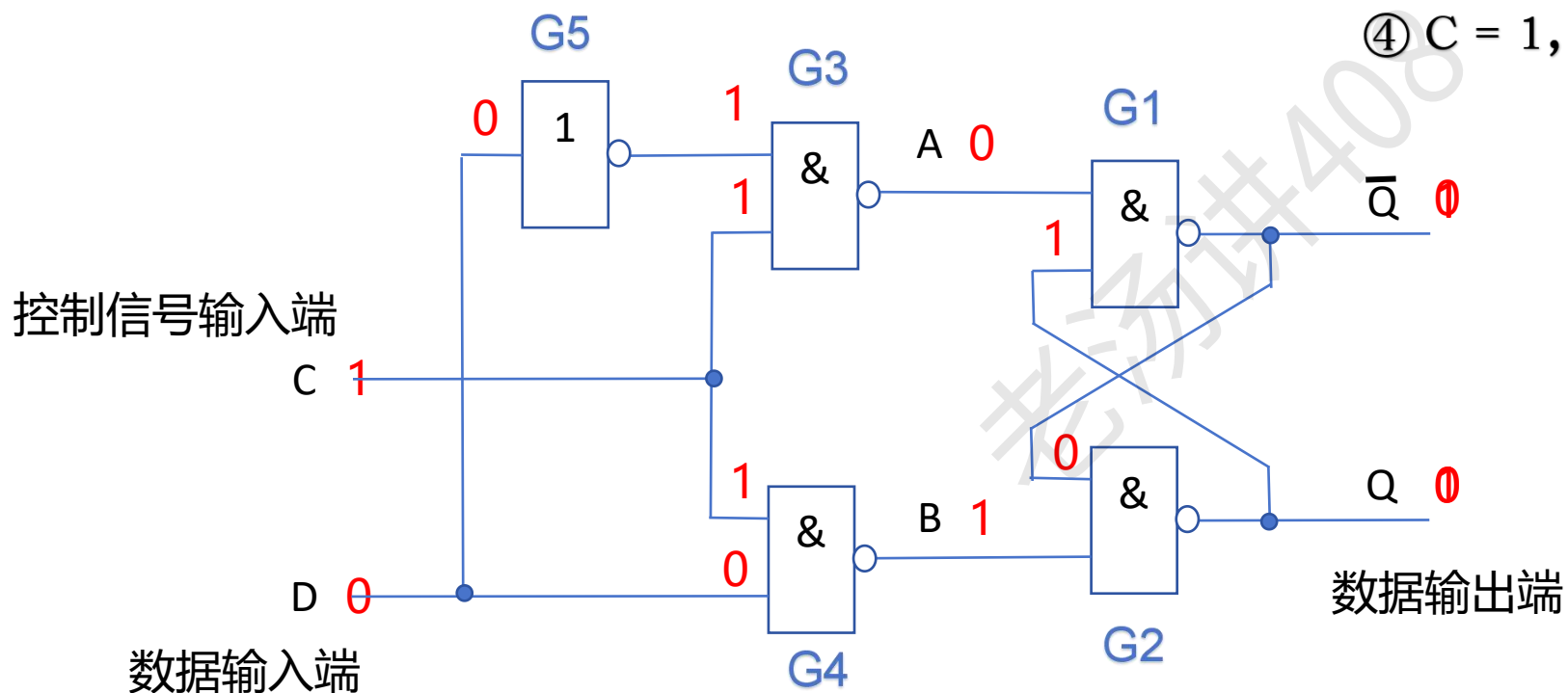
门控 D 锁存器

① $C = 0$, $D = x$, 读取锁存器中保存的二进制 0

② $C = 1$, $D = 1$, 将二进制 1, 保存到锁存器

③ $C = 0$, $D = x$, 读取锁存器中保存的二进制 1

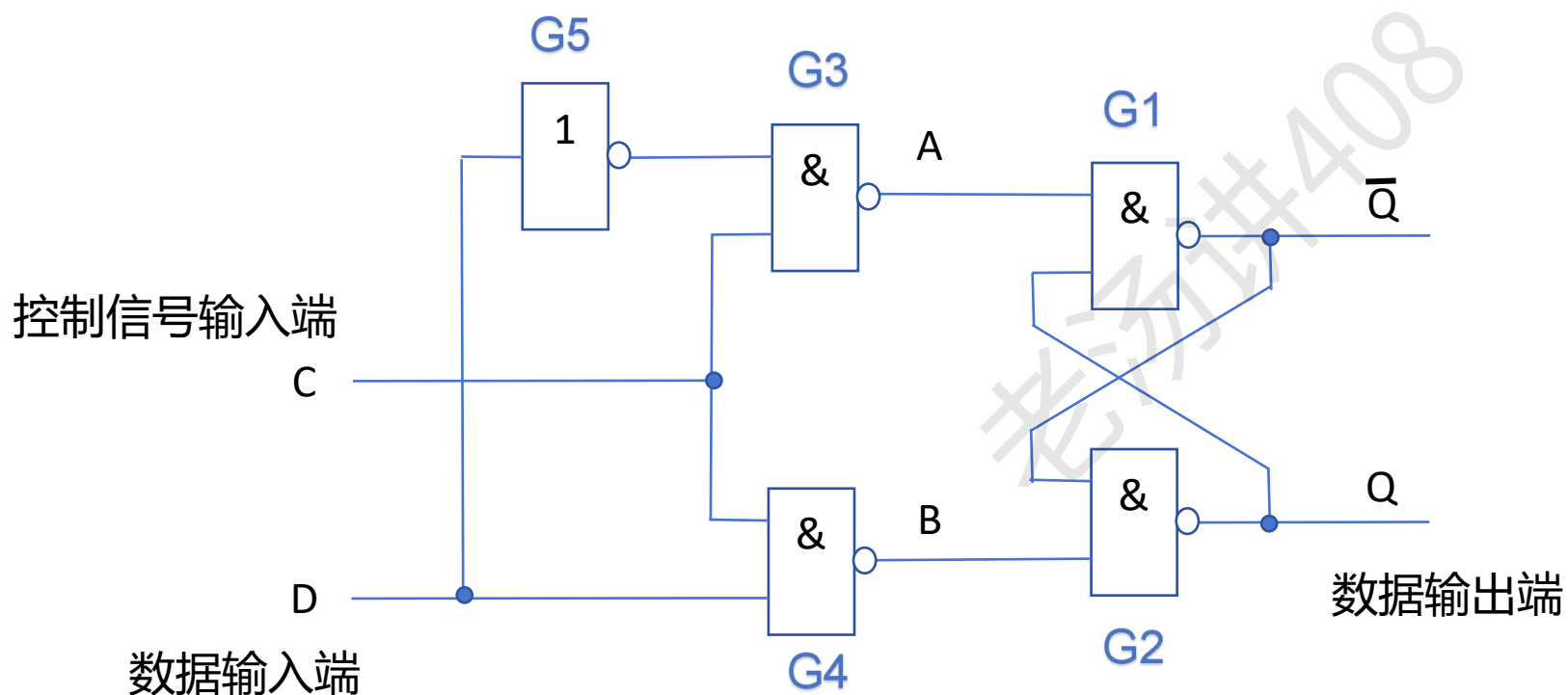
④ $C = 1$, $D = 0$, 将二进制 0, 保存到锁存器



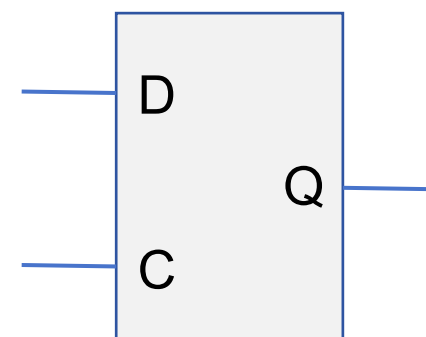
与非逻辑真值表

A	B	$F = \overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

门控 D 锁存器



门控 D 锁存器



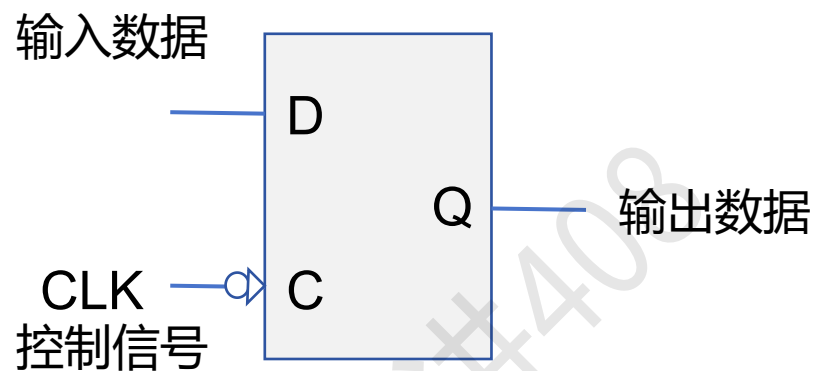
$C = 0$, 读取二进制

$C = 1$, $D = 1$, 保存二进制 1

$C = 1$, $D = 0$, 保存二进制 0

门控 D 锁存器

门控触发器

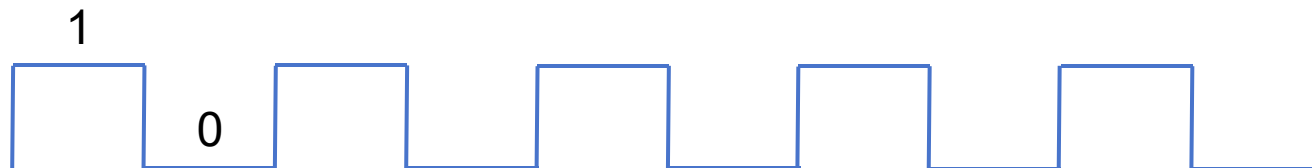


$C = 0$, 读取二进制

$C = 1, D = 1$, 保存二进制 1

$C = 1, D = 0$, 保存二进制 0

CLK: CLOCK 的缩写, 表示时钟信号

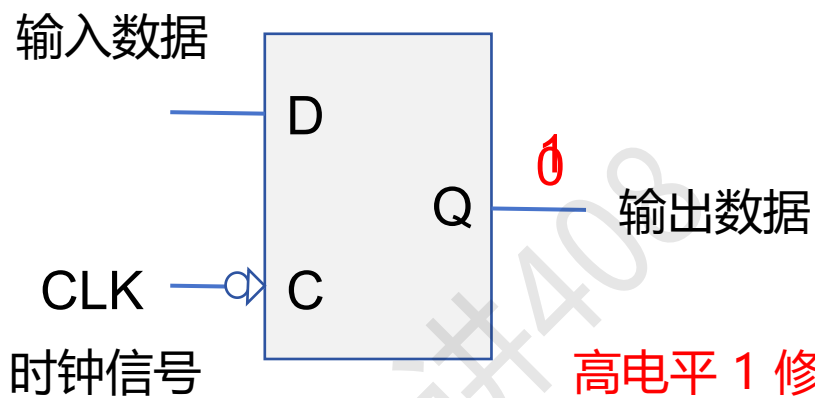


D 触发器

$C = 0$, 读取二进制

$C = 1, D = 1$, 保存二进制 1

$C = 1, D = 0$, 保存二进制 0



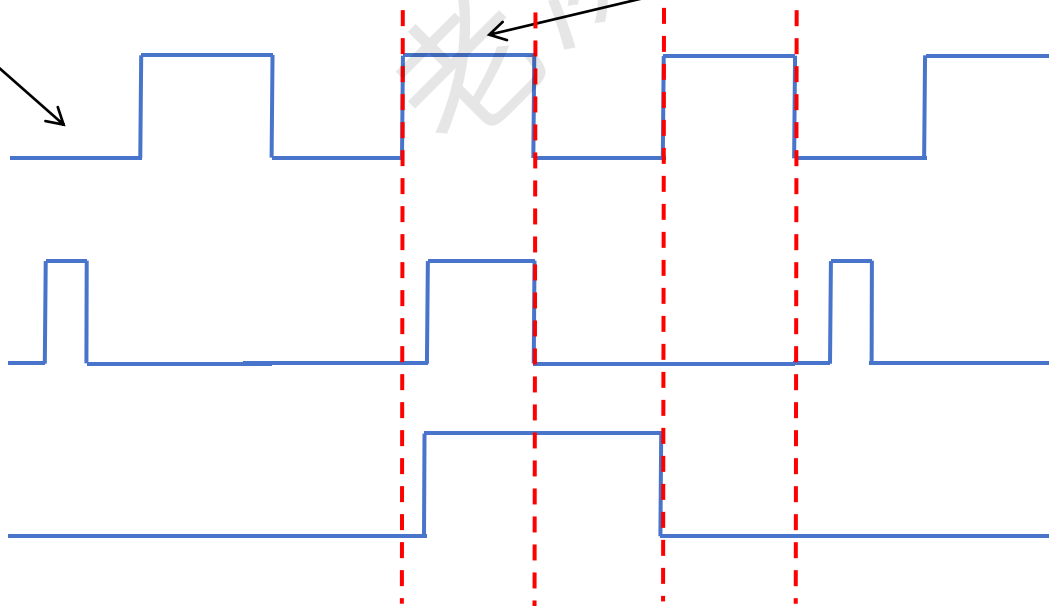
低电平 0 不修改触发器

高电平 1 修改触发器

CLK 时钟信号

数据输入 D

数据输出 Q



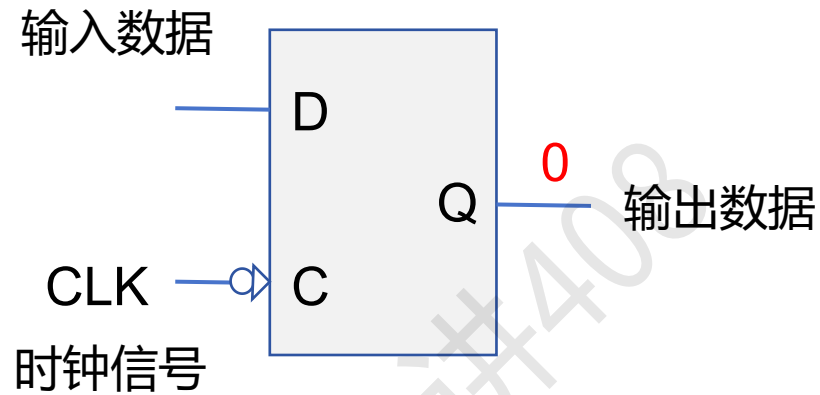
高电平触发修改触发器状态

D 触发器

$C = 0$, 读取二进制

$C = 1, D = 1$, 保存二进制 1

$C = 1, D = 0$, 保存二进制 0

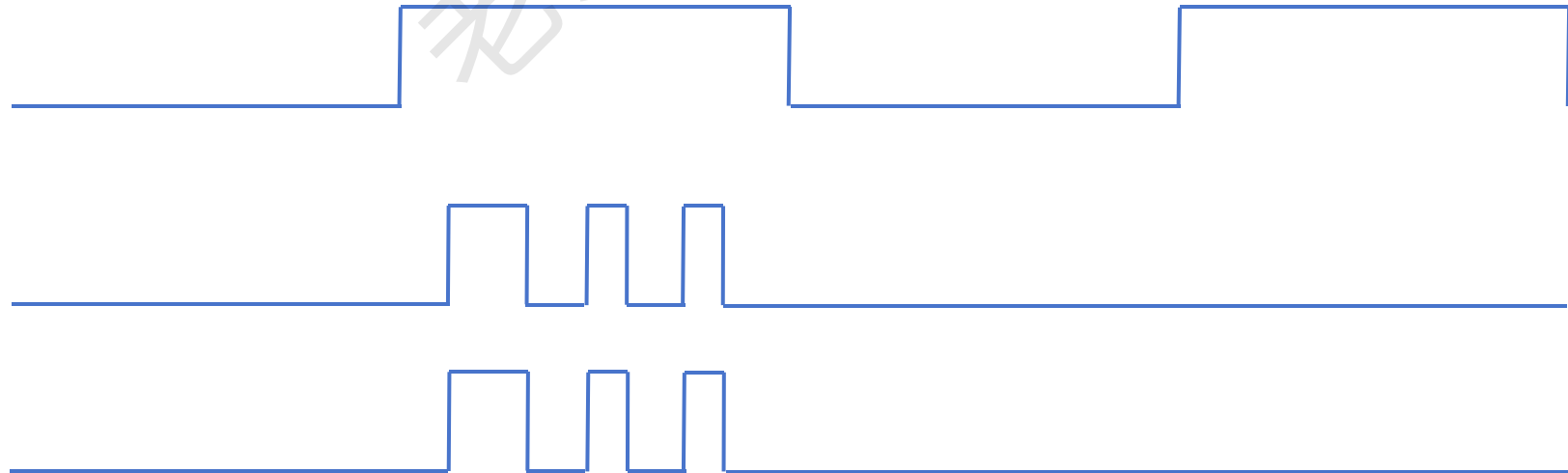


触发器的空翻

CLK 时钟信号

数据输入 D

数据输出 Q

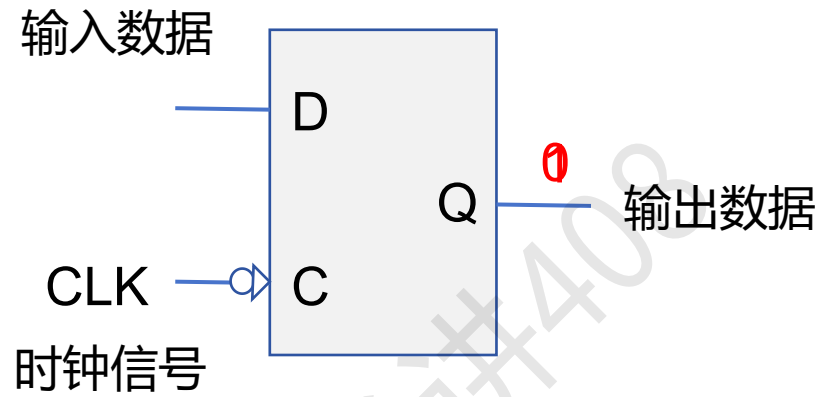


D 触发器

$C = 0$, 读取二进制

$C = 1, D = 1$, 保存二进制 1

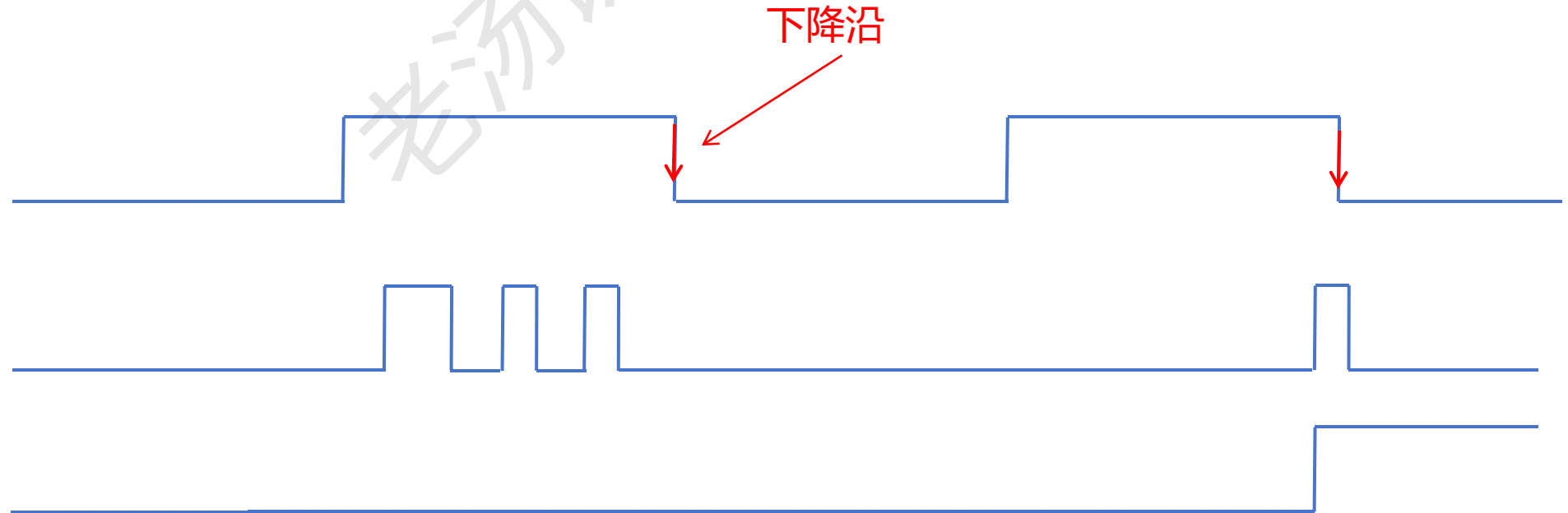
$C = 1, D = 0$, 保存二进制 0



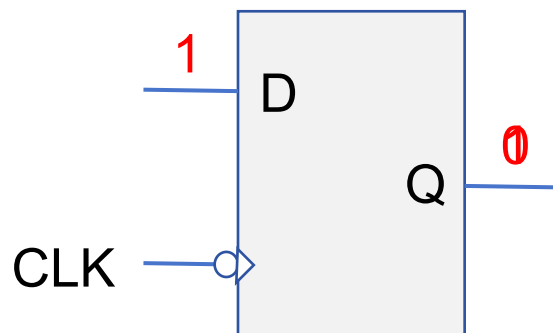
CLK 时钟信号

数据输入 D

数据输出 Q

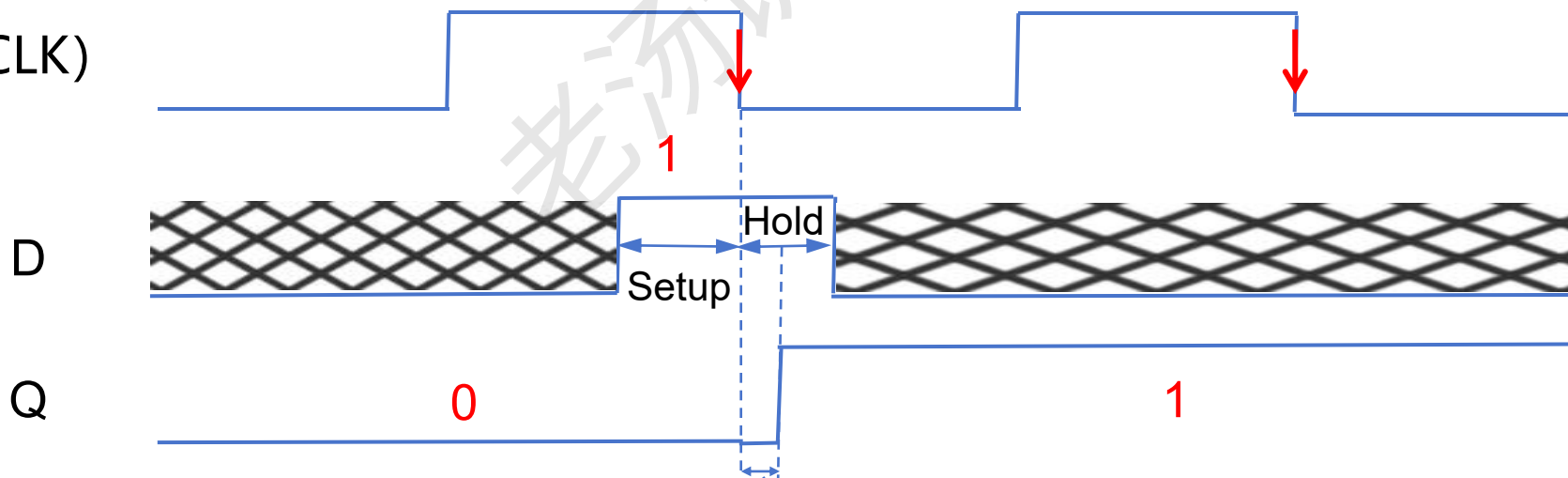


边沿 D 触发器



CLK: CLOCK 的缩写, 表示时钟信号

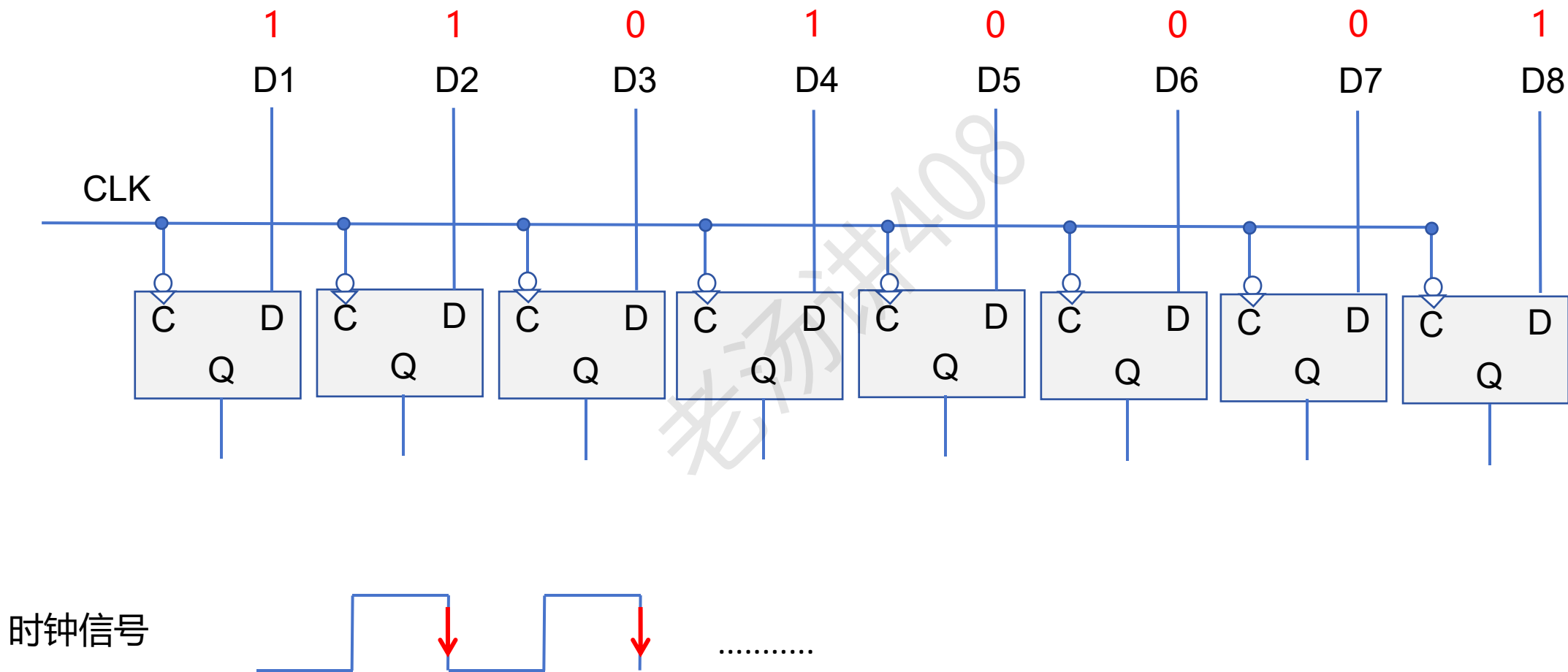
时钟信号 (CLK)



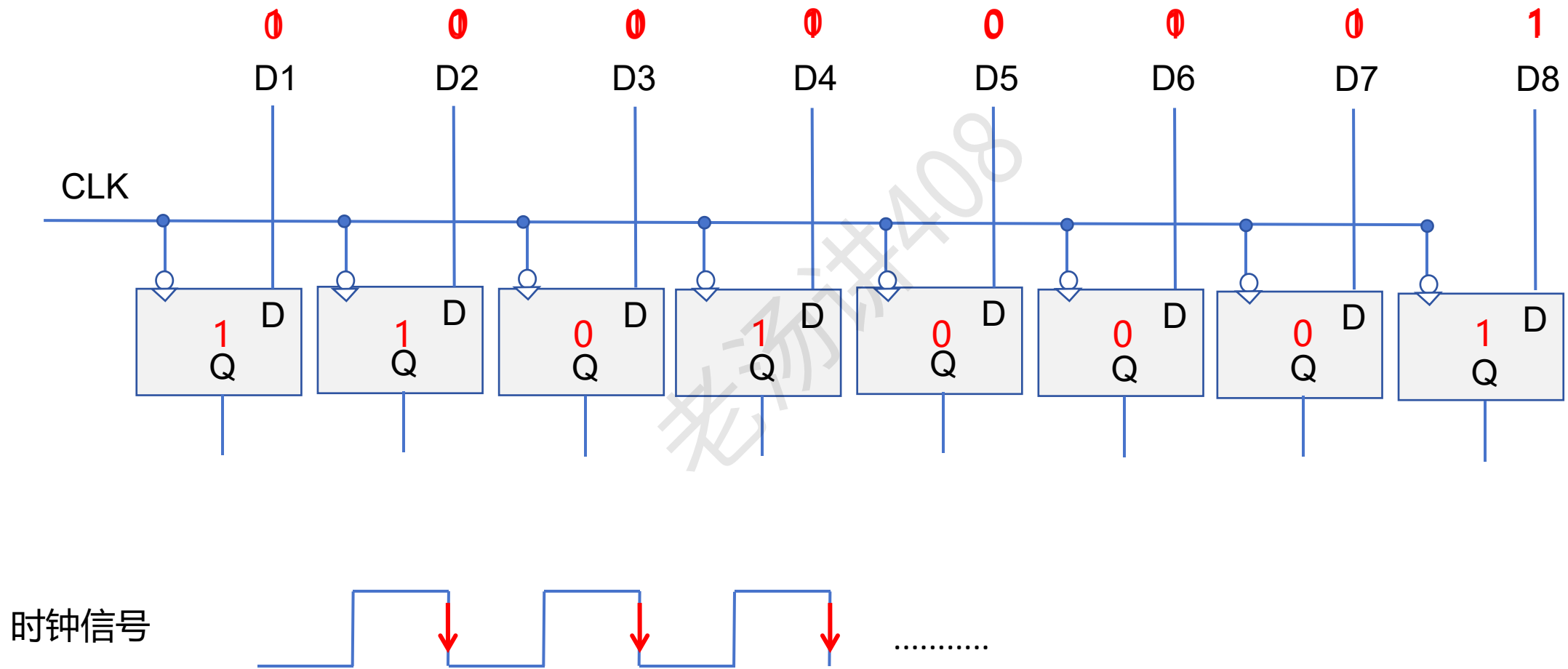
Setup: 建立时间 Hold: 保持时间

延迟时间 (Clock-to-Q 时间)

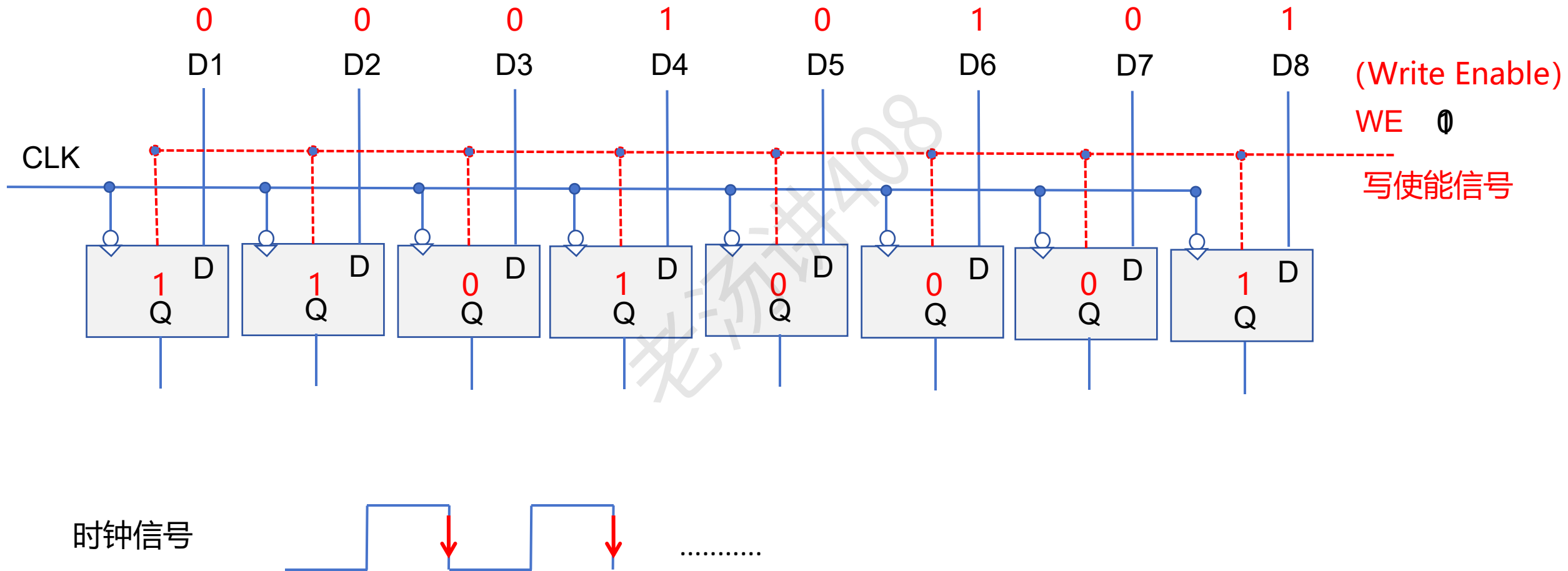
寄存器：边沿 D 触发器组成



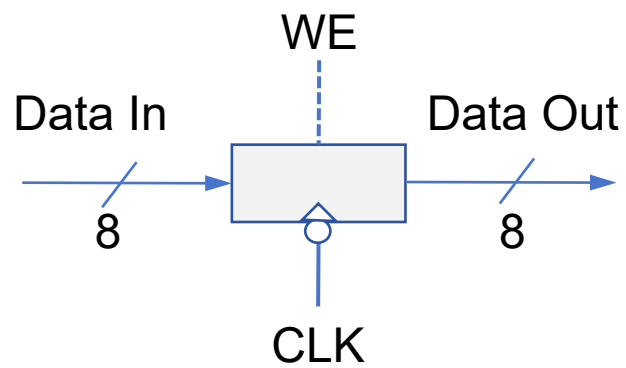
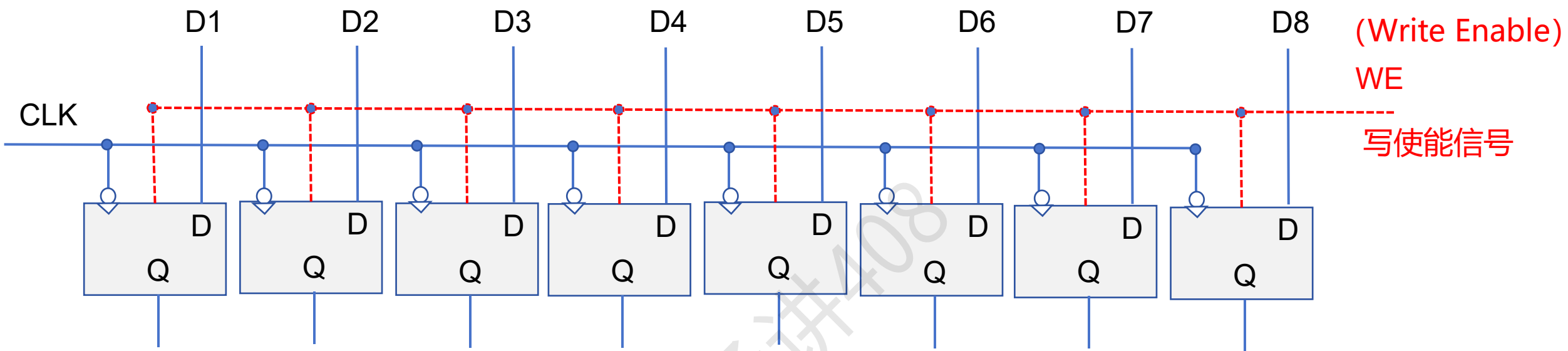
寄存器：边沿 D 触发器组成



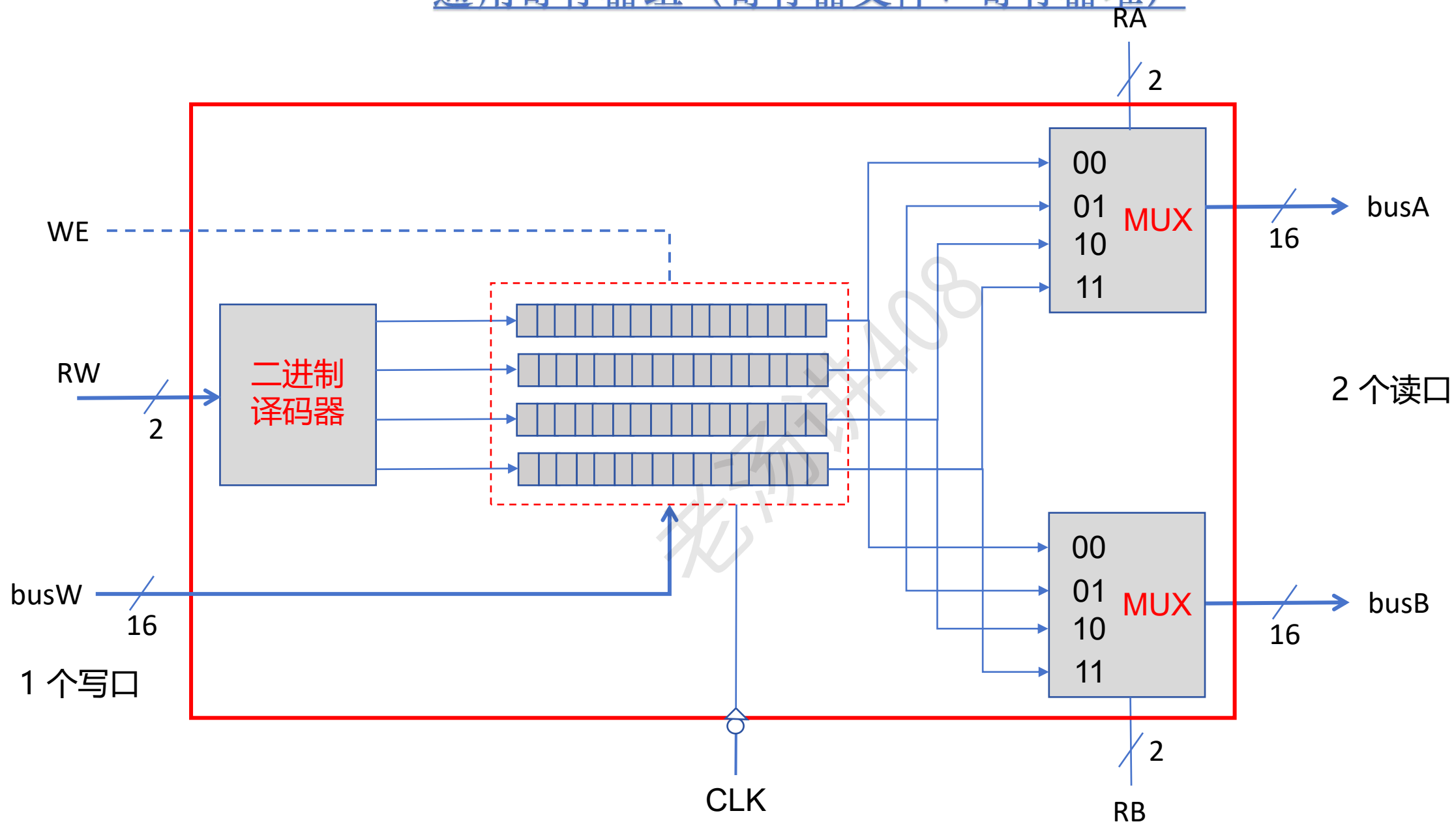
寄存器：边沿 D 触发器组成



暂存寄存器



通用寄存器组（寄存器文件、寄存器堆）



通用寄存器组



重要概念：机器字长

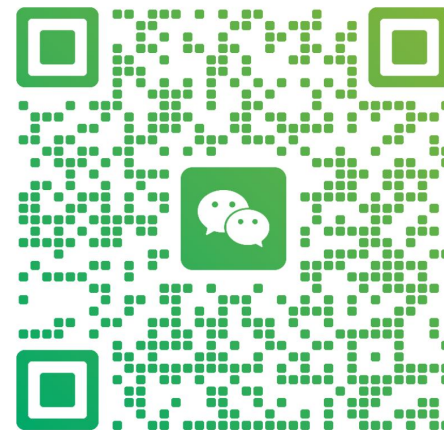
机器字长是指 CPU 一次能处理数据的位数，通常与 CPU 的寄存器位数有关

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



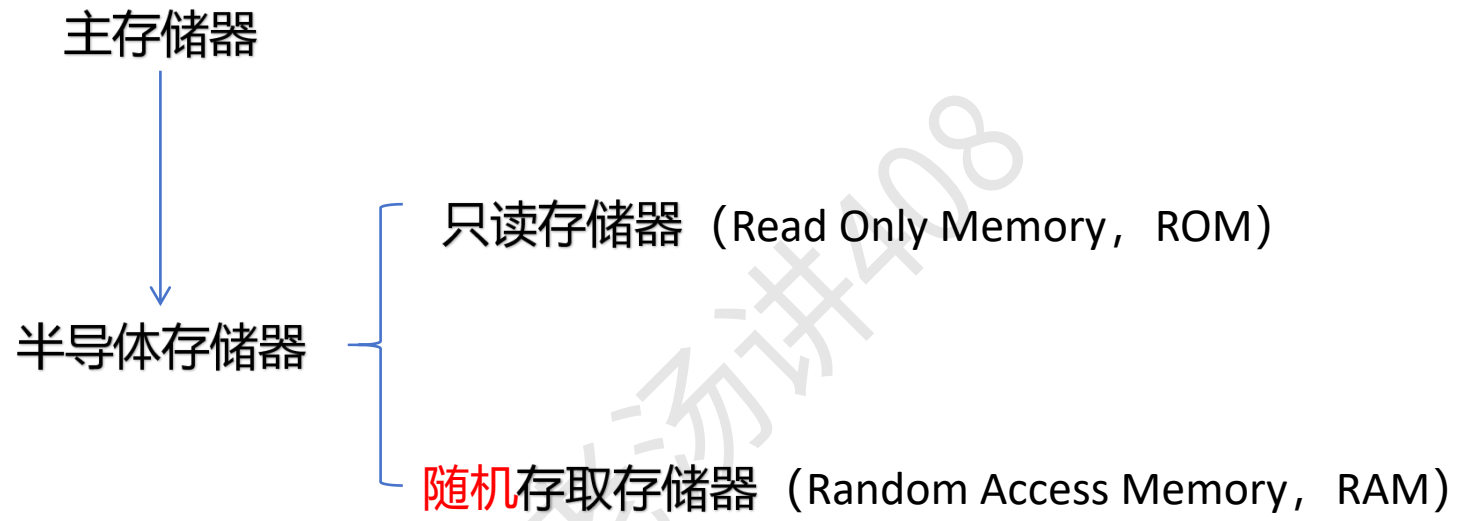
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

主存储器存储原理



存储容量单位

计算机内部，数据大小的转换单位是： $1024 = 2^{10}$

1 字节 = 8 位 $\longrightarrow$ 1 B = 8 bit

1024 字节 = 1 千字节 $\longrightarrow$ 1024 B = 1 KB

1024 千字节 = 1 兆字节 $\longrightarrow$ 1024 KB = 1 MB

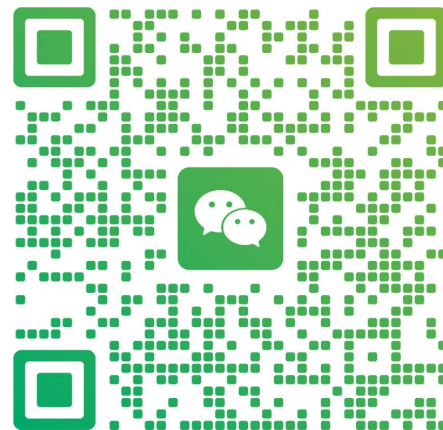
1024 兆字节 = 1 吉字节 $\longrightarrow$ 1024 MB = 1 GB

1024 吉字节 = 1 太字节 $\longrightarrow$ 1024 GB = 1 TB

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

存储容量单位

计算机内部，数据大小的转换单位是： $1024 = 2^{10}$

通信领域，数据大小的转换单位是： $1000 = 10^3$

1 字节 = 8 位 $\longrightarrow$ 1 B = 8 bit

1024 字节 = 1 千字节 $\longrightarrow$ 1024 B = 1 KB

1024 千字节 = 1 兆字节 $\longrightarrow$ 1024 KB = 1 MB

1024 兆字节 = 1 吉字节 $\longrightarrow$ 1024 MB = 1 GB

1024 吉字节 = 1 太字节 $\longrightarrow$ 1024 GB = 1 TB

1T = 1000G 1G = 1000M 1M = 1000K 1K = 1000

1TB = 1000GB

1GB = 1000MB

1MB = 1000KB

1KB = 1000B = 1000 * 8bit

1Gbps = 1000Mbps

1Mbps = 1000Kbps

1Kbps = 1000bps

bps: bit per second

1G B/s = 1000M B/s

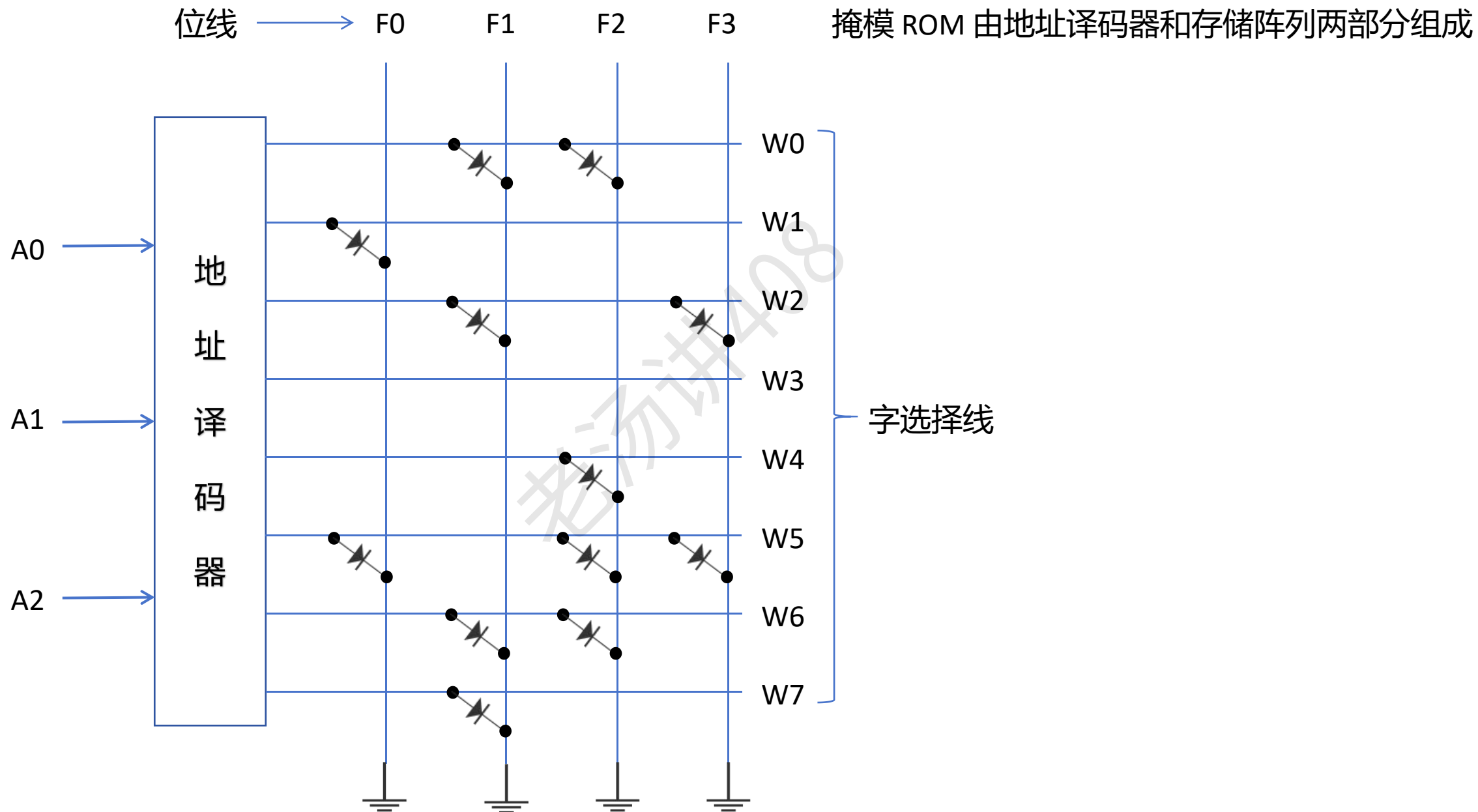
1M B/s = 1000K B/s

1K B/s = 1000 B/s = 1000 * 8 bit/s

只读存储器 (ROM)



掩膜只读存储器 (Mask ROM, MROM)

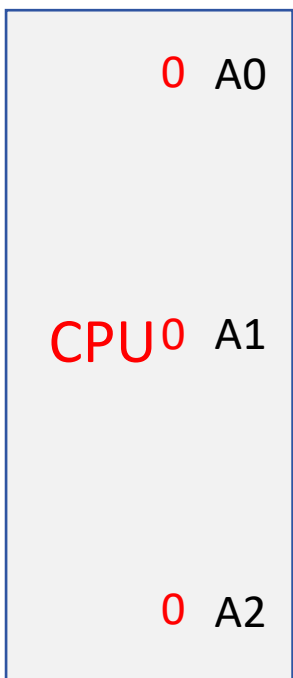


掩膜只读存储器 (Mask ROM, MROM)

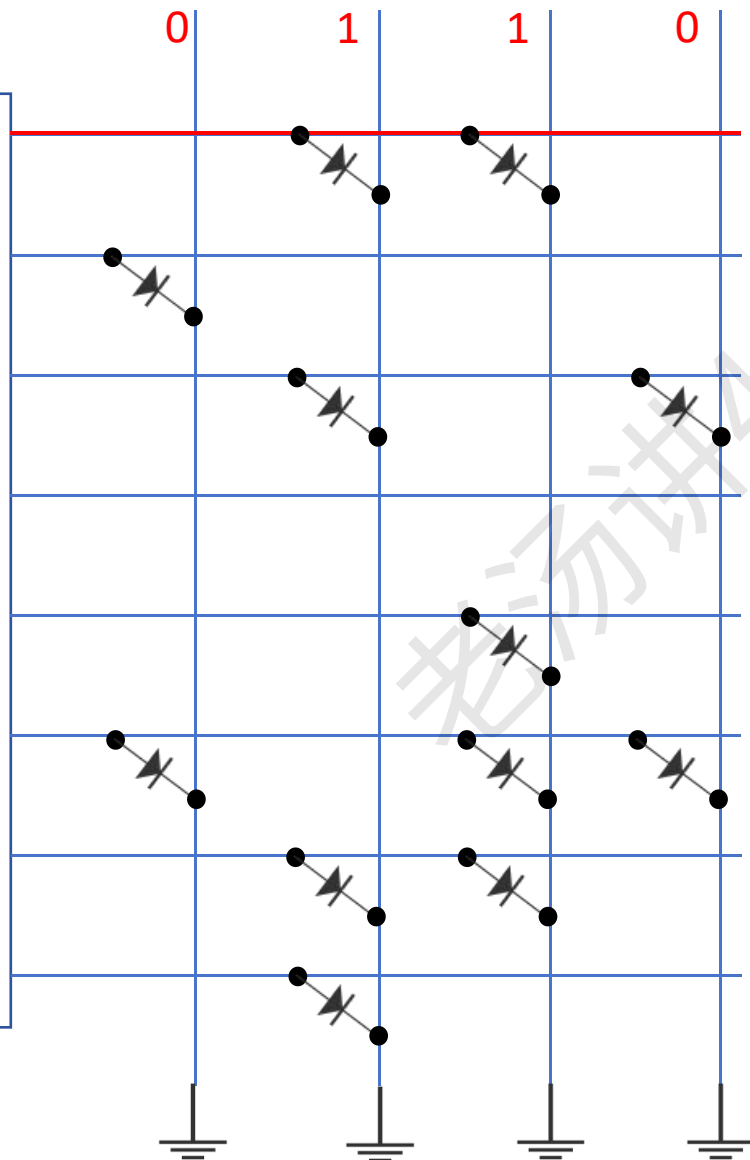
位线 → F0 F1 F2 F3

掩模 ROM 由地址译码器和存储阵列两部分组成

地址线



地址译码器



字选择线

地址

A2 A1 A0

0 0 0

数据输出

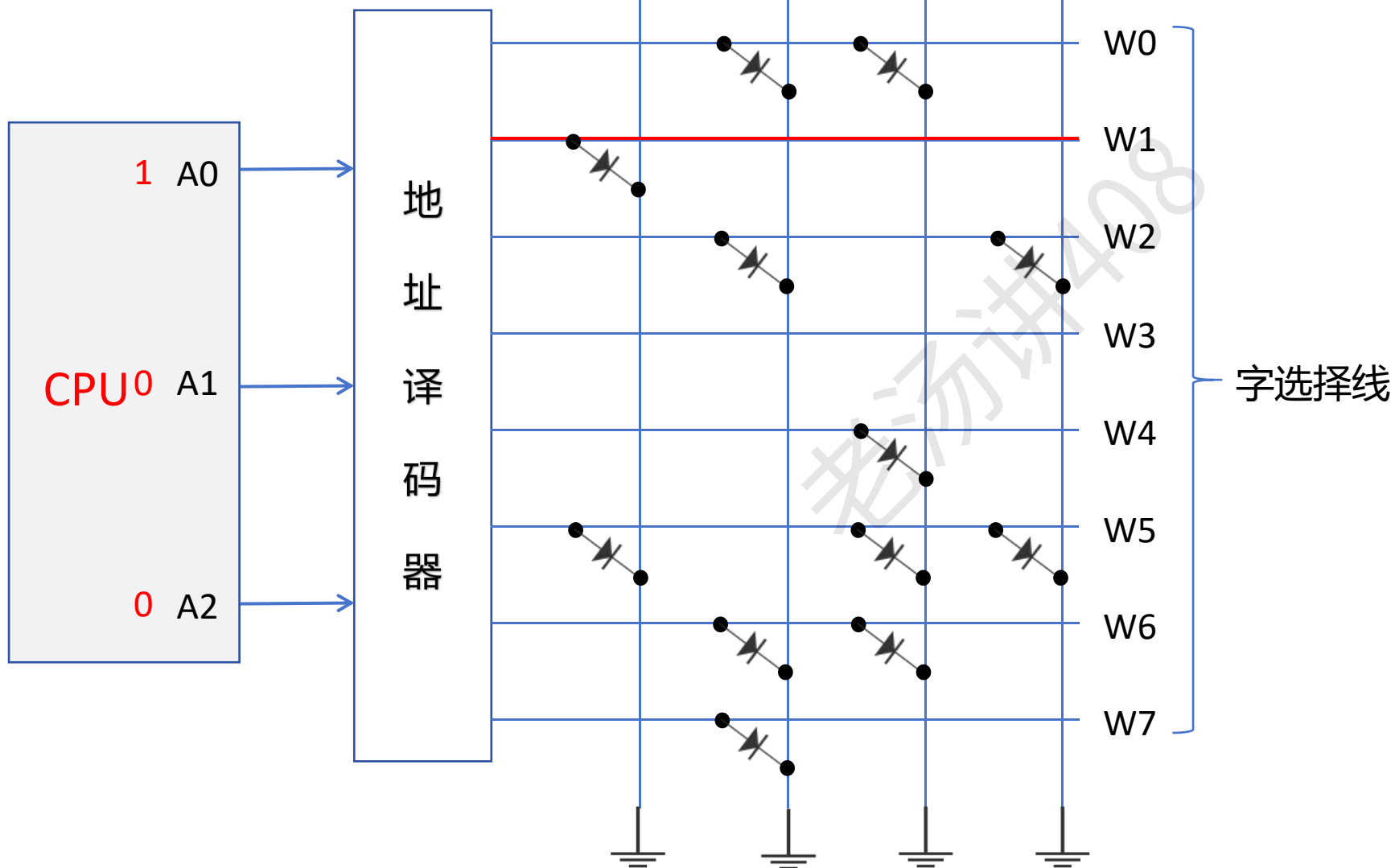
F0 F1 F2 F3

0 1 1 0

掩膜只读存储器 (Mask ROM, MROM)

位线 → F0 F1 F2 F3

掩模 ROM 由地址译码器和存储阵列两部分组成

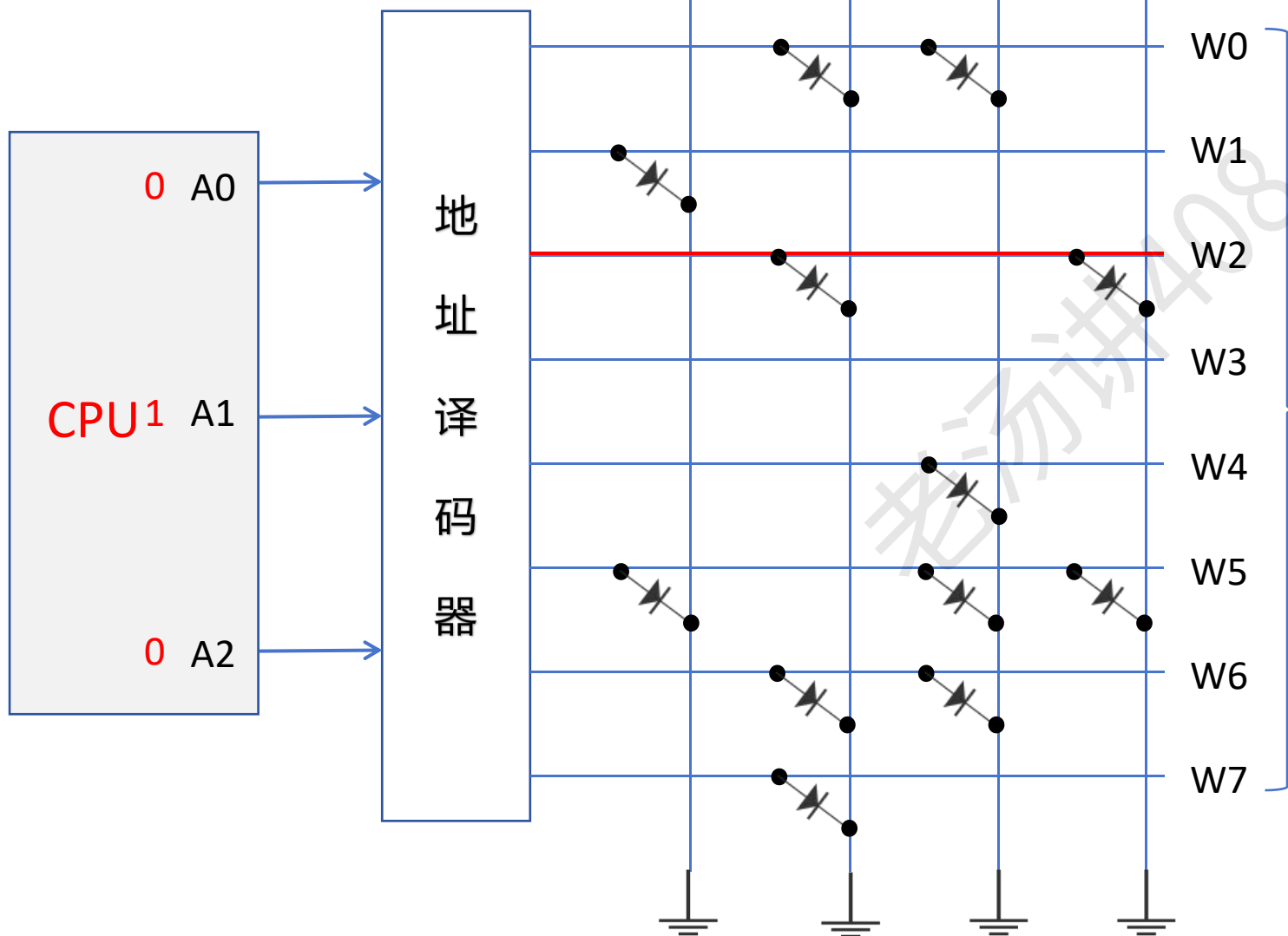


地址			数据输出			
A2	A1	A0	F0	F1	F2	F3
0	0	0	0	1	1	0
0	0	1	1	0	0	0

掩膜只读存储器 (Mask ROM, MROM)

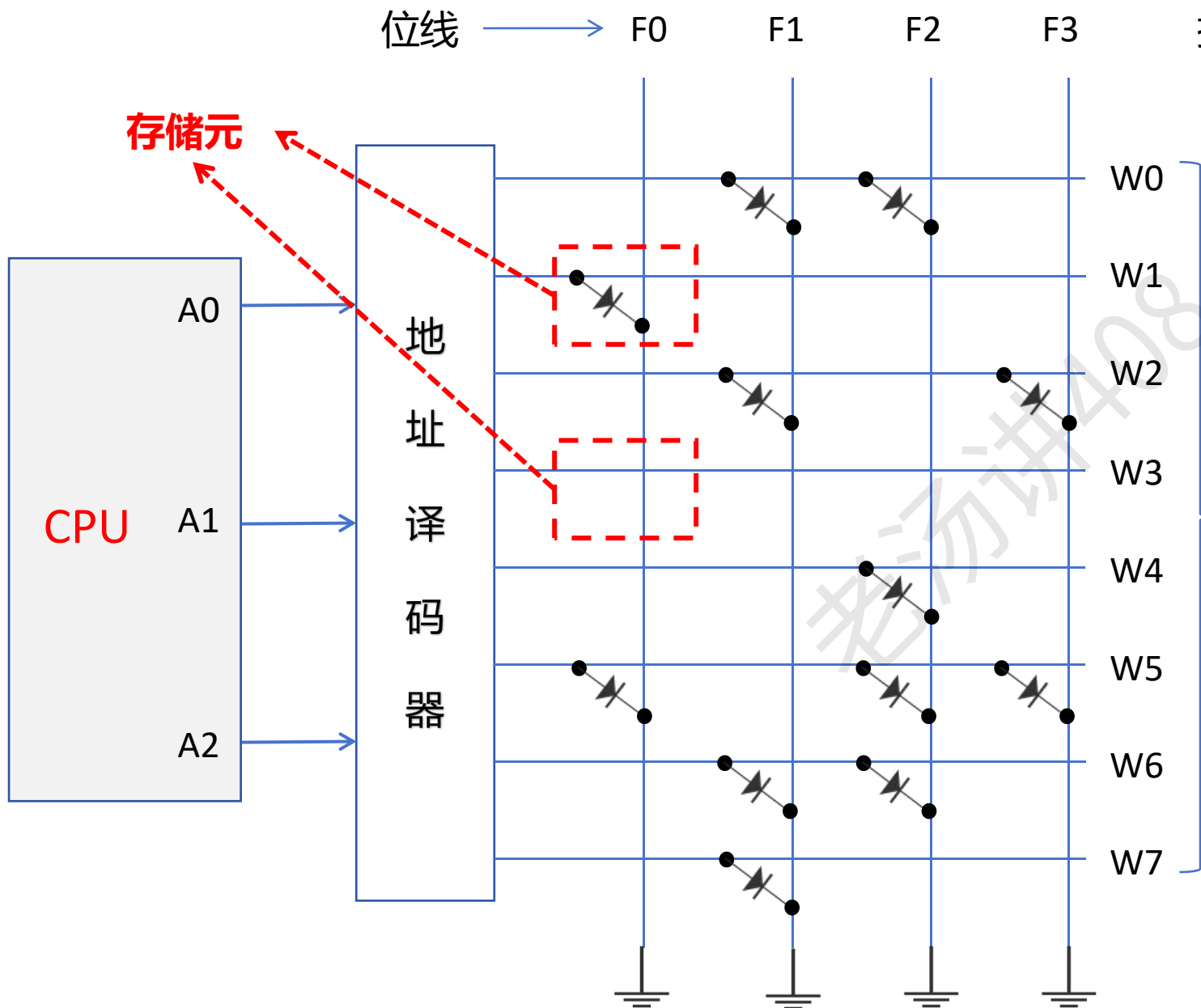
位线 → F0 F1 F2 F3

掩模 ROM 由地址译码器和存储阵列两部分组成



地址			数据输出			
A2	A1	A0	F0	F1	F2	F3
0	0	0	0	1	1	0
0	0	1	1	0	0	0
0	1	0	0	1	0	1
0	1	1	0	0	0	0
1	0	0	0	0	1	0
1	0	1	1	0	1	1
1	1	0	0	1	1	0
1	1	1	0	1	0	0

掩膜只读存储器 (Mask ROM, MROM)



掩模 ROM 由地址译码器和存储阵列两部分组成

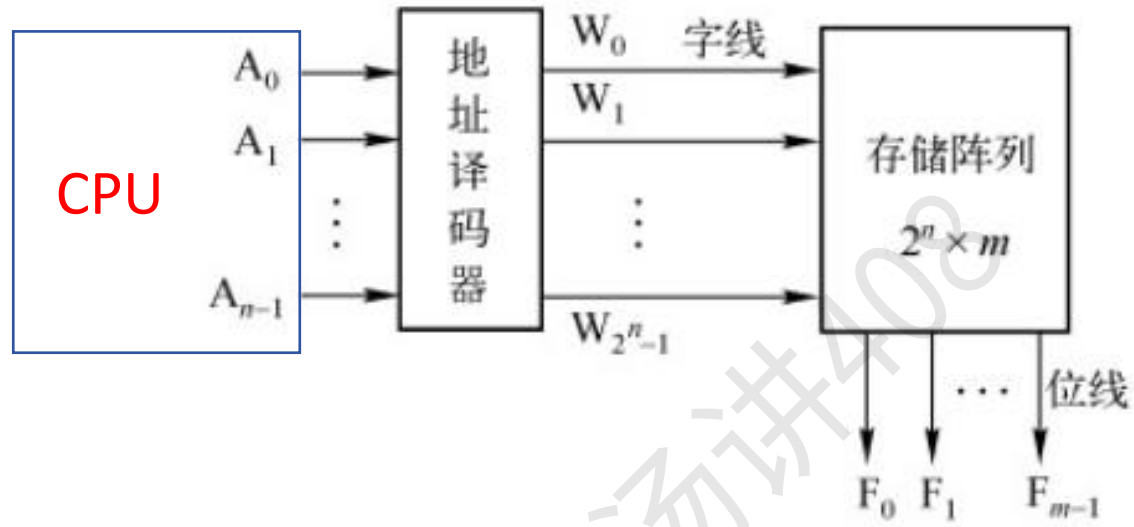
地址			数据输出			
A2	A1	A0	F0	F1	F2	F3
0	0	0	0	1	1	0
0	0	1	1	0	0	0
0	1	0	0	1	0	1
0	1	1	0	0	0	0
1	0	0	0	0	1	0
1	0	1	1	0	1	1
1	1	0	0	1	1	0
1	1	1	0	1	0	0

字选择线

3 根地址线, 有 $2^3 = 8$ 个地址, 每个地址存放 4 位二进制, 总容量 $8 * 4 = 32$ bit

一次性读出来的 4 位二进制称为一个**存储字**, 而**存储字长**就是 4 **存储单元**

掩膜只读存储器 (Mask ROM, MROM)



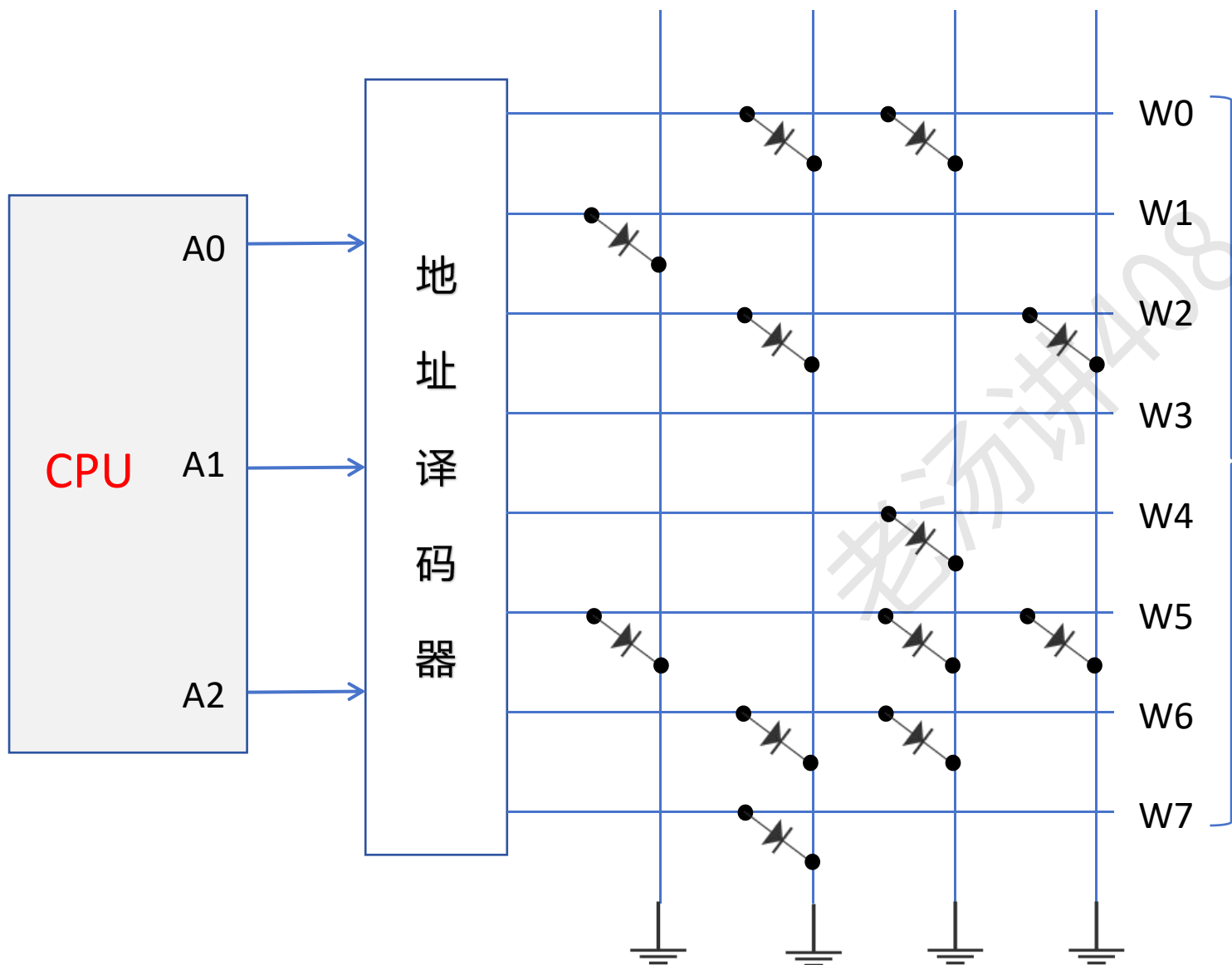
$$n = 10, m = 8$$

$$2^{10} \times 8 = 1K \times 8 = 1KB$$

掩膜只读存储器 (Mask ROM, MROM)

位线 → F0 F1 F2 F3

掩模 ROM 由地址译码器和存储阵列两部分组成



字选择线

地址			数据输出			
A2	A1	A0	F0	F1	F2	F3
0	0	0	0	1	1	0
0	0	1	1	0	0	0
0	1	0	0	1	0	1
0	1	1	0	0	0	0
1	0	0	0	0	1	0
1	0	1	1	0	1	1
1	1	0	0	1	1	0
1	1	1	0	1	0	0

存储信息只读，不能修改

存储内容固定，可靠性高

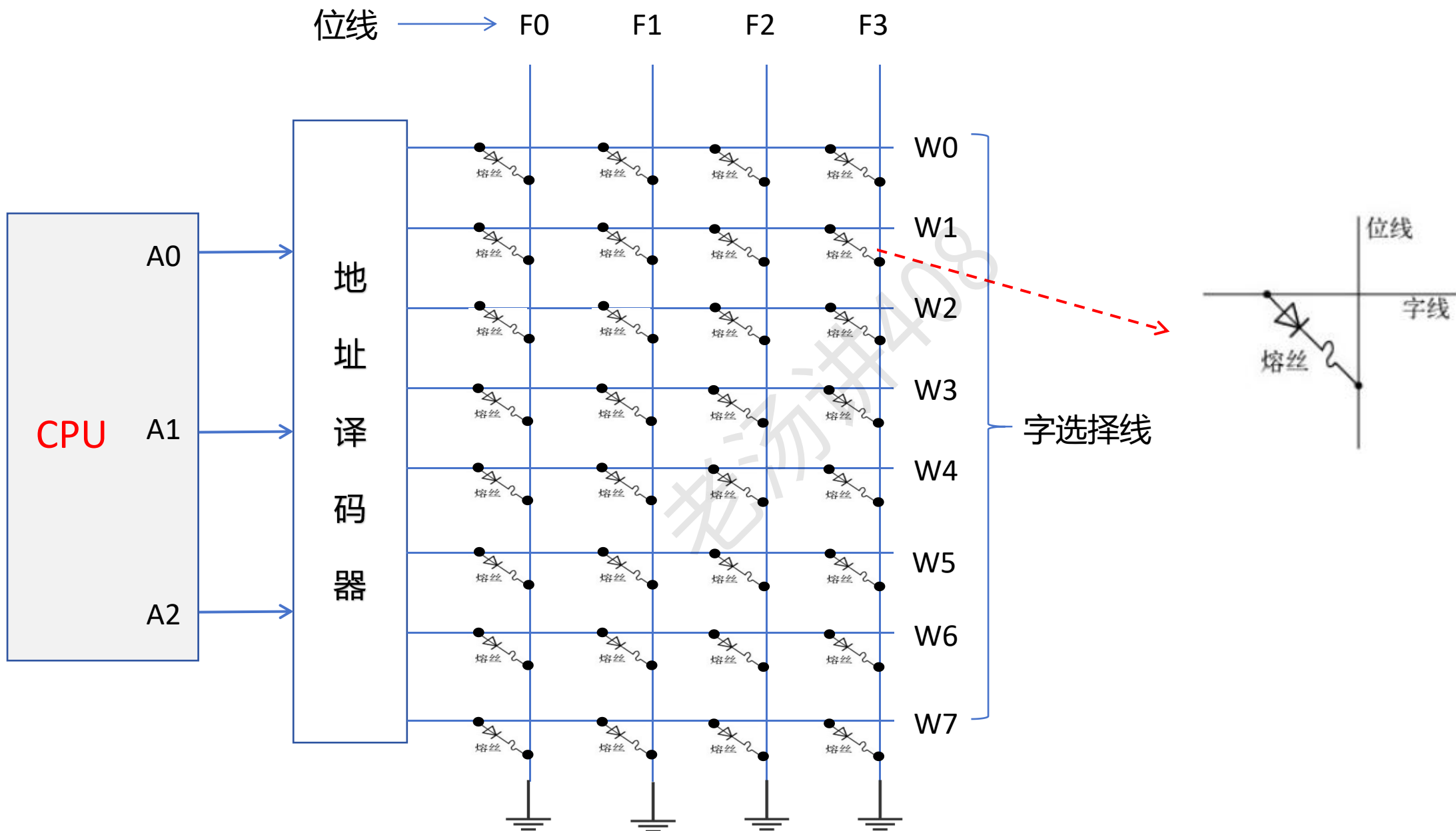
灵活性差，生产周期长

只读存储器 (ROM)

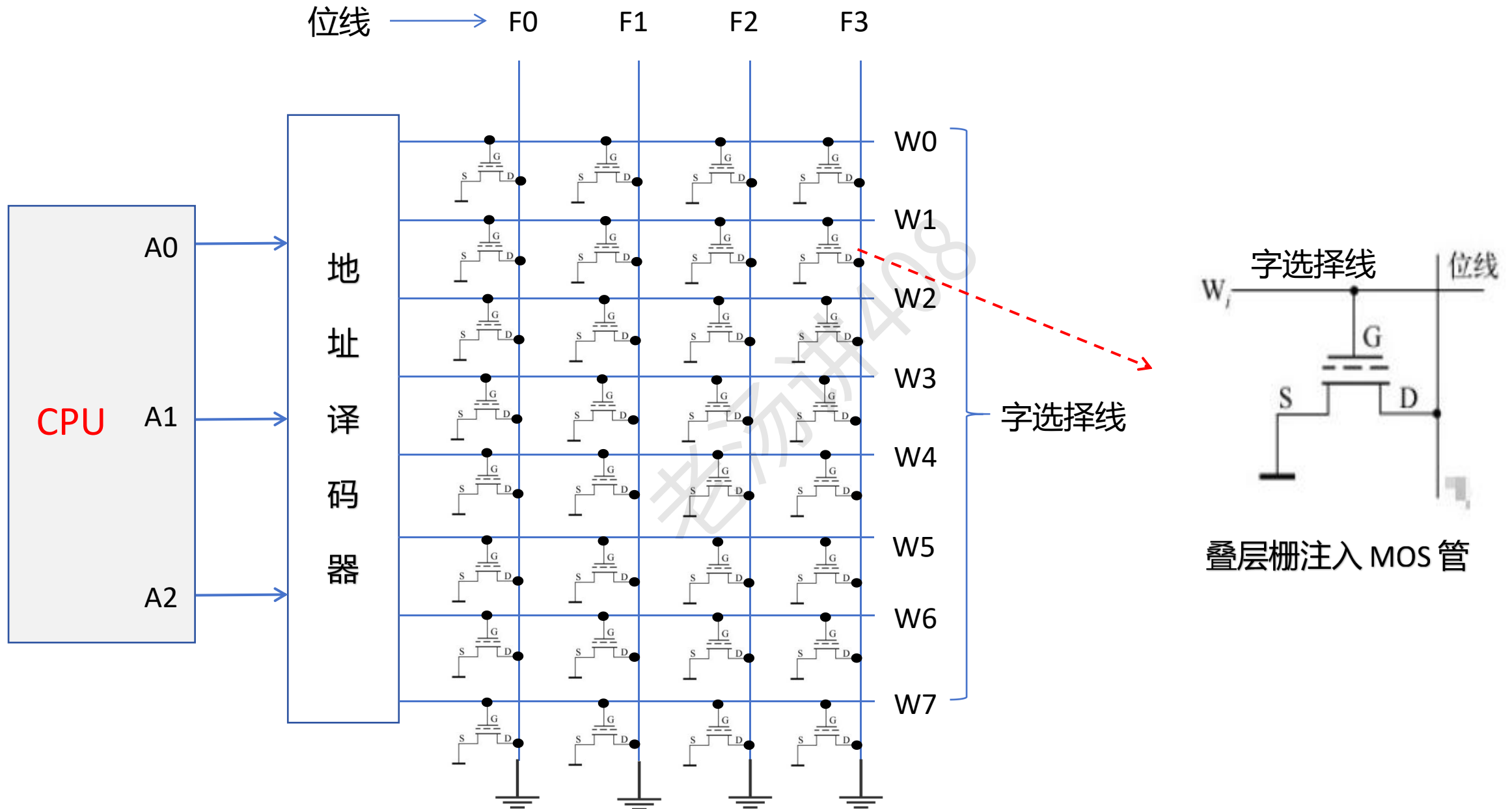


不同点: 字选择线和位线交叉点的电路

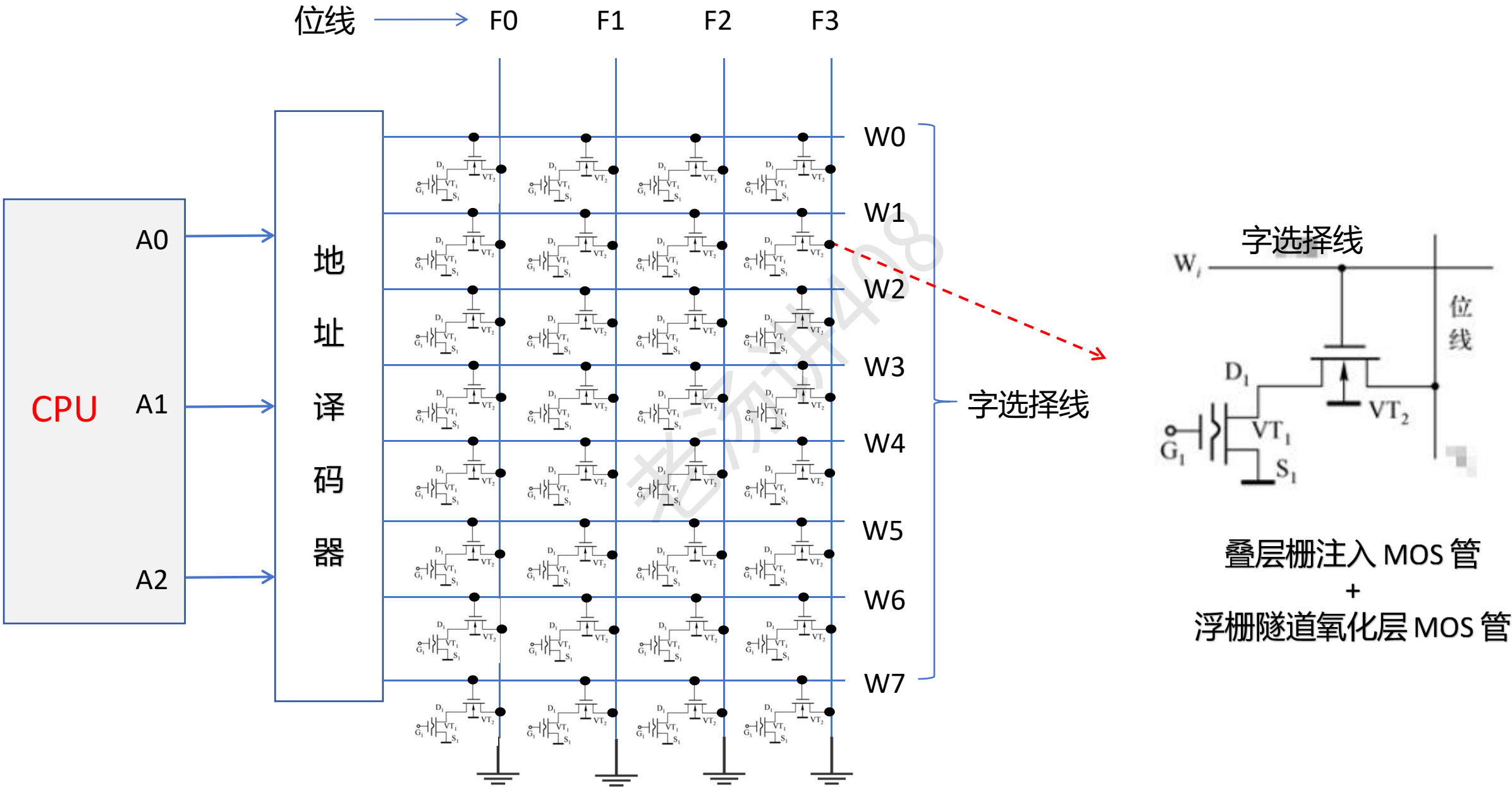
可编程只读存储器 (Programmable ROM, PROM)



可擦除可编程只读存储器 (Erasable Programmable ROM, EPROM)



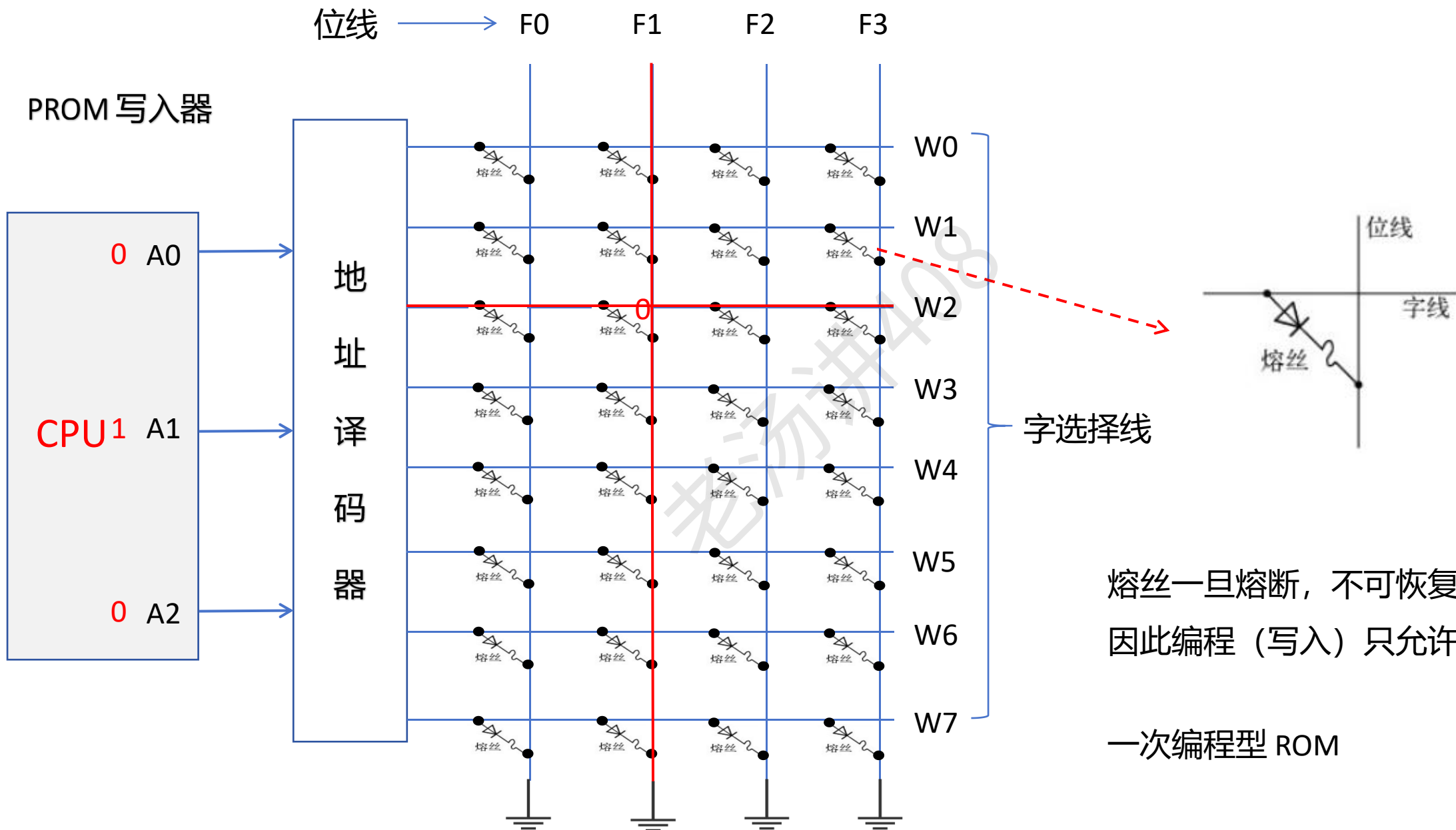
电擦除电改写只读存储器 (Electrically Erasable Programmable ROM, EEPROM)



只读存储器 (ROM)



可编程只读存储器 (Programmable ROM, PROM)



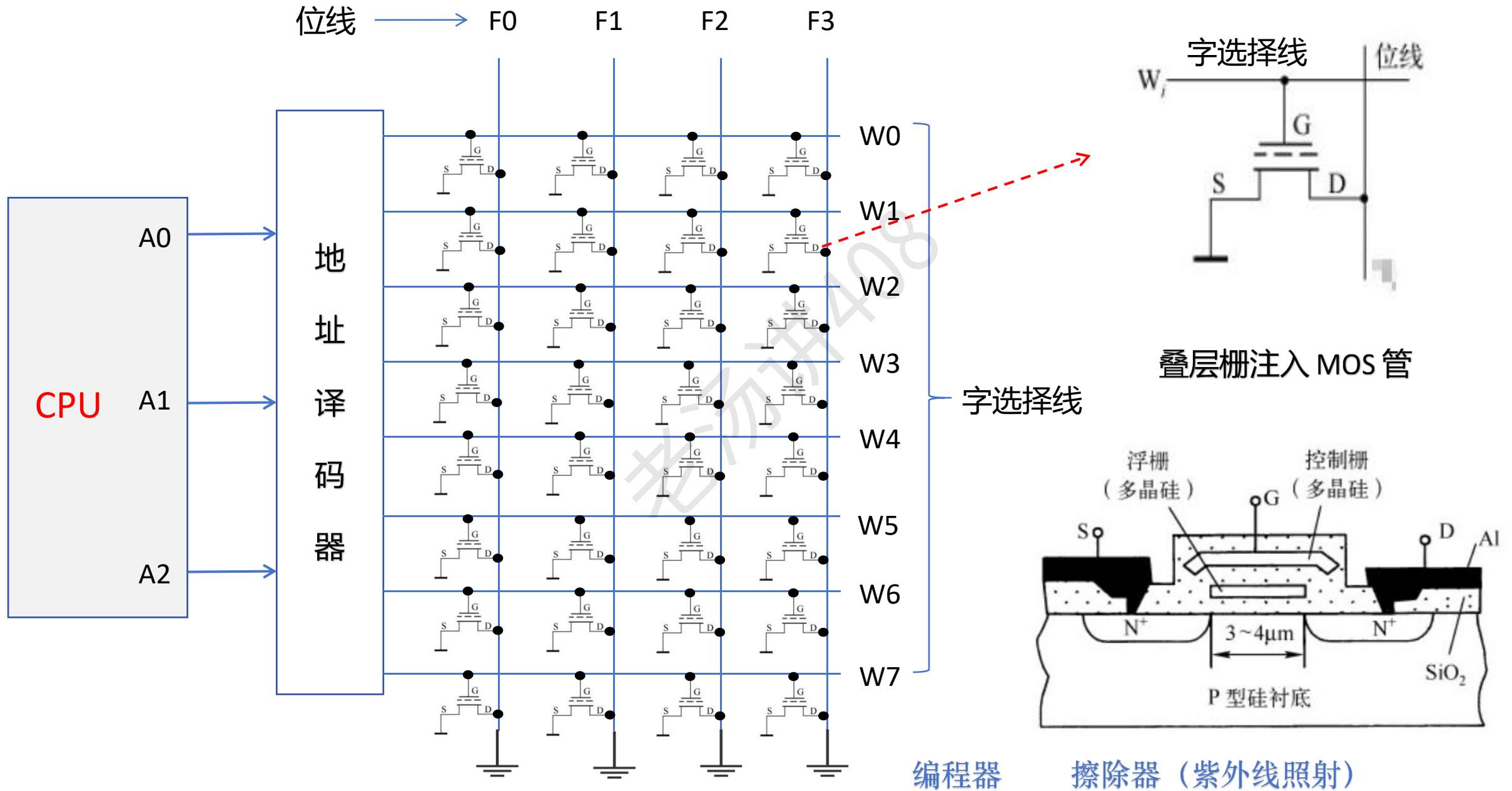
熔丝一旦熔断，不可恢复
因此编程（写入）只允许进行一次

一次编程型 ROM

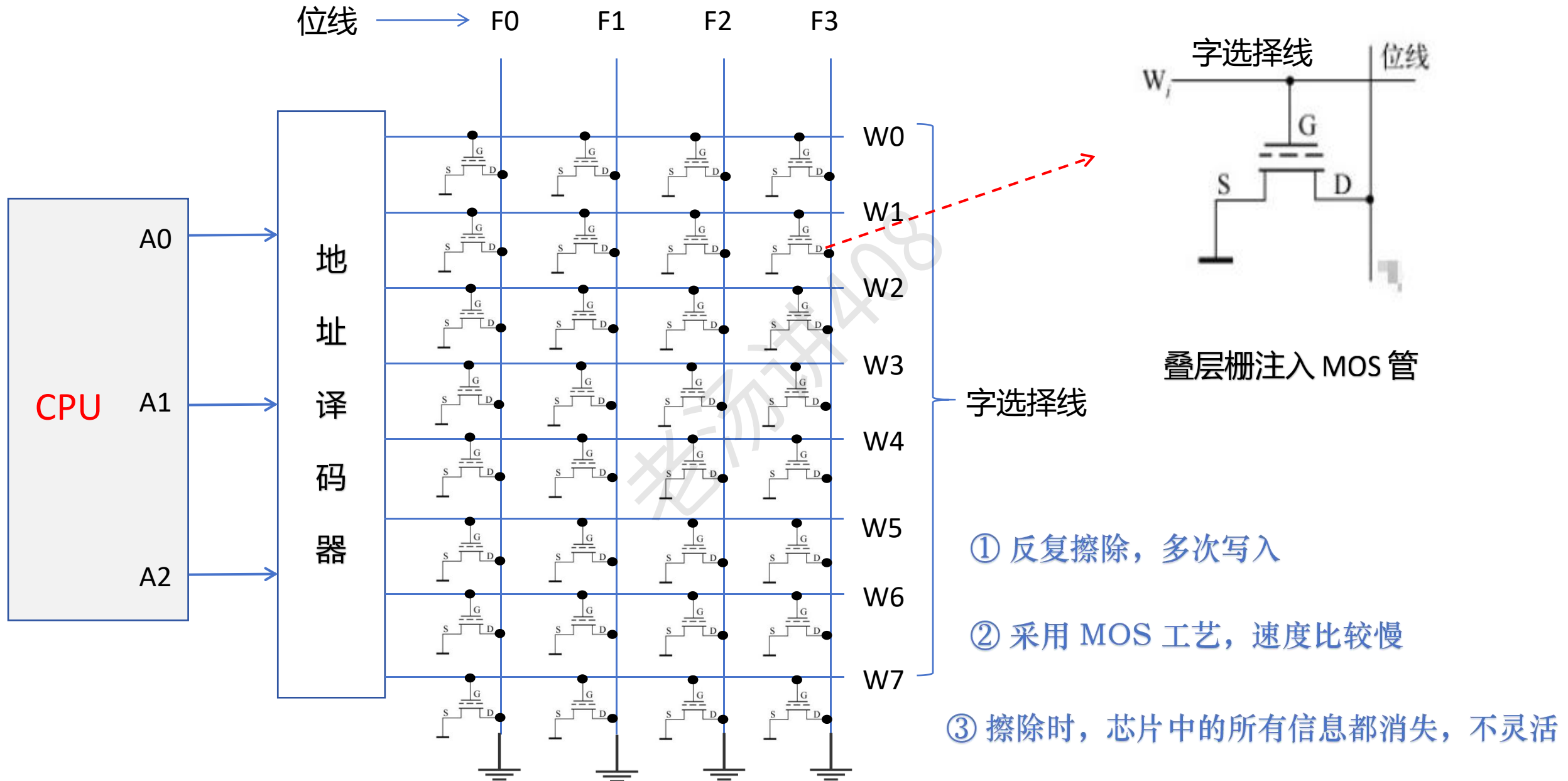
只读存储器 (ROM)



可擦除可编程只读存储器 (Erasable Programmable ROM, EPROM)



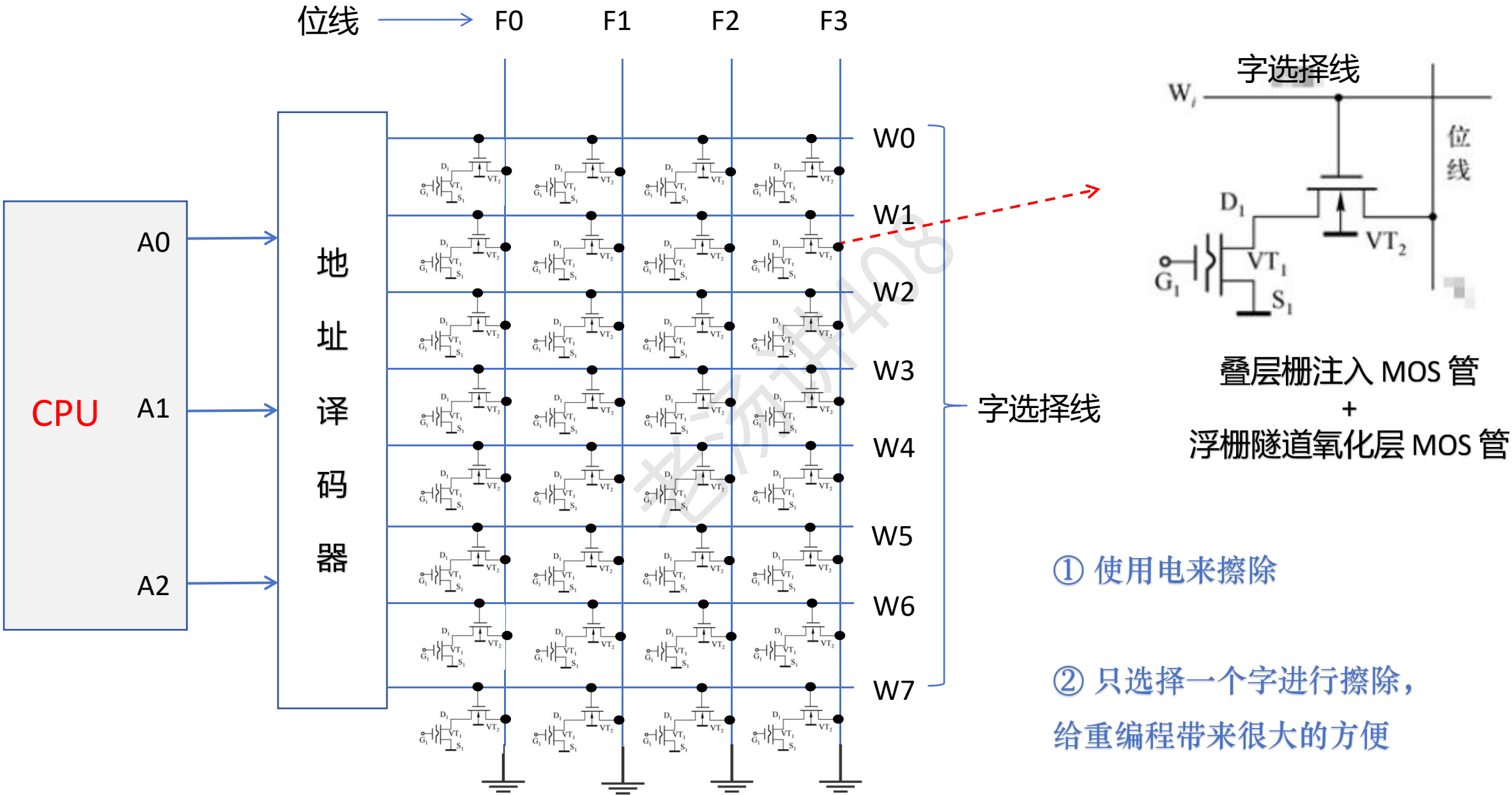
可擦除可编程只读存储器 (Erasable Programmable ROM, EPROM)



只读存储器 (ROM)



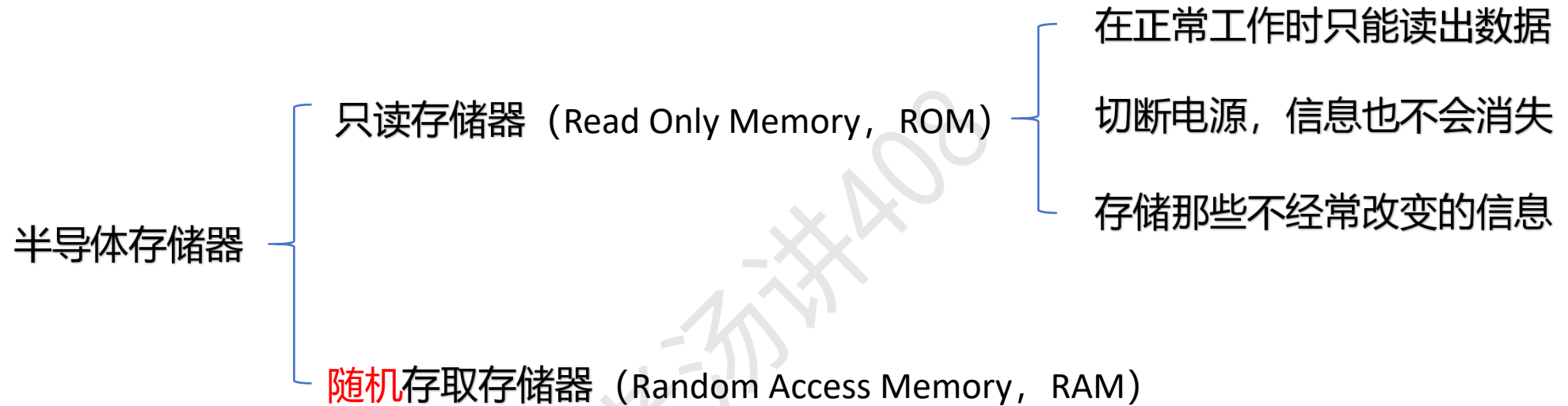
电擦除电改写只读存储器 (Electrically Erasable Programmable ROM, EEPROM)



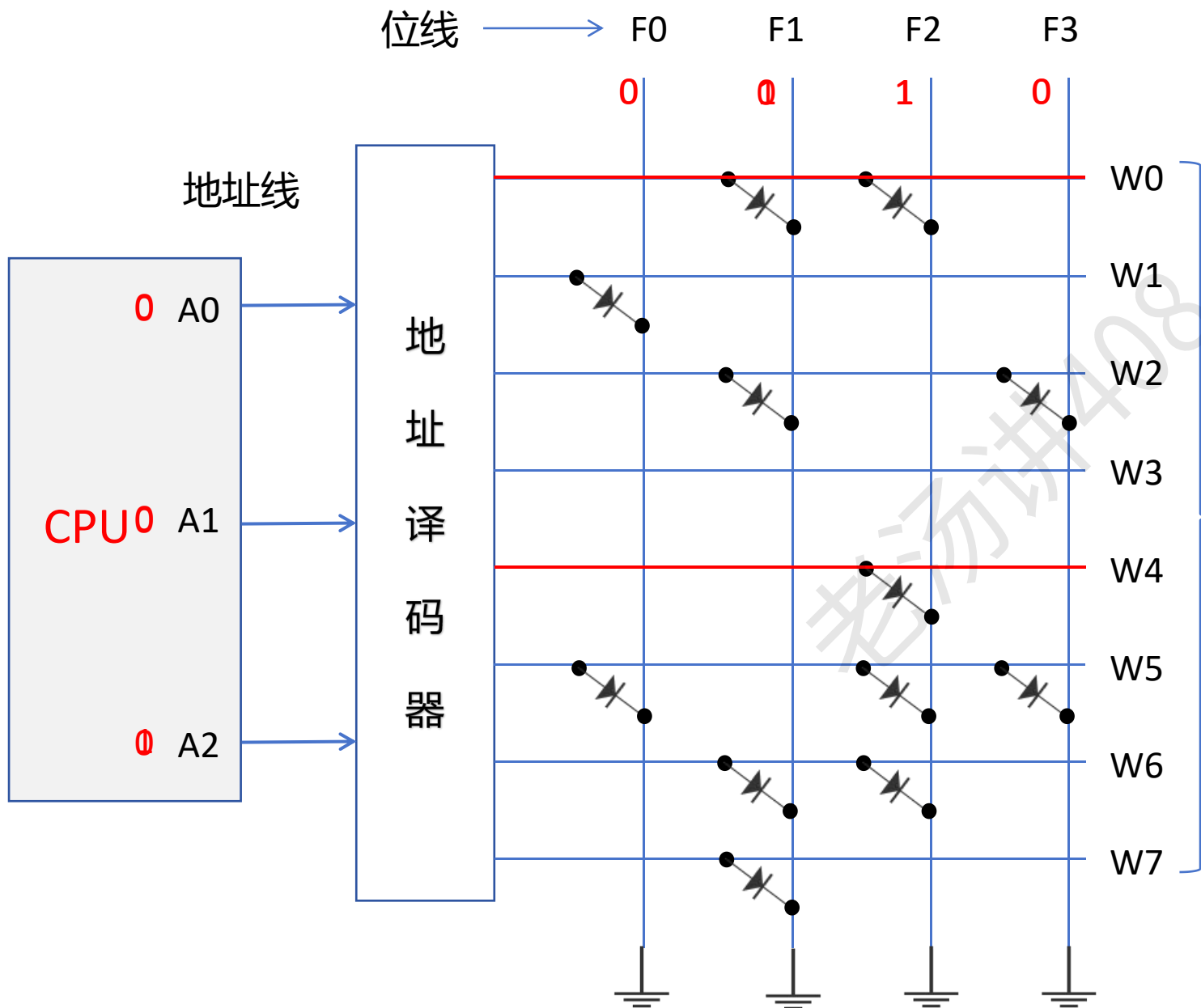
只读存储器 (ROM)



半导体存储器



什么是“随机”？

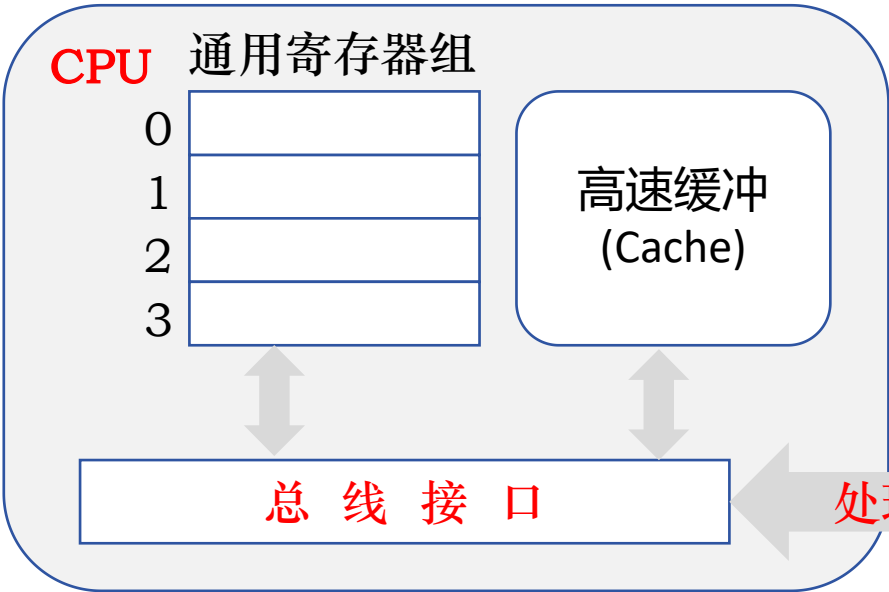


随机的意思就是按地址访问存储单元，因为每个地址译码的时间是一样的，所以，每个单元的访问时间是一个常数，与地址无关

ROM 是随机只读存储器

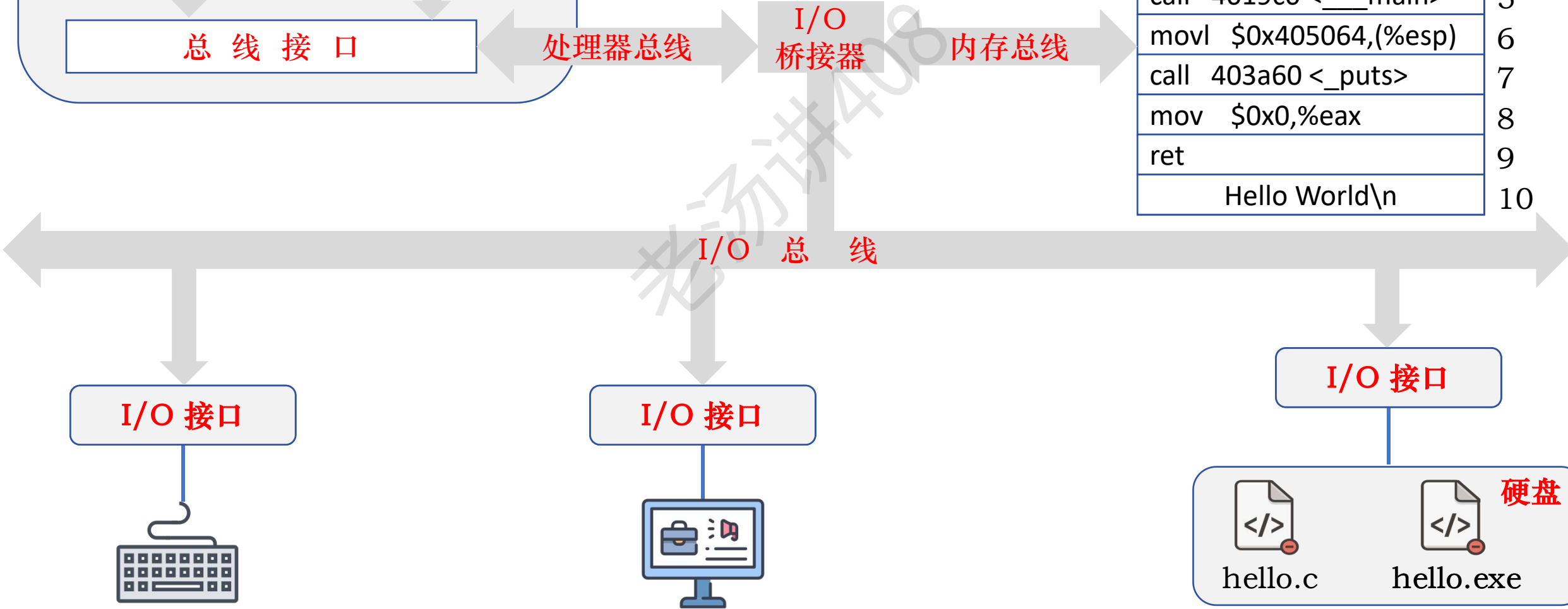
半导体存储器



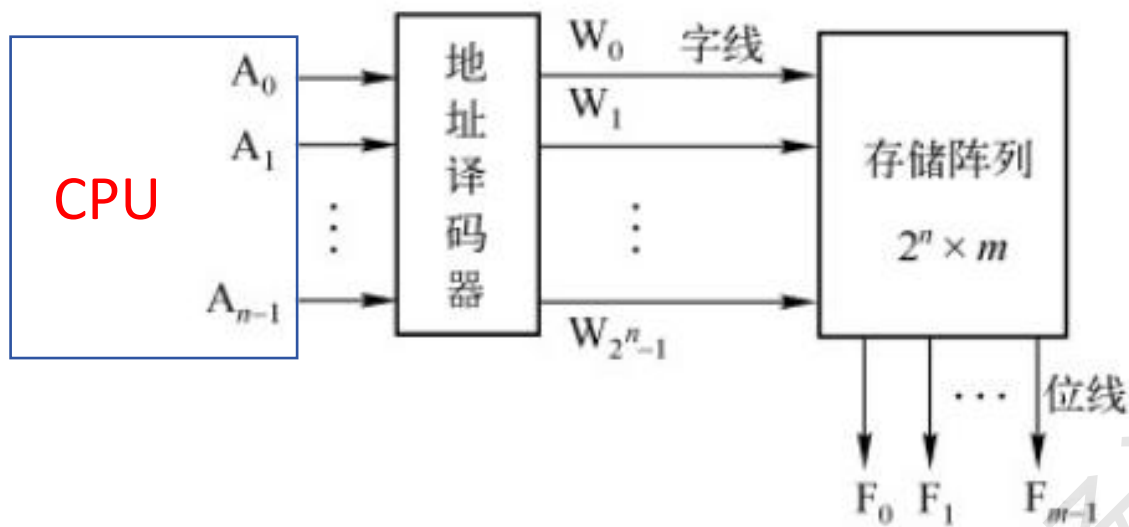


主存储器 (主存、内存)

hello	0
push %ebp	1
mov %esp,%ebp	2
and \$0xffffffff0,%esp	3
sub \$0x10,%esp	4
call 4019c0 <__main>	5
movl \$0x405064,(%esp)	6
call 403a60 <_puts>	7
mov \$0x0,%eax	8
ret	9
Hello World\n	10



主存储器



一个存储字的大小是 8B (64 位)

$$4\text{GB} = 4 * 1024 * 1024 * 1024 \text{ B} / 8\text{B}$$

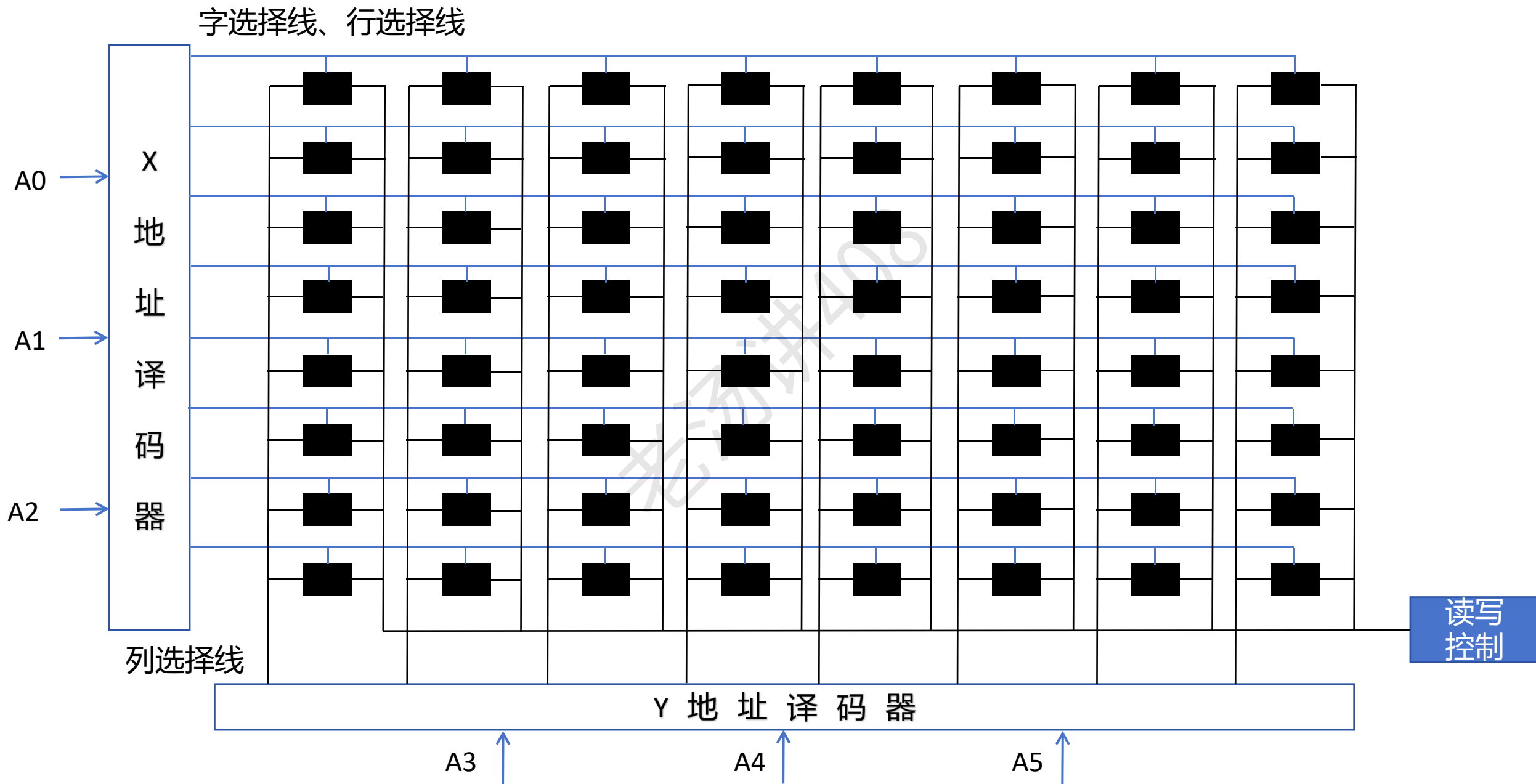
$$= 1024 * 1024 * 512 \text{ 个存储字}$$

$$= 2^{29} \text{ 个存储字}$$

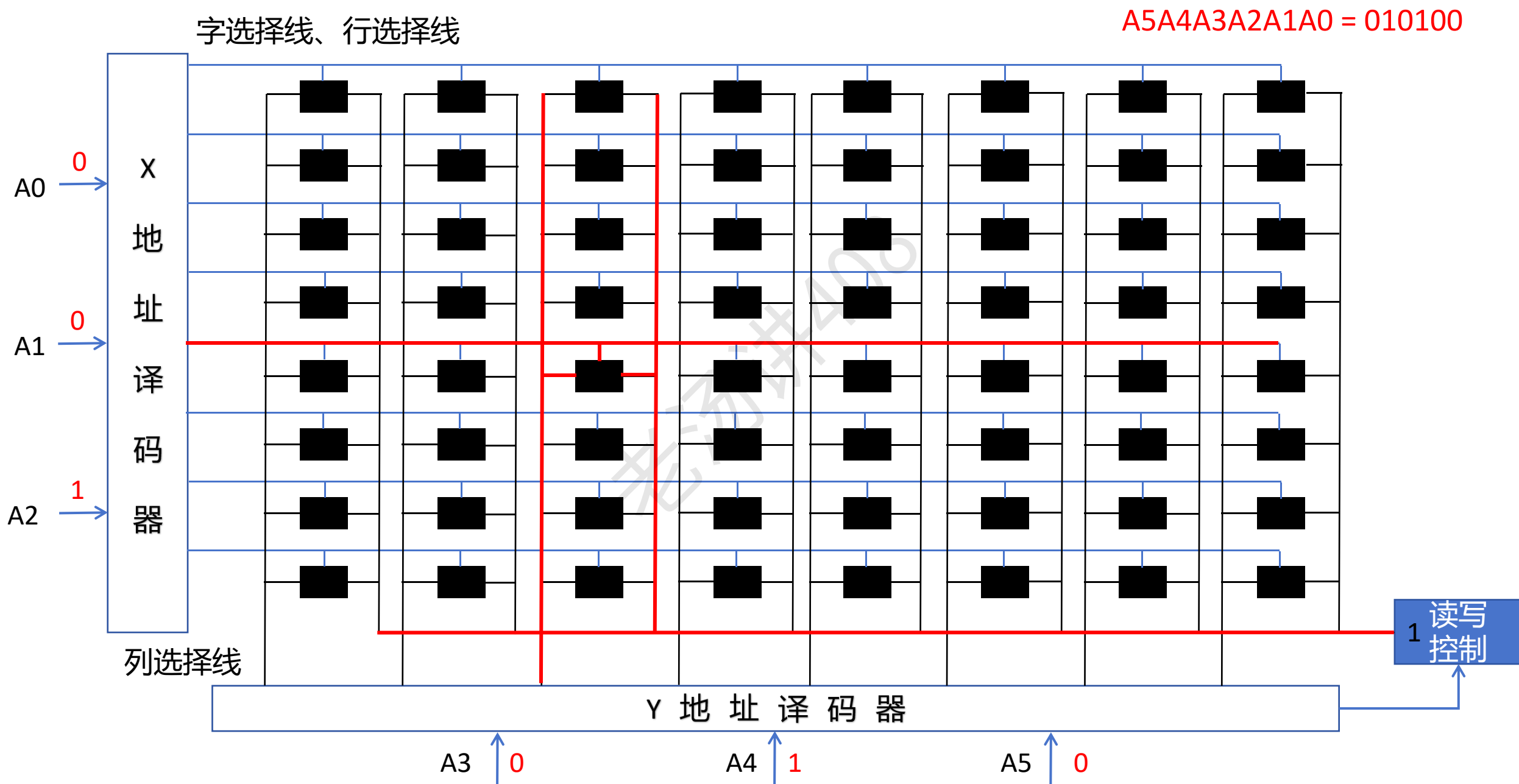
$n = 29, m = 64$ 字线数量为 2^{29} , 太多, 成本高

一维译码, 或者称为线选法、单译码法 适用于小容量的存储器

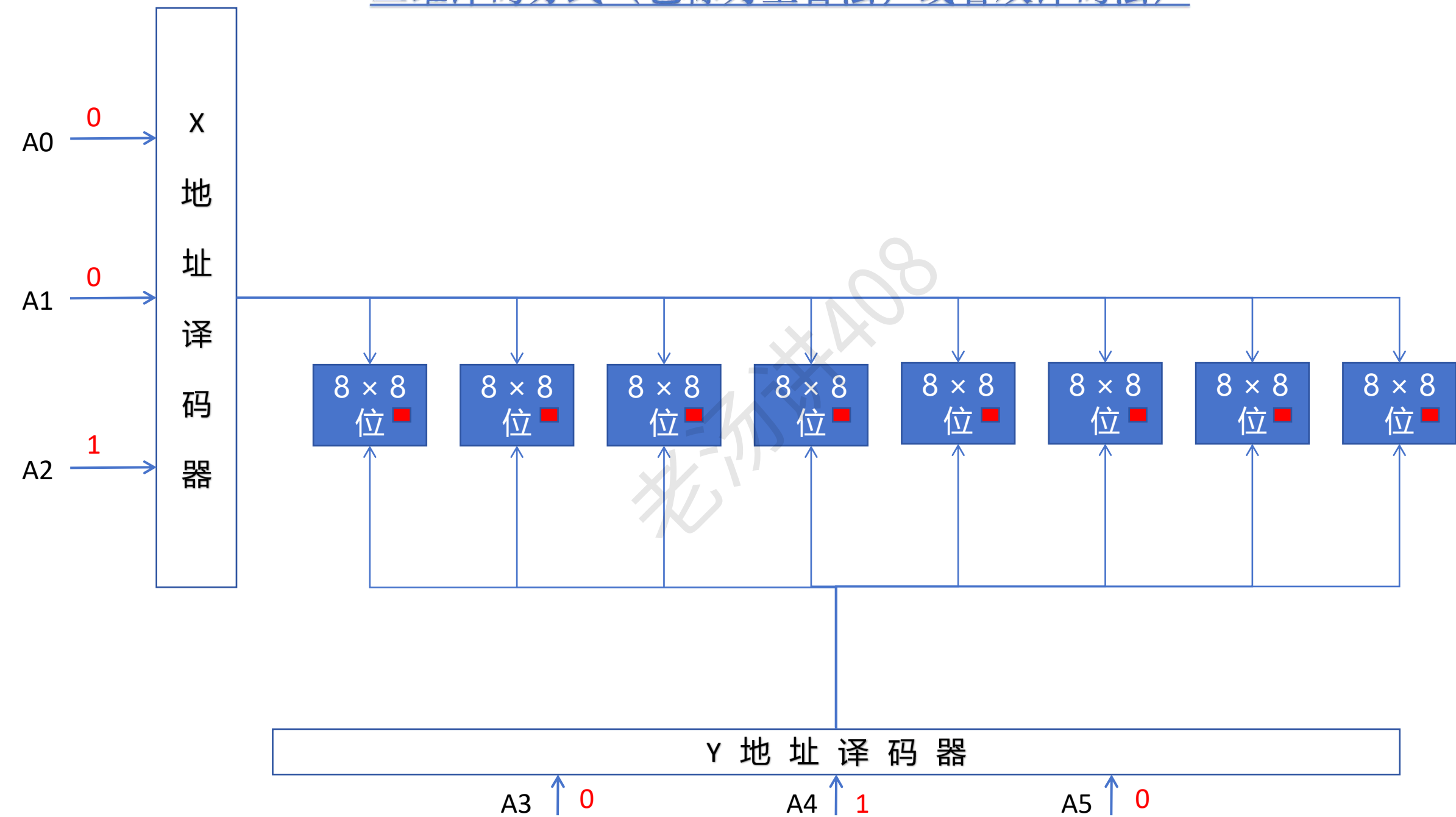
二维译码方式（也称为重合法，或者双译码法）



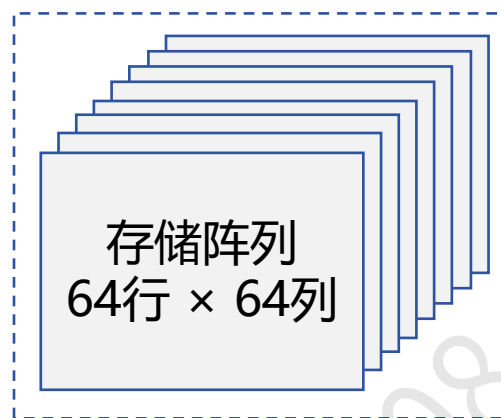
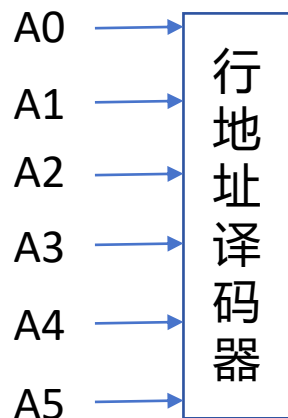
二维译码方式（也称为重合法，或者双译码法）



二维译码方式（也称为重合法，或者双译码法）



双译码结构存储器芯片内部结构图

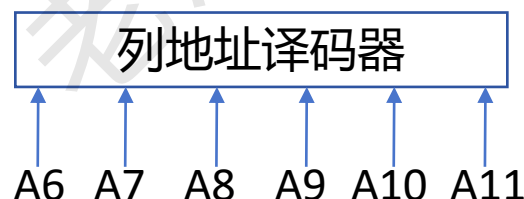


→ 存储体

从存储体中一次性读写的位数
就是一个**存储单元**

存储字

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

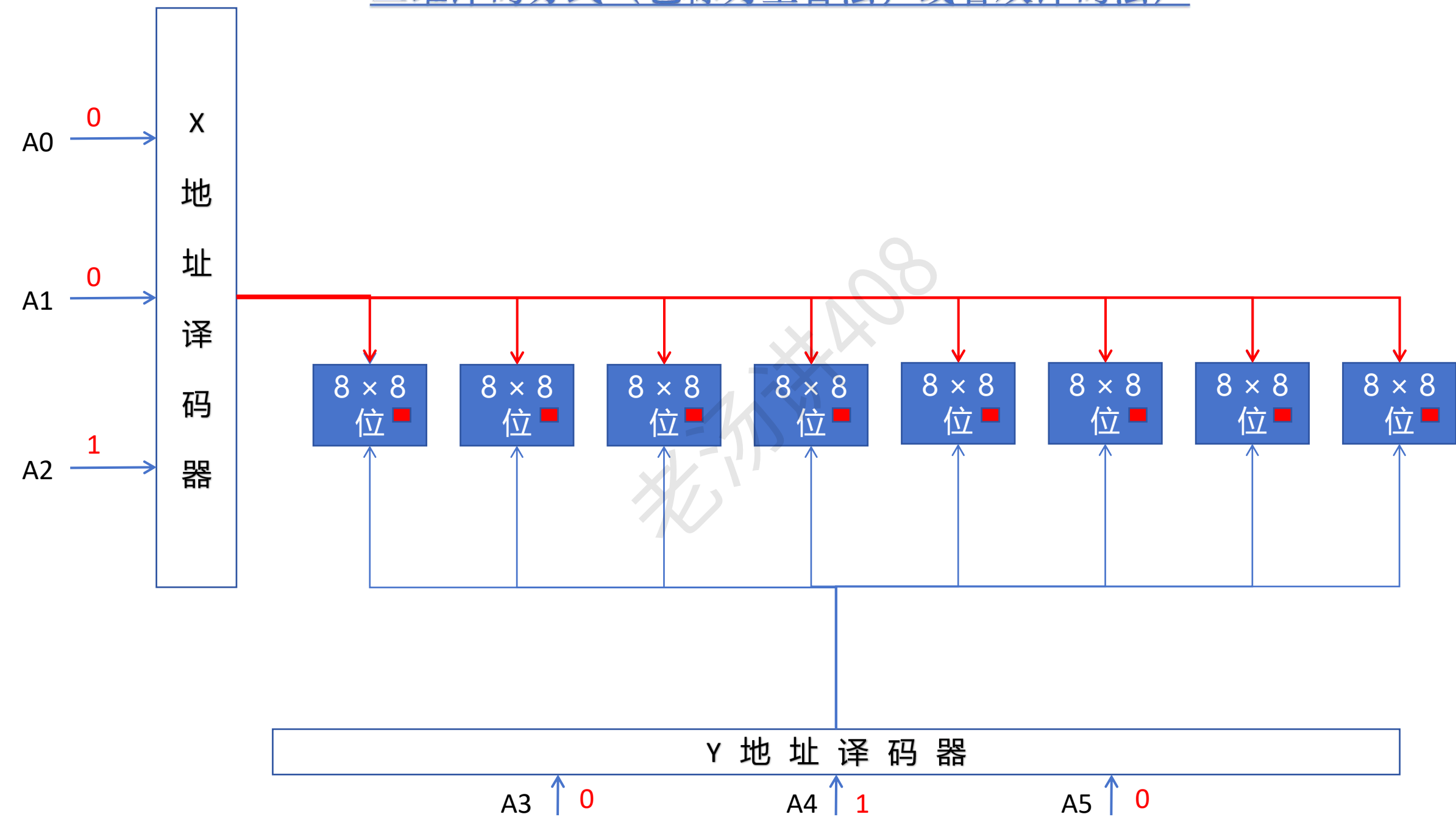


老汤
浙江 杭州

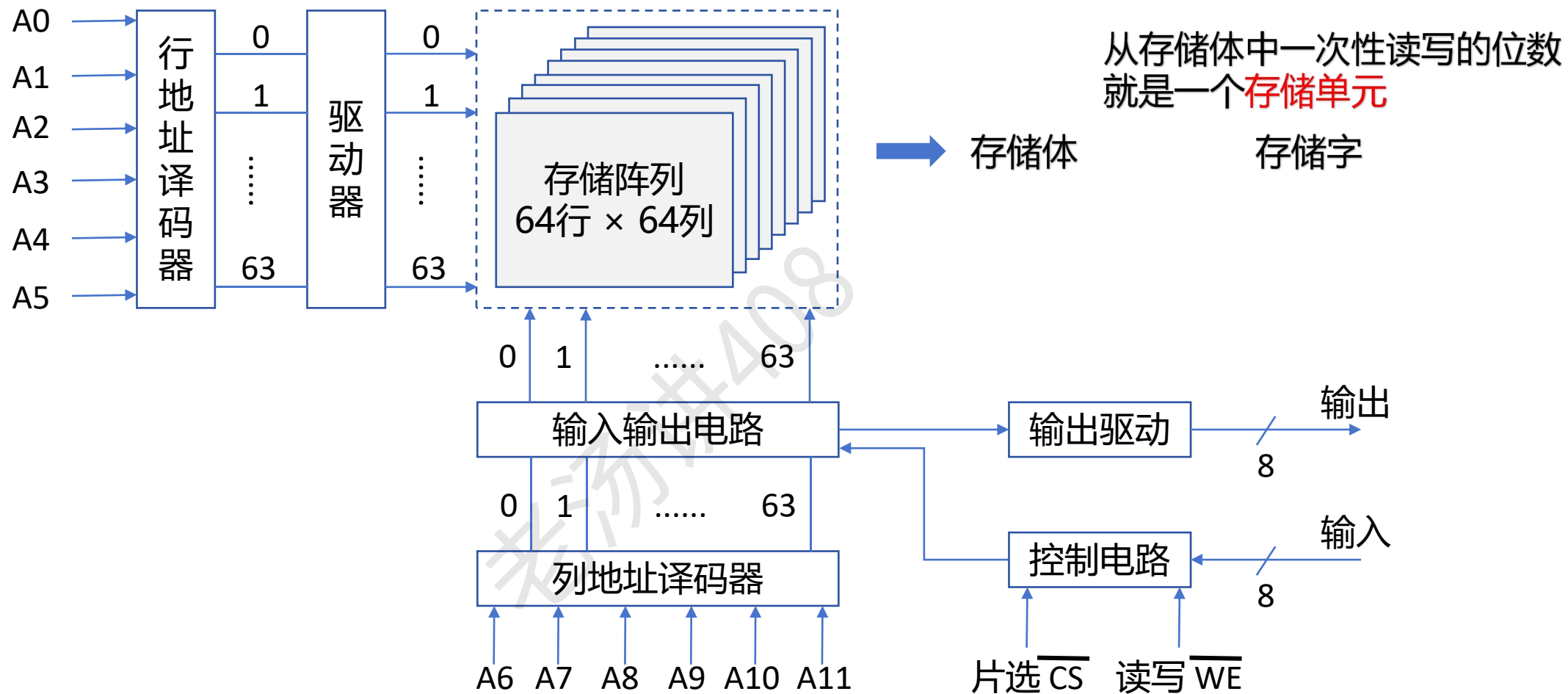


扫一扫上面的二维码图案，加我为朋友。

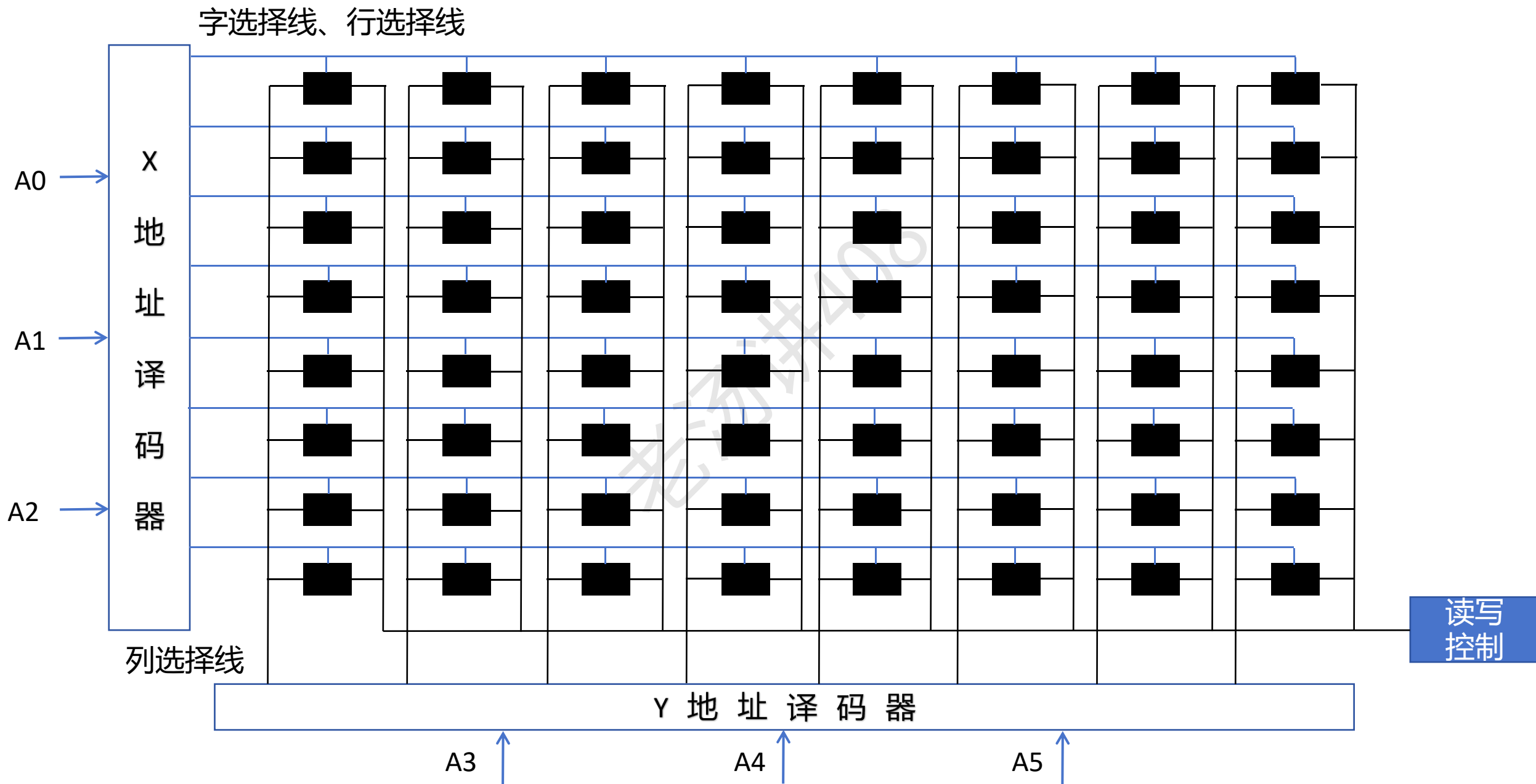
二维译码方式（也称为重合法，或者双译码法）



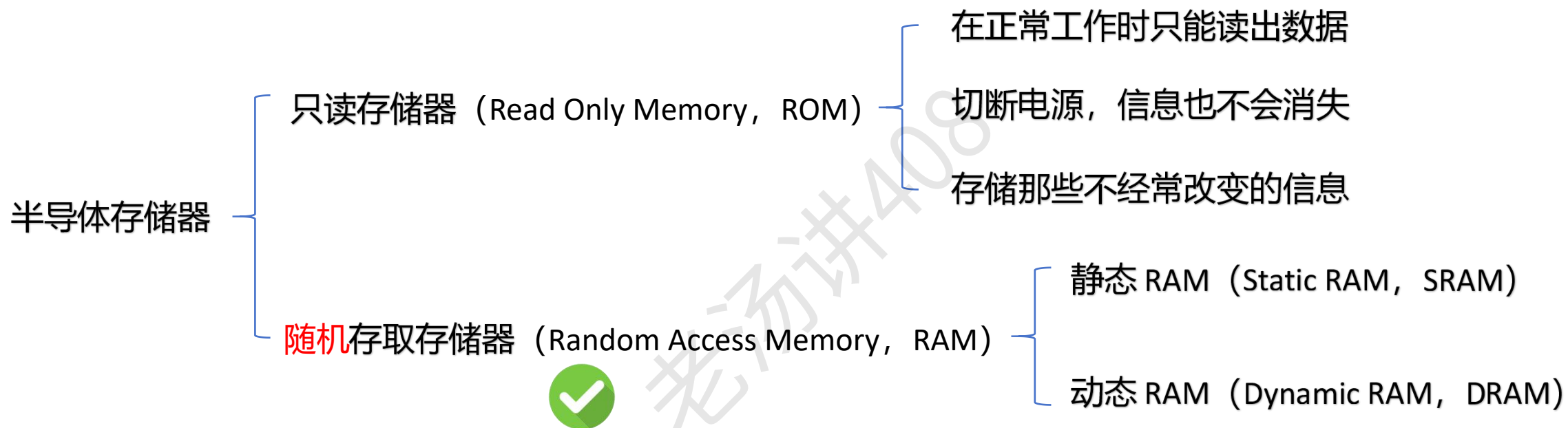
双译码结构存储器芯片内部结构图



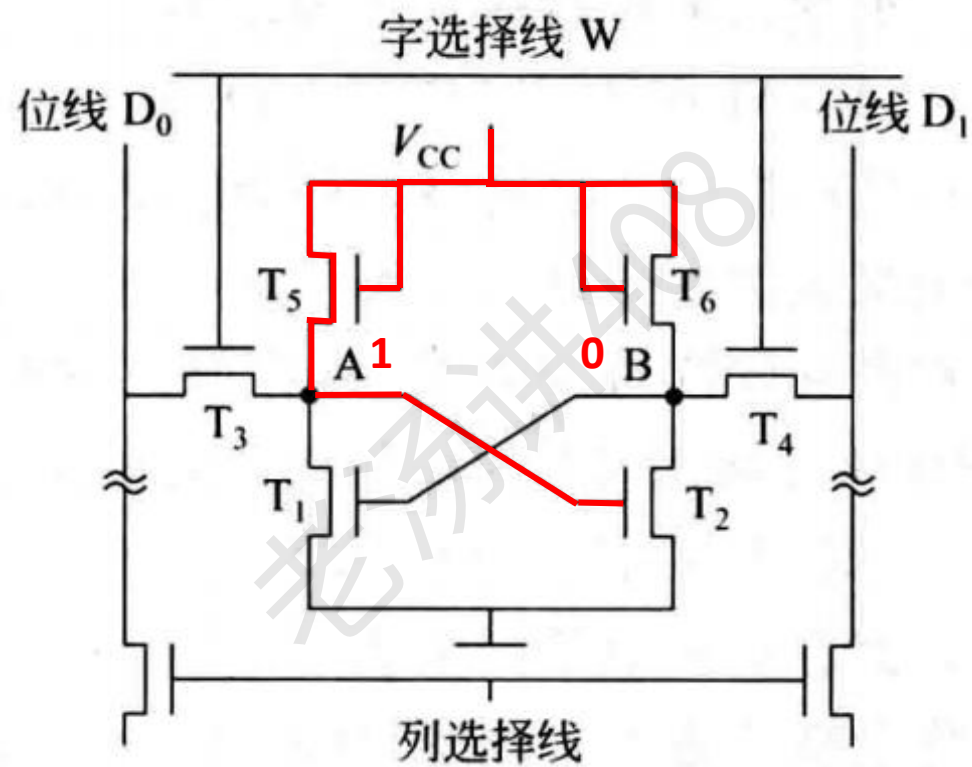
二维译码方式（也称为重合法，或者双译码法）



半导体存储器

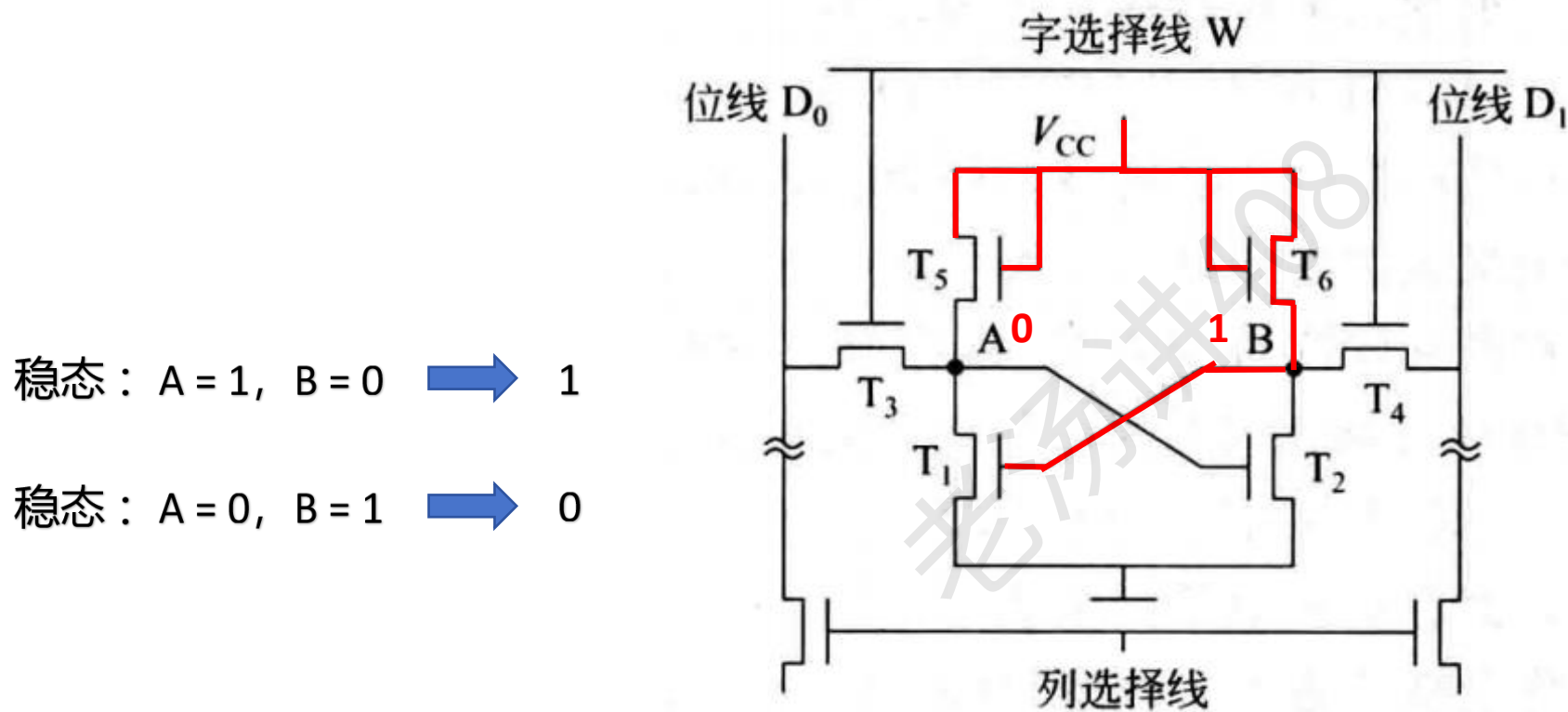


静态 RAM (Static RAM, SRAM)



SRAM 存储元电路结构

静态 RAM (Static RAM, SRAM)



稳态: $A = 1, B = 0 \rightarrow 1$

稳态: $A = 0, B = 1 \rightarrow 0$

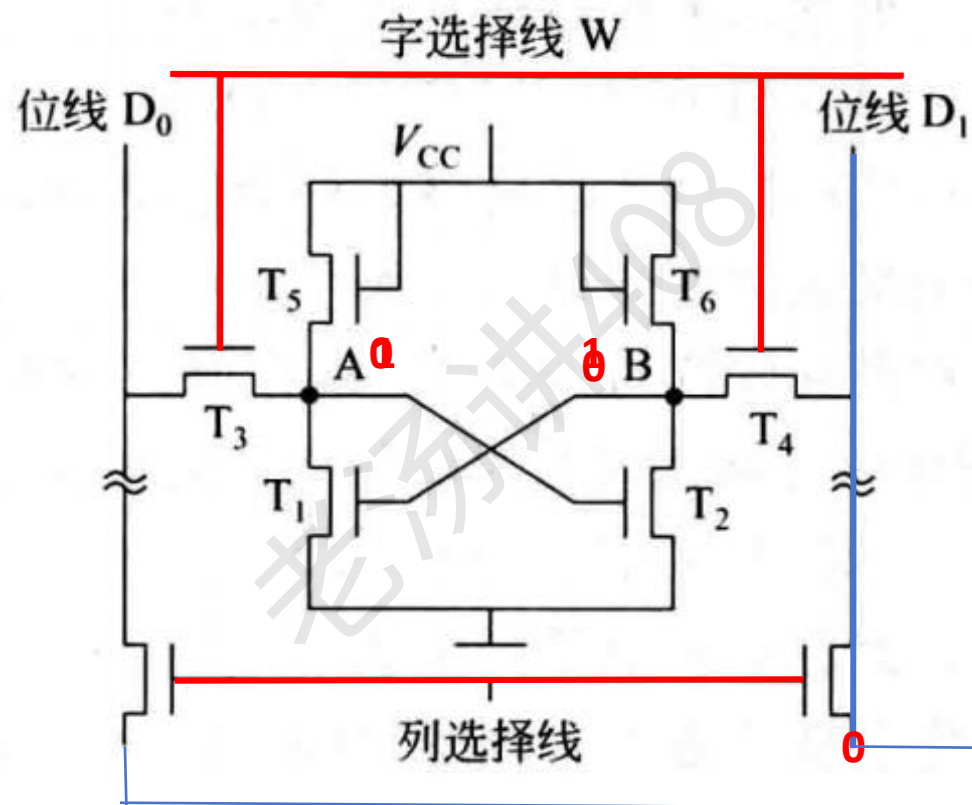
SRAM 存储元电路结构

SRAM 存储元一开始时存储的是 0 还是 1, 这个是随机的

静态 RAM (Static RAM, SRAM) - 写 1 操作

稳态：A = 1, B = 0 $\rightarrow$ 1

稳态：A = 0, B = 1 $\rightarrow$ 0



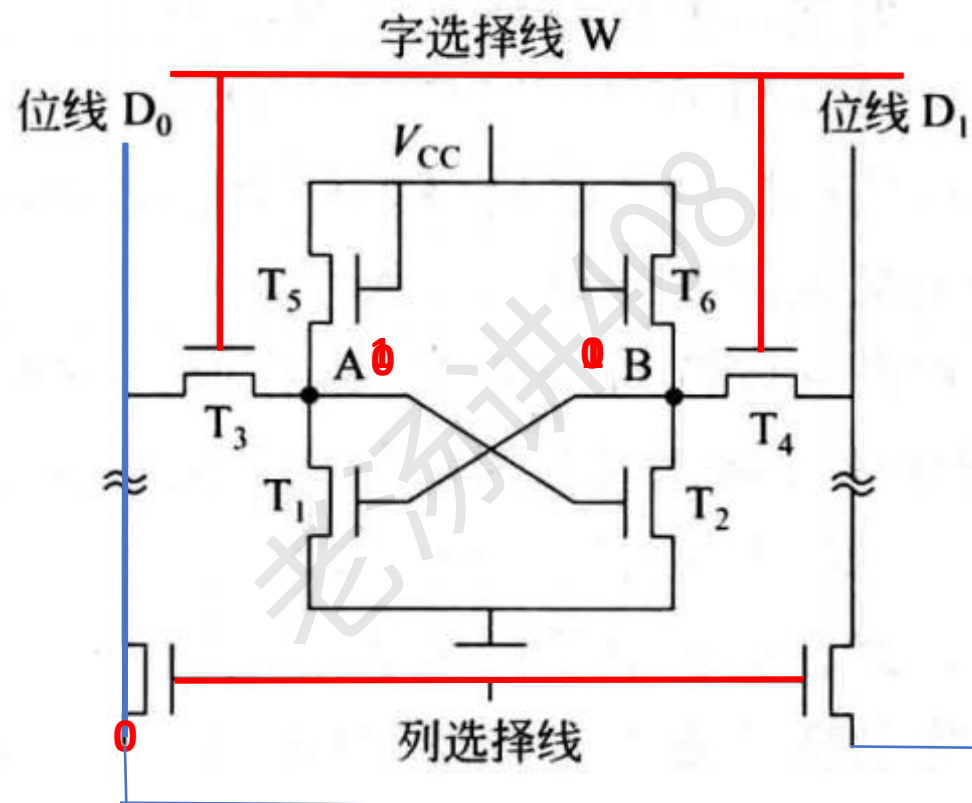
SRAM 存储元电路结构

读写
控制

静态 RAM (Static RAM, SRAM) - 写 0 操作

稳态：A = 1, B = 0 $\rightarrow$ 1

稳态：A = 0, B = 1 $\rightarrow$ 0



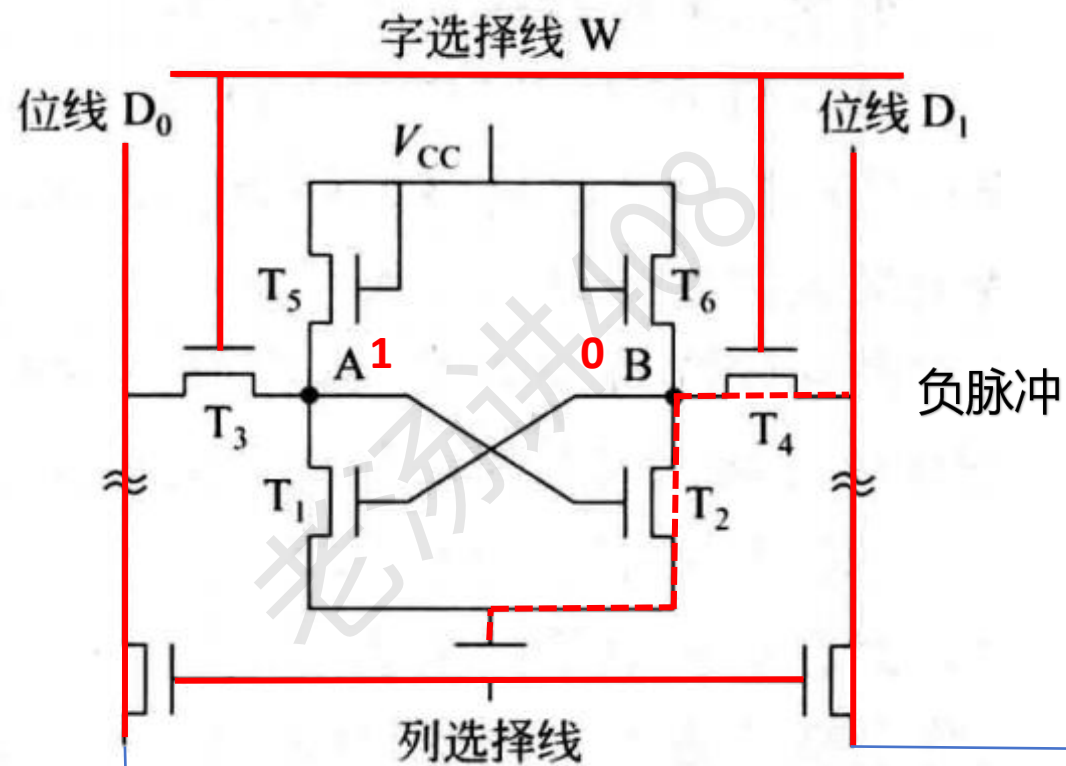
SRAM 存储元电路结构

读写
控制

静态 RAM (Static RAM, SRAM) - 读出操作

稳态：A = 1, B = 0 $\rightarrow$ 1

稳态：A = 0, B = 1 $\rightarrow$ 0



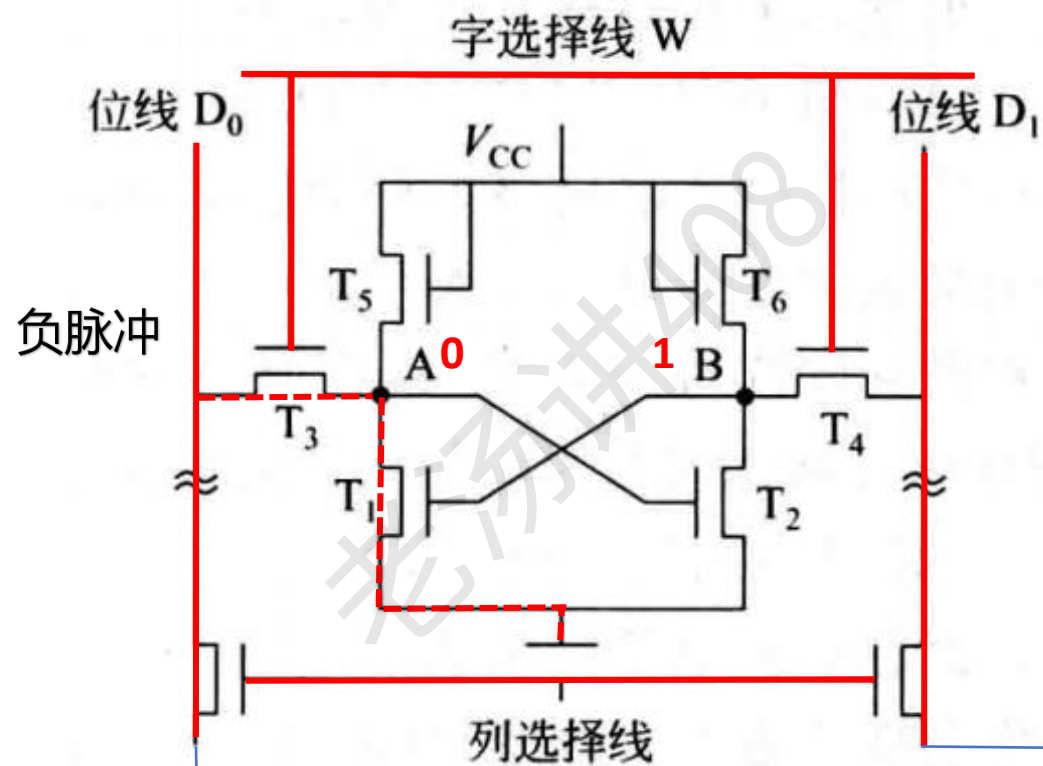
SRAM 存储元电路结构

读写
控制

静态 RAM (Static RAM, SRAM) - 读出操作

稳态：A = 1, B = 0 $\rightarrow$ 1

稳态：A = 0, B = 1 $\rightarrow$ 0



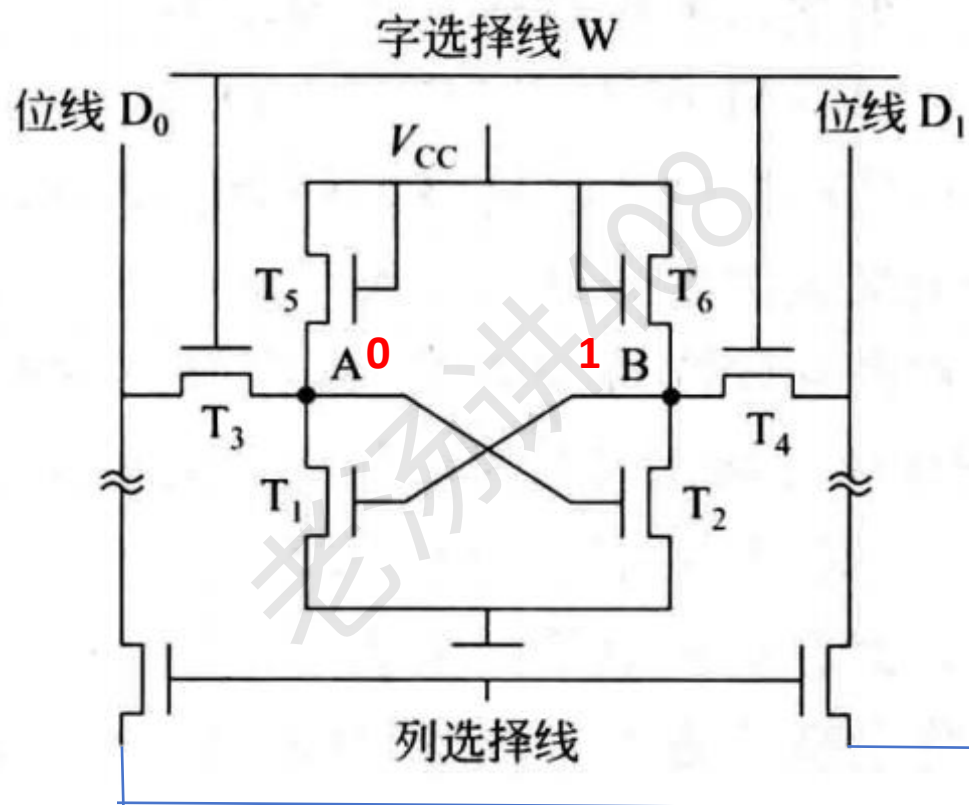
SRAM 存储元电路结构

读写
控制

静态 RAM (Static RAM, SRAM) - 读出操作

稳态：A = 1, B = 0 $\rightarrow$ 1

稳态：A = 0, B = 1 $\rightarrow$ 0



0 表示读, 1 表示写

写使能 (Write Eable)

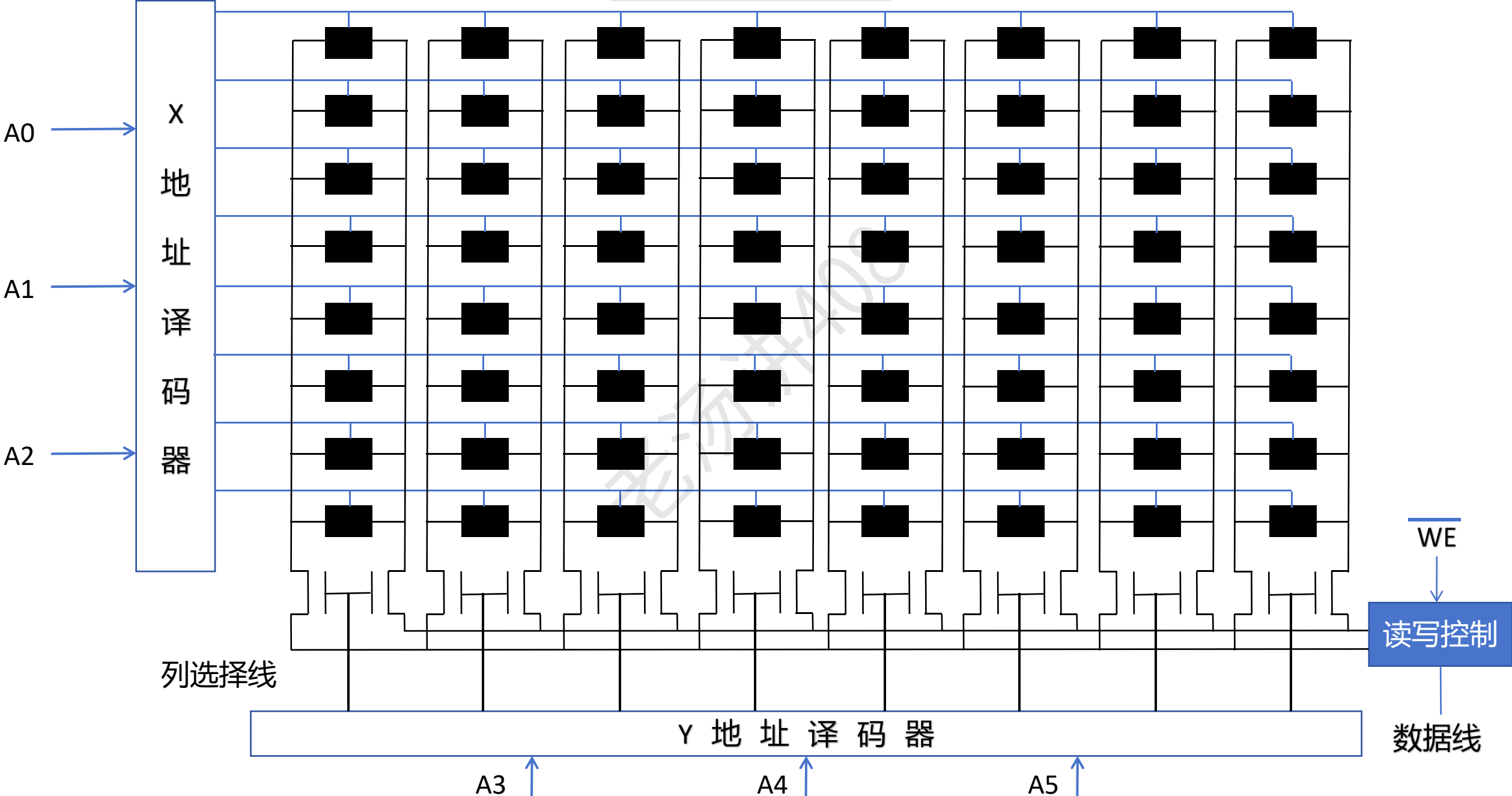
$\overline{\text{WE}}$

读写
控制

SRAM 存储元电路结构

字选择线、行选择线

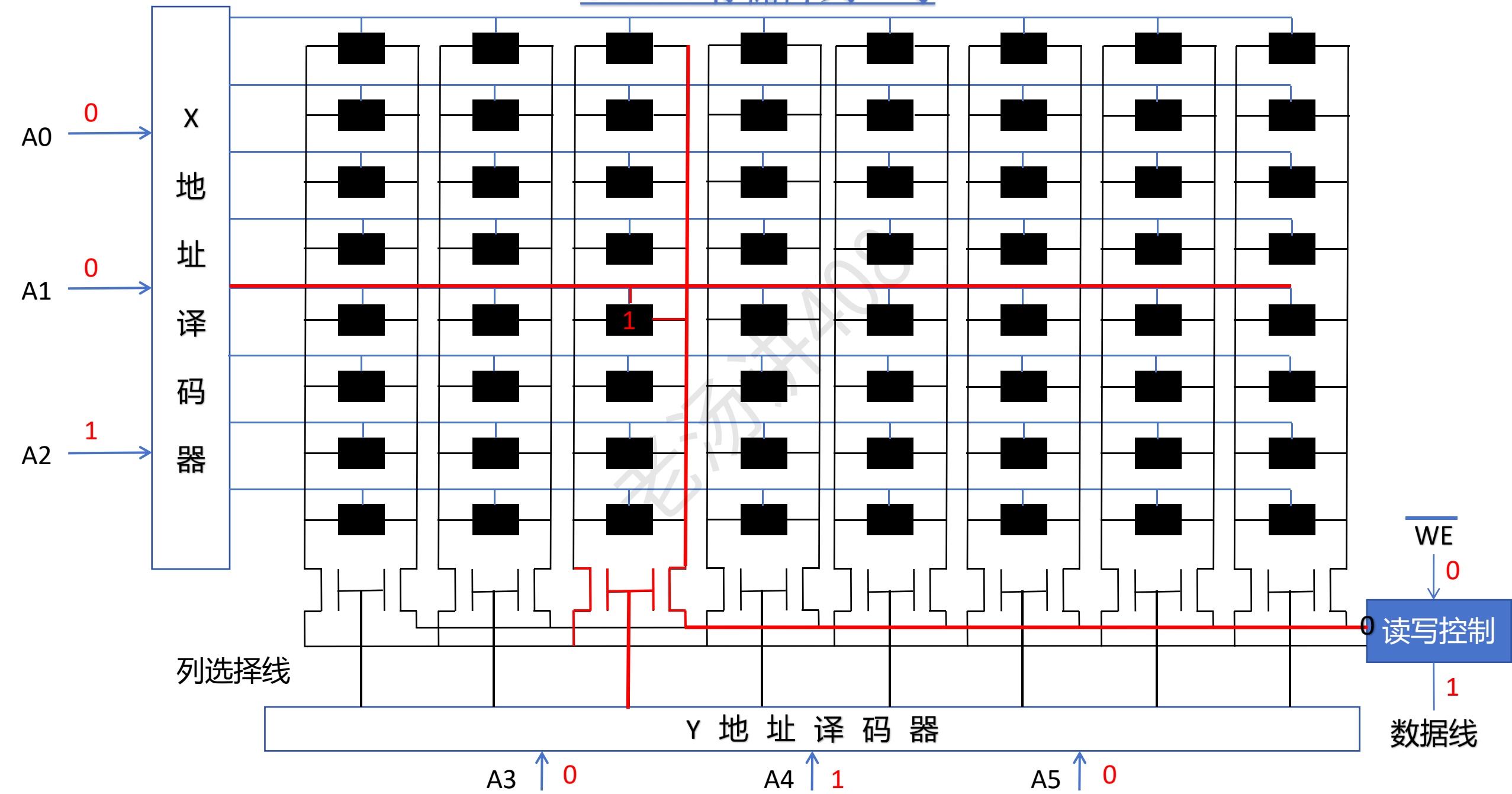
SRAM 存储阵列



字选择线、行选择线

SRAM 存储阵列 - 写

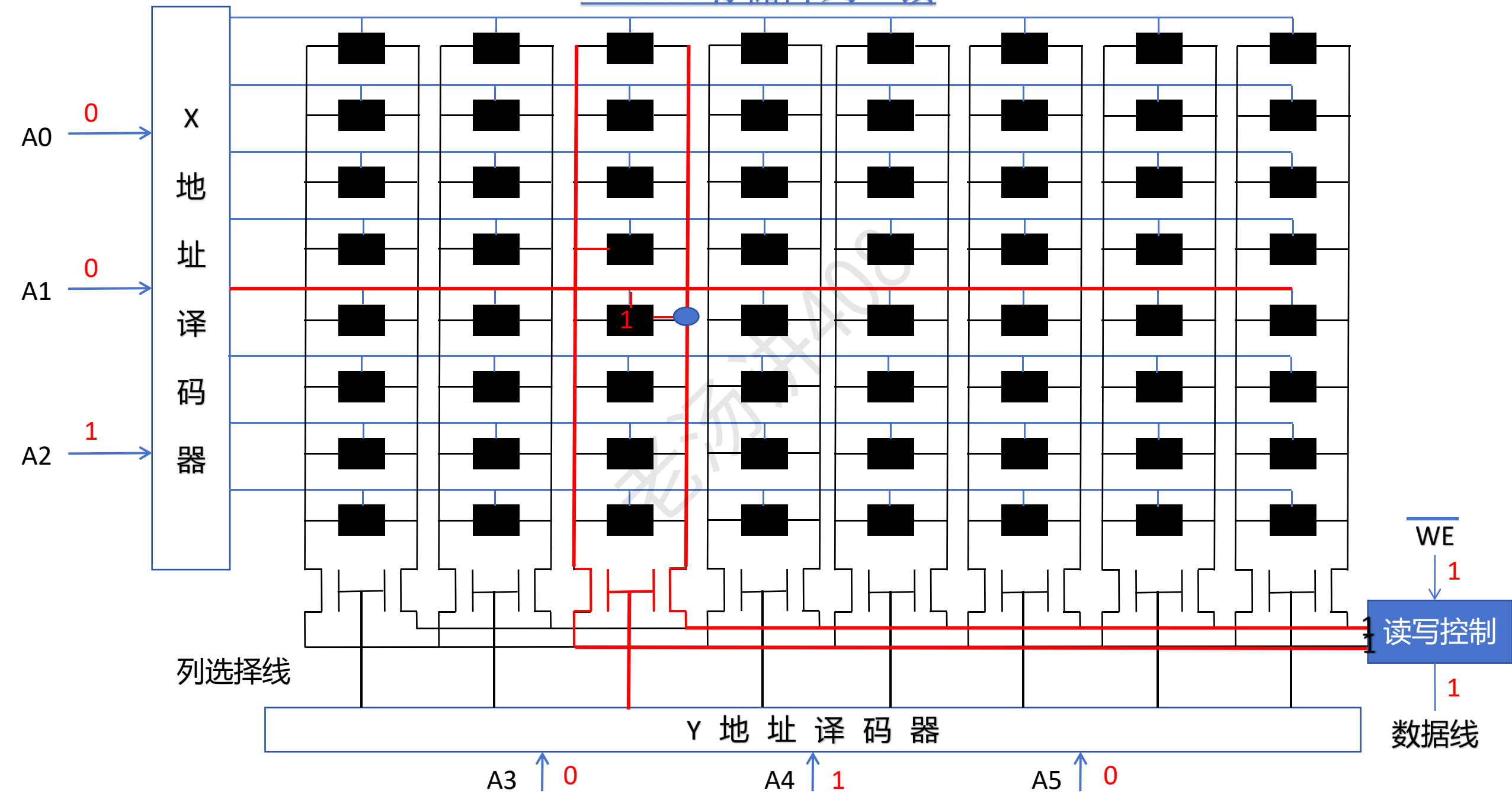
A5A4A3A2A1A0 = 010100



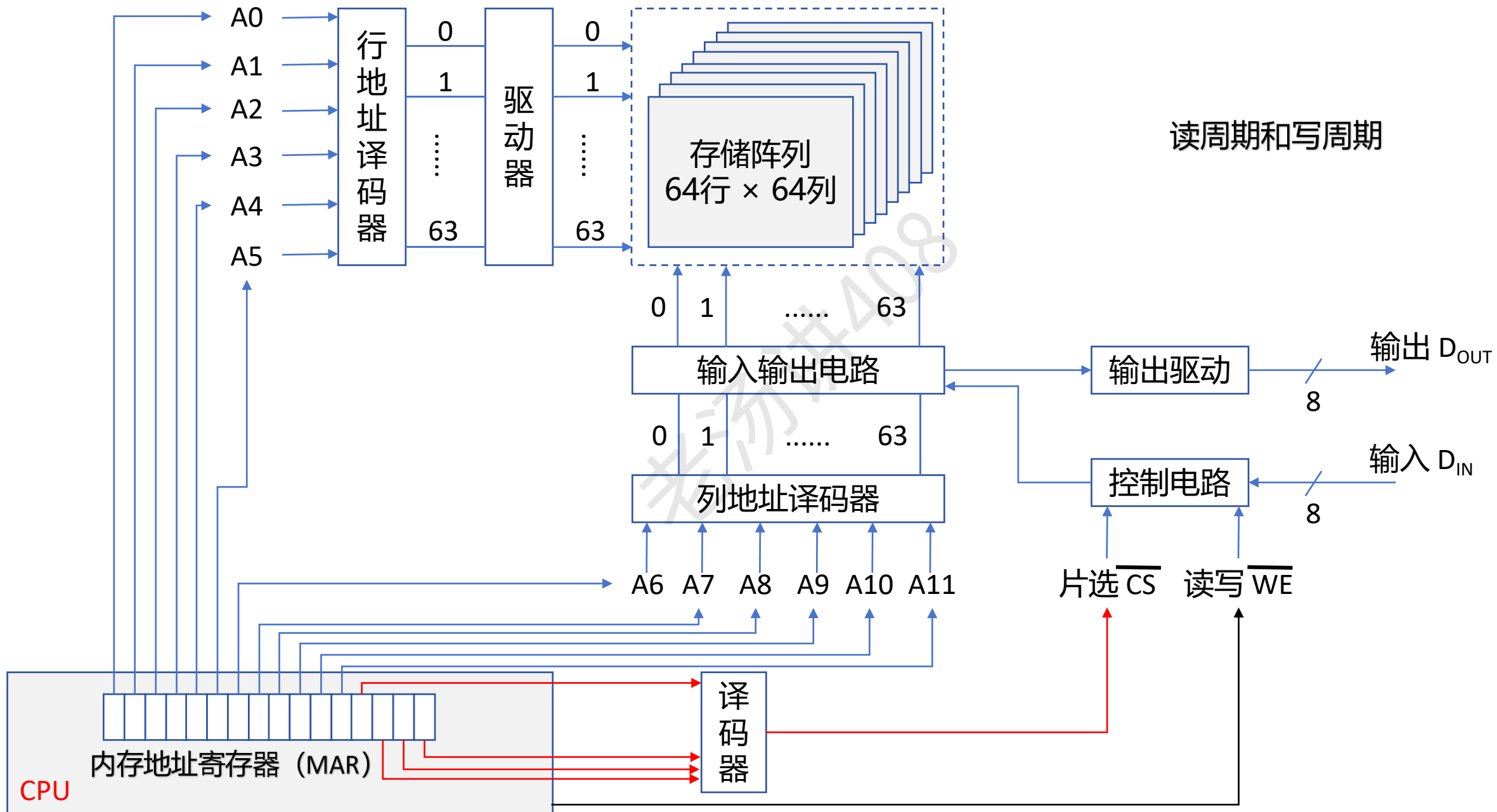
字选择线、行选择线

SRAM 存储阵列 - 读

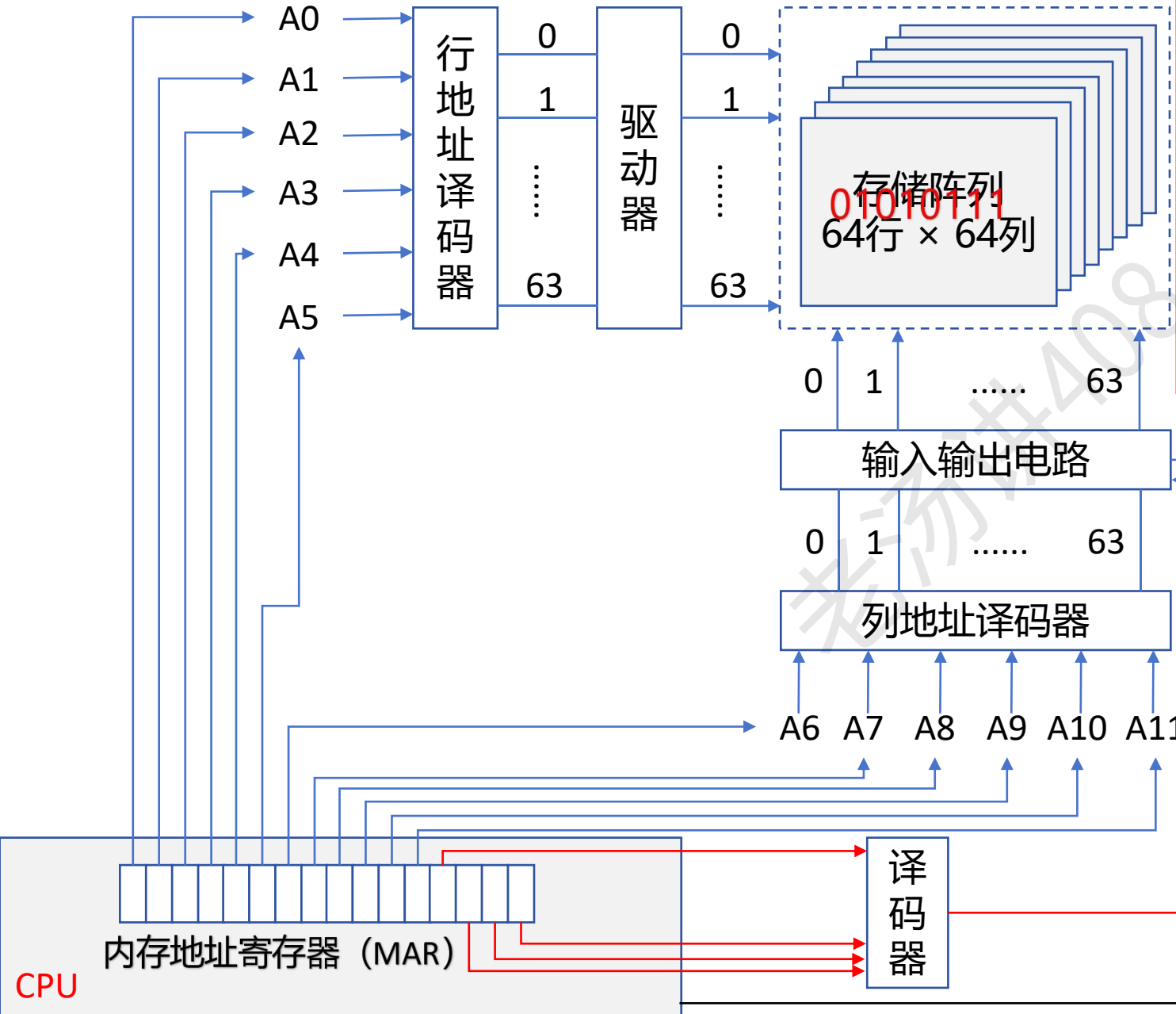
A5A4A3A2A1A0 = 010100



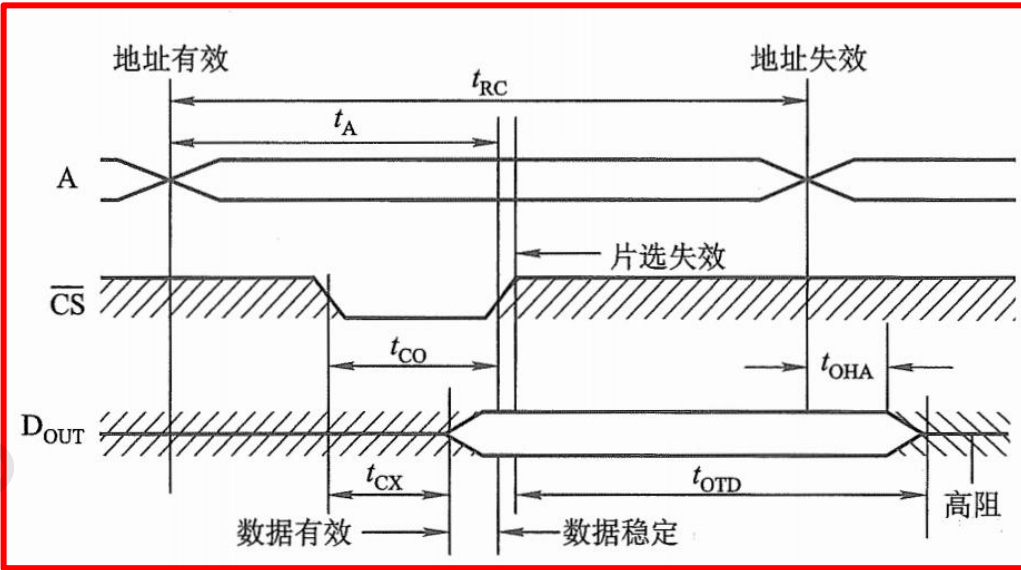
SRAM 存储芯片结构



SRAM 存储芯片读周期

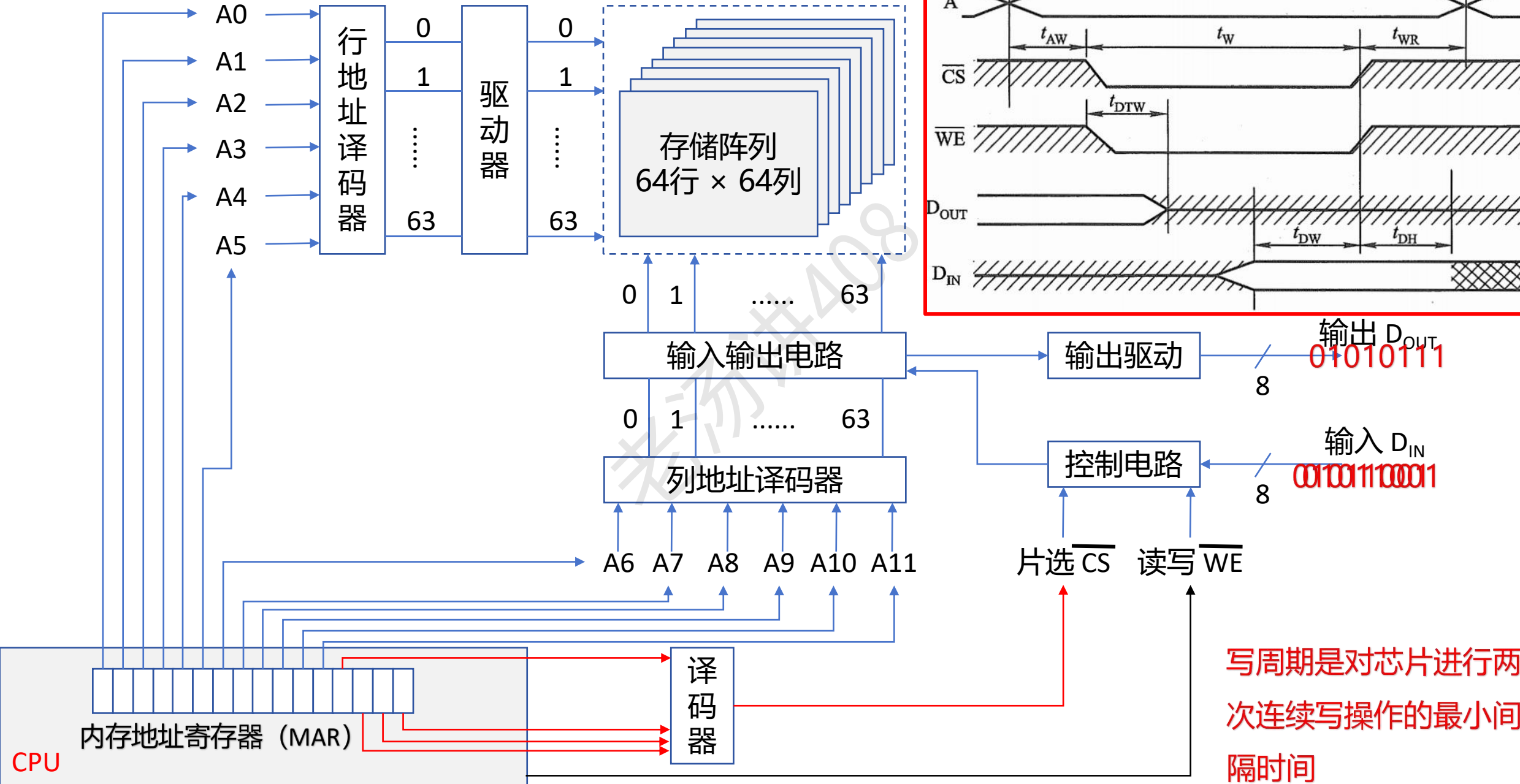


SRAM 读周期时序图



读周期是对芯片进行两次连续读操作的最小间隔时间

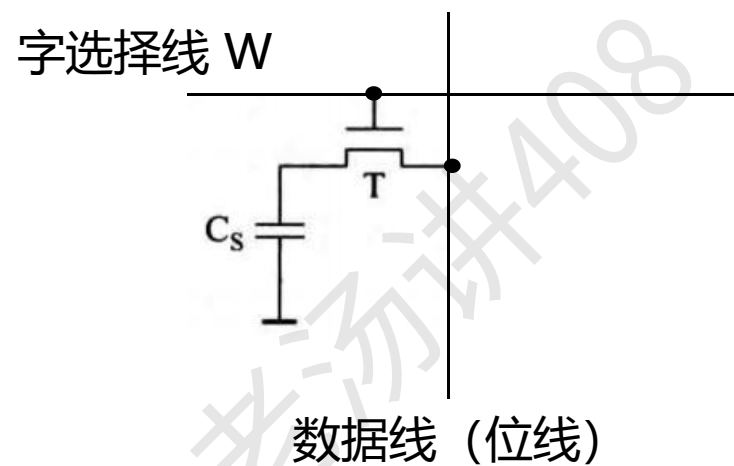
SRAM 存储芯片写周期



动态 RAM (Dynamic RAM, DRAM)

电容 C_S 有电荷 $\rightarrow$ 存储 1

电容 C_S 无电荷 $\rightarrow$ 存储 0



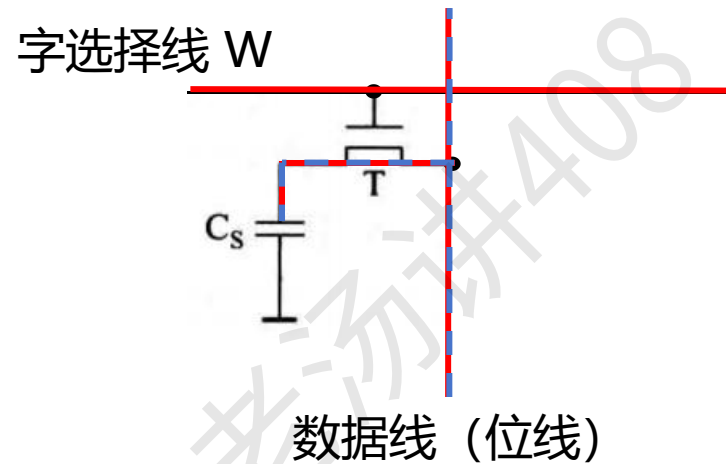
一个 MOS 管 T
一个电容 C_S

} 单管动态存储元件

动态 RAM (Dynamic RAM, DRAM) : 写入

电容 C_s 有电荷 $\rightarrow$ 存储 1

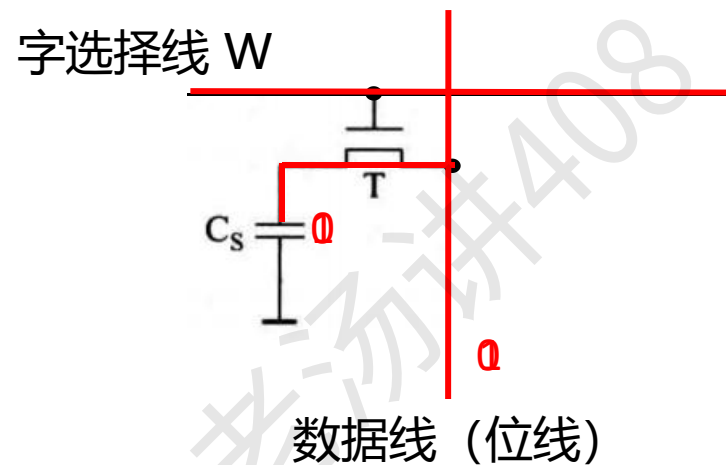
电容 C_s 无电荷 $\rightarrow$ 存储 0



动态 RAM (Dynamic RAM, DRAM) : 读出

电容 C_s 有电荷 $\rightarrow$ 存储 1

电容 C_s 无电荷 $\rightarrow$ 存储 0



加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

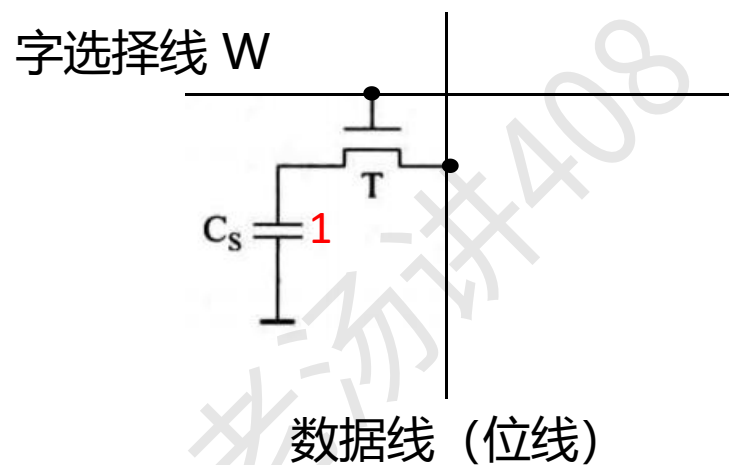
破坏性读出

再生

动态 RAM (Dynamic RAM, DRAM) : 刷新

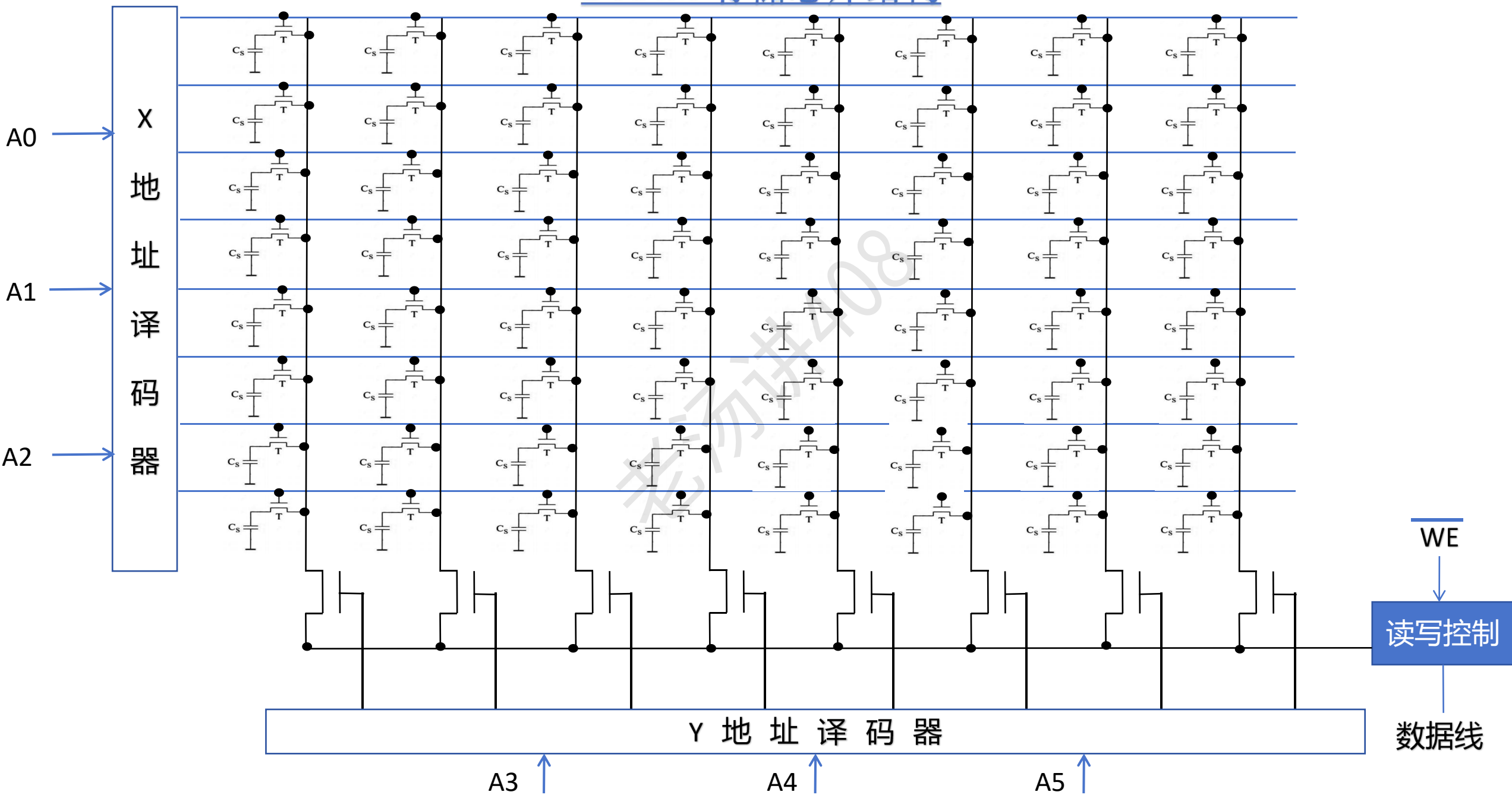
电容 C_s 有电荷 $\rightarrow$ 存储 1

电容 C_s 无电荷 $\rightarrow$ 存储 0

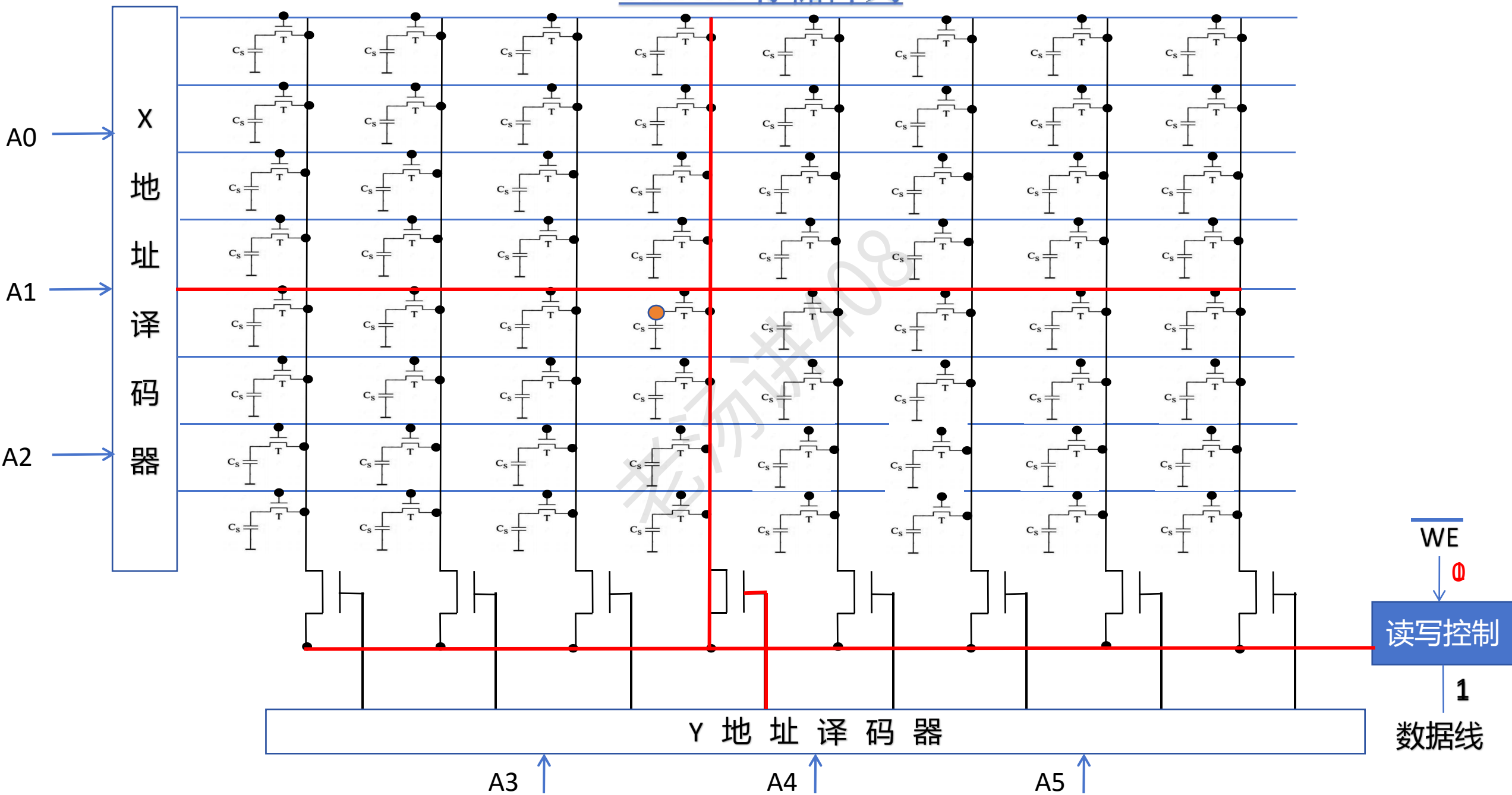


电容 C_s 上存储的电荷会缓慢的放电, 超过一定的时间, 就会丢失信息, 这个是电容的特点, 因此必须定时给电容 C_s 充电, 这一个过程称为**刷新**

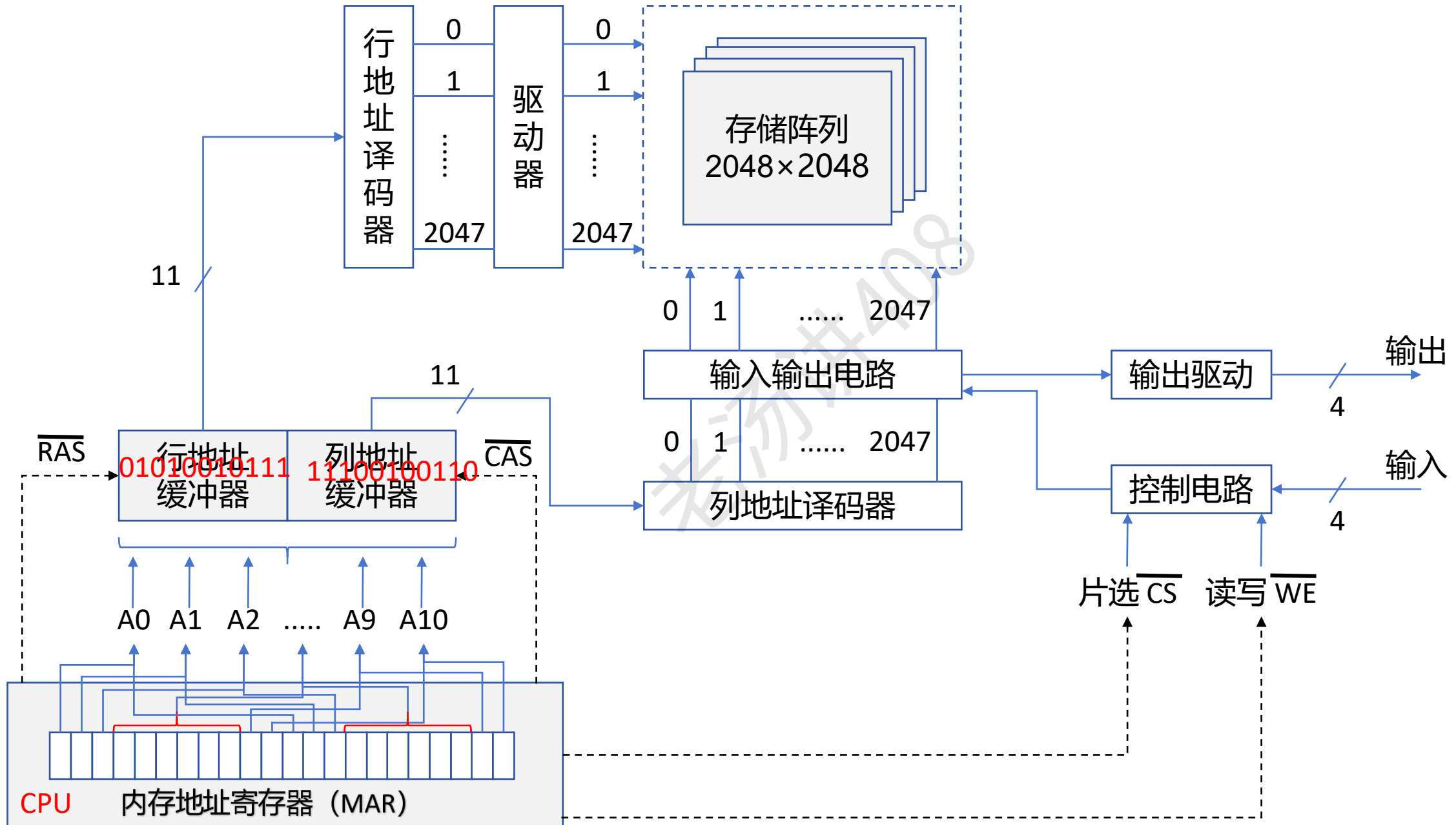
DRAM 存储芯片结构



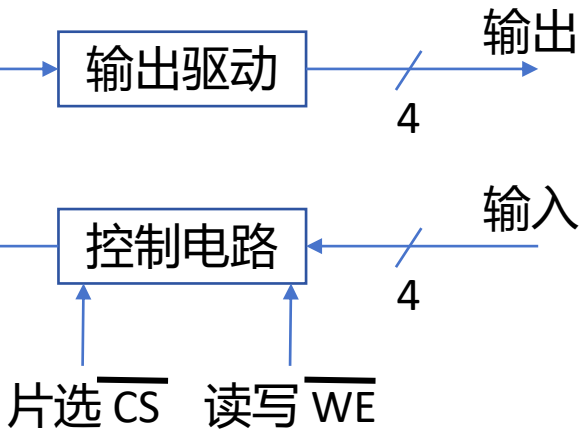
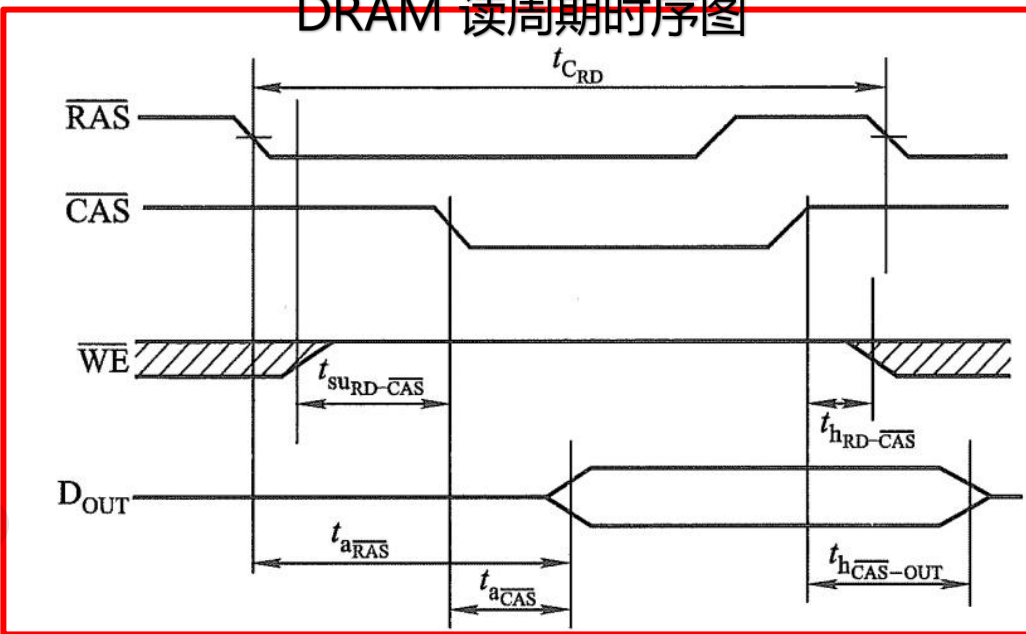
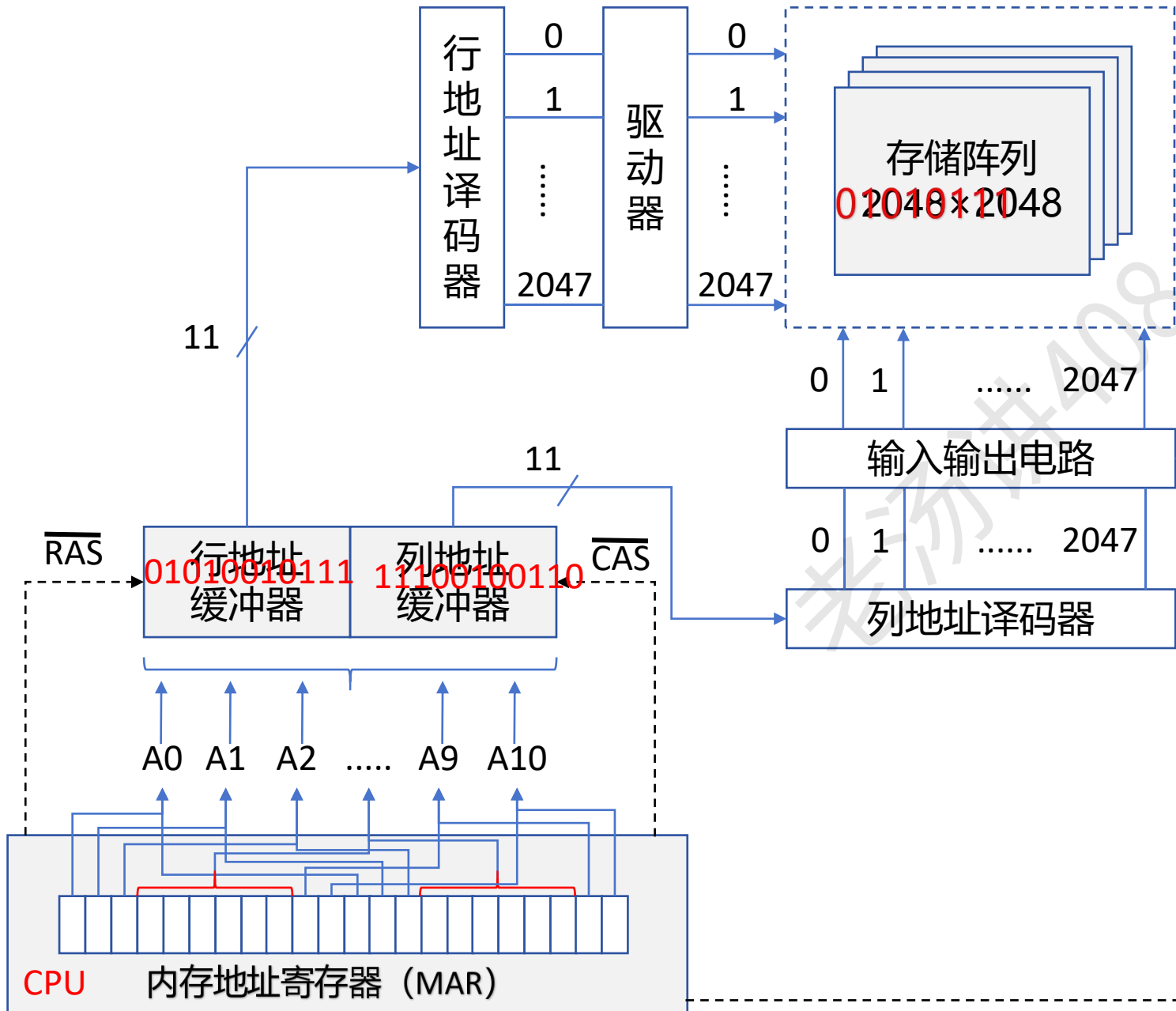
DRAM 存储阵列



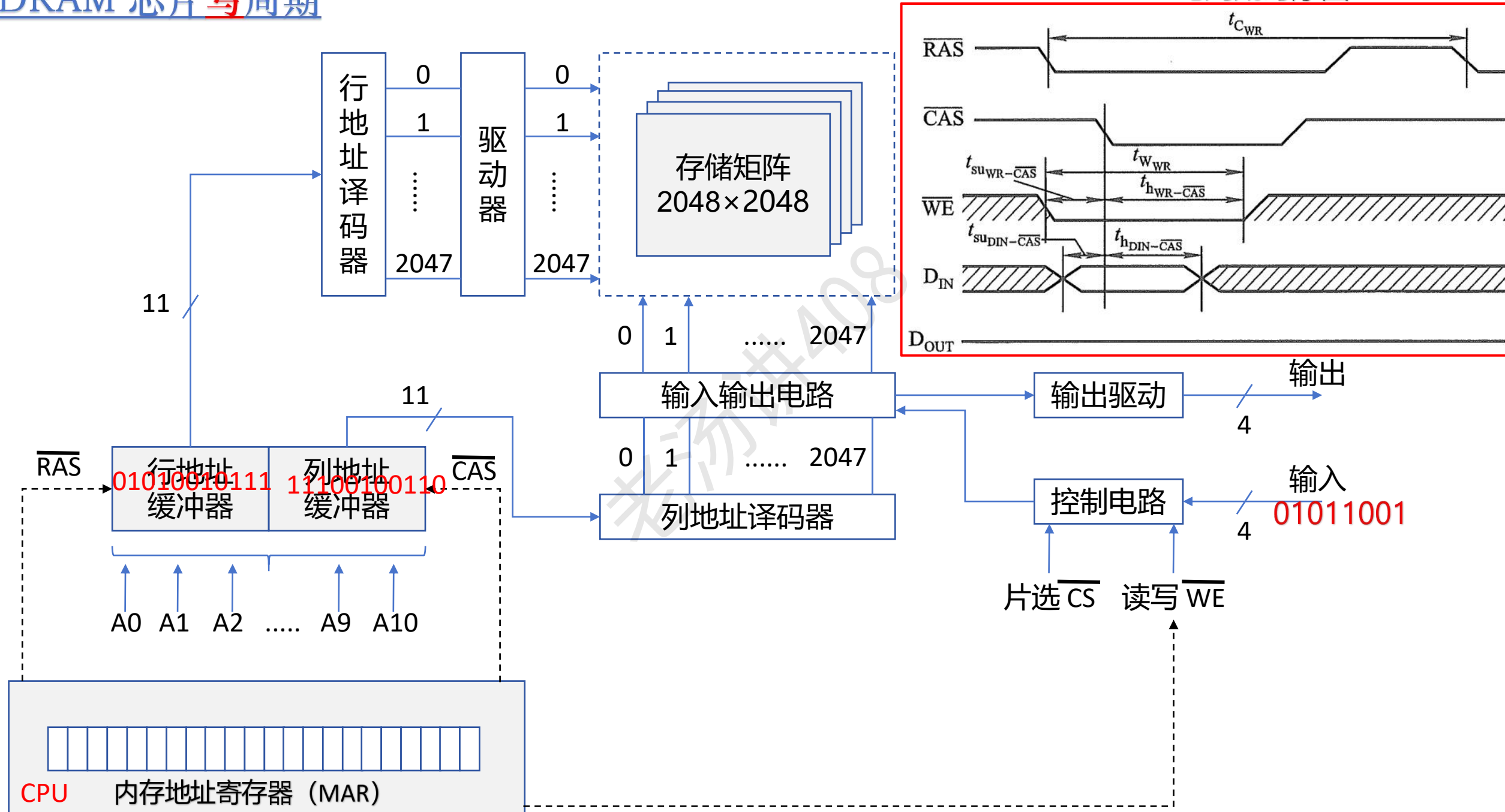
2048 × 2048 × 4 位 (2MB) 的 DRAM 存储芯片结构



DRAM 芯片读周期

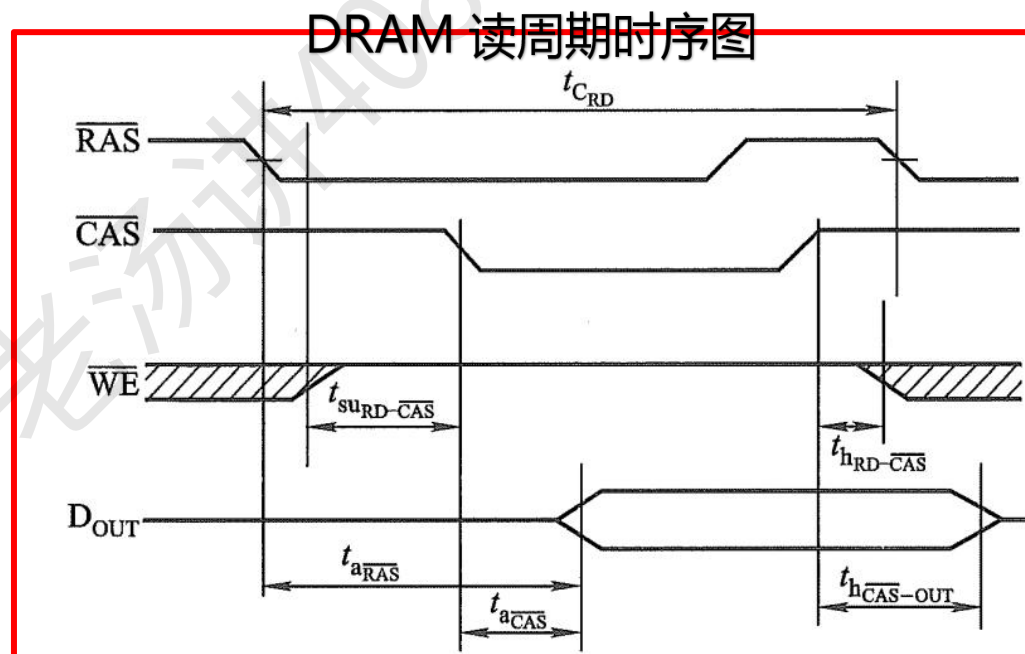


DRAM 芯片写周期



存储器的存储周期

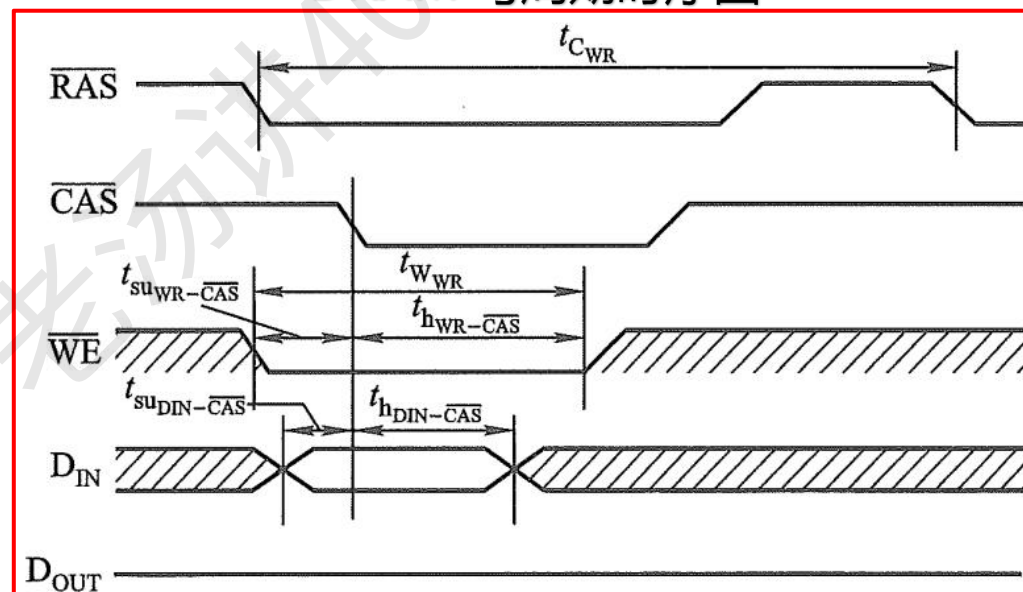
读出时间 T_A ：指从存储器接收到读命令开始，至信息被送到数据线上所需的时间



存储器的存储周期

写入时间 T_w ：指从存储器接收到写命令开始，至信息被写入存储器所需的时间

DRAM 写周期时序图

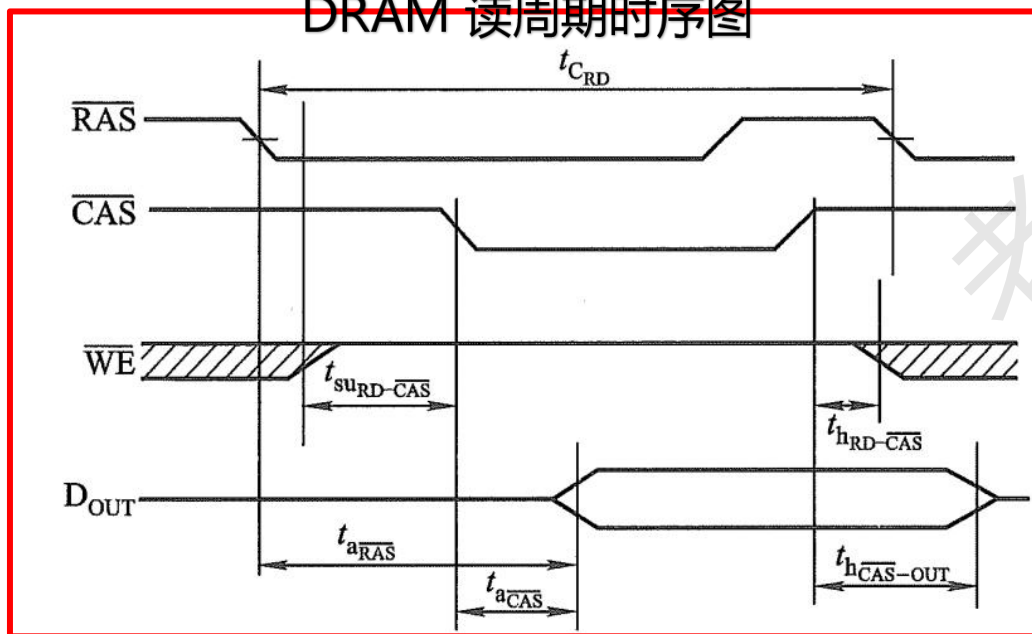


存储器的存储周期

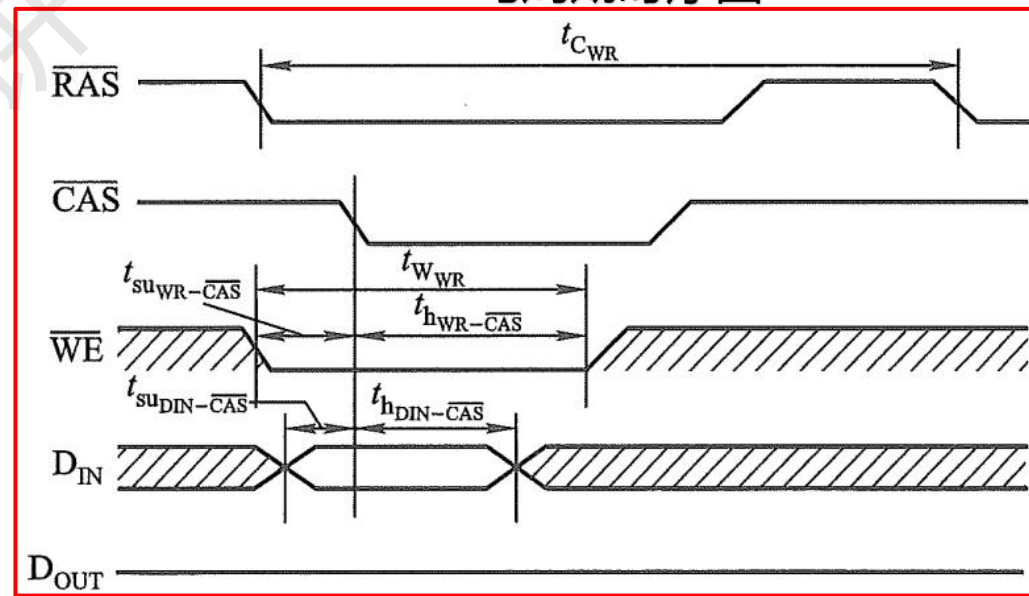
存储周期 T_M : 指存储器进行一次读写操作所需要的全部时间, 也就是存储器进行连续读写操作所允许的最短间隔时间

它应该等于读出/写入时间加上一次存取开始前后所要求的附加时间

DRAM 读周期时序图



DRAM 写周期时序图



本节课的内容

① DRAM 的刷新操作

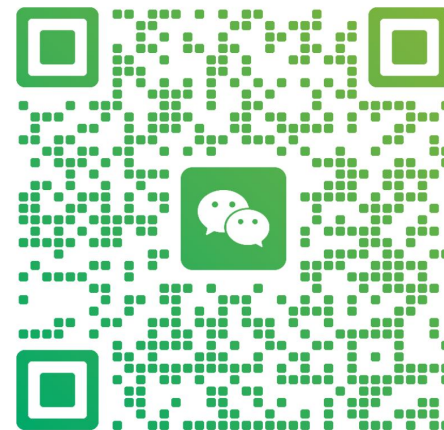
② SRAM 对比 DRAM

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州

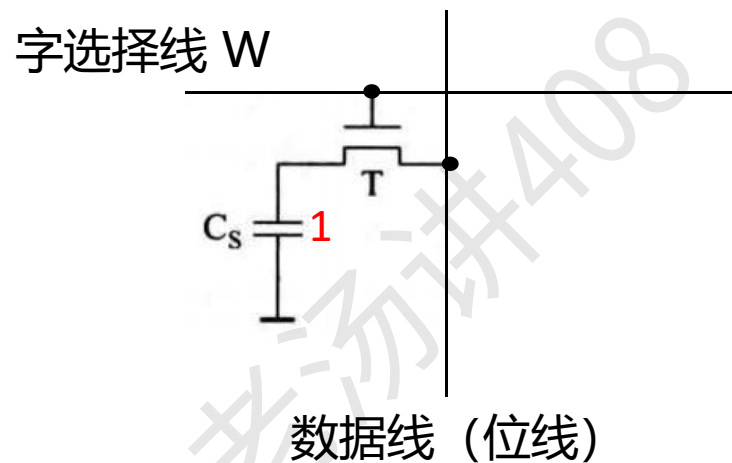


扫一扫上面的二维码图案，加我为朋友。

DRAM 的刷新

电容 C_S 有电荷 $\rightarrow$ 存储 1

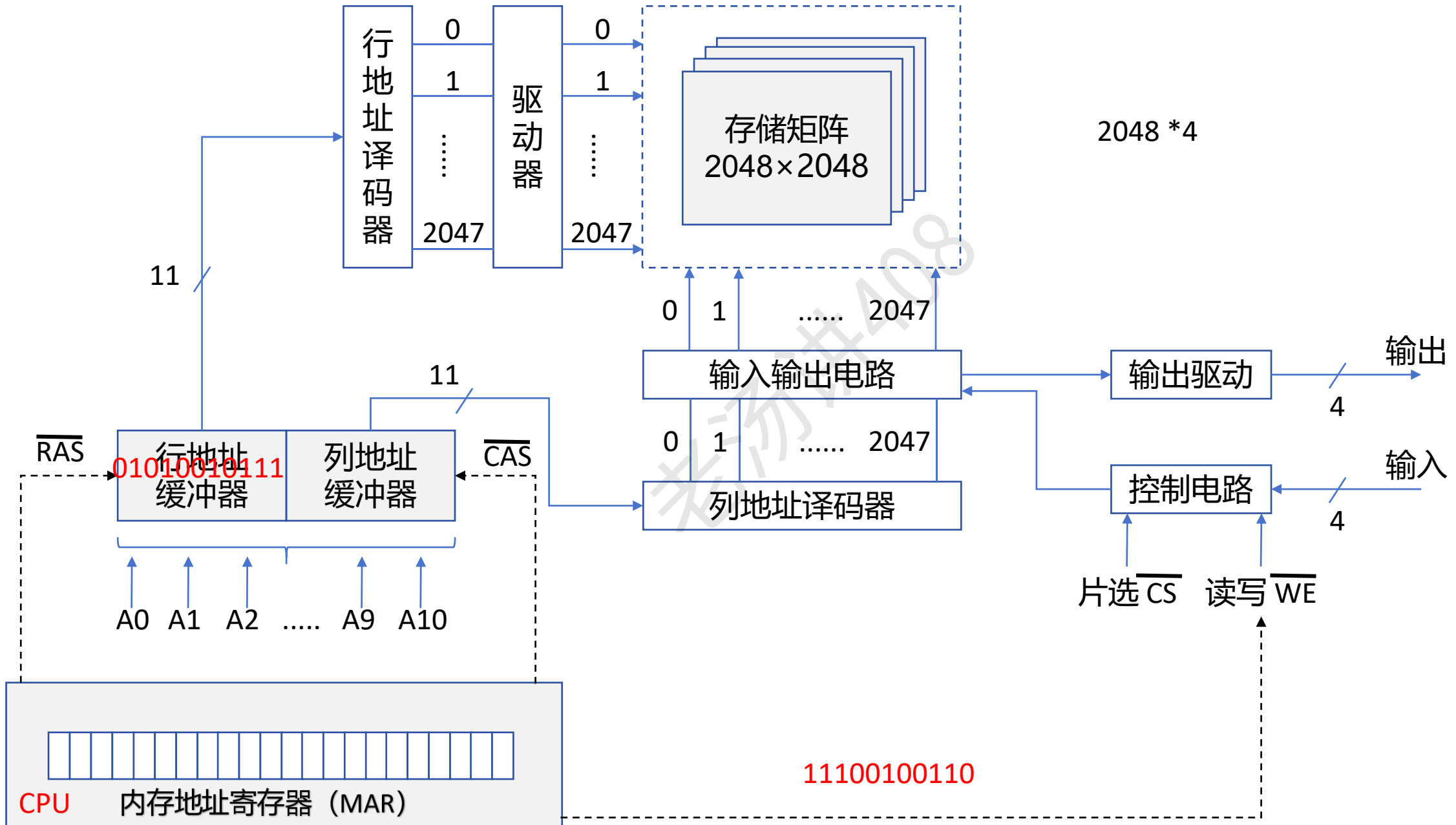
电容 C_S 无电荷 $\rightarrow$ 存储 0



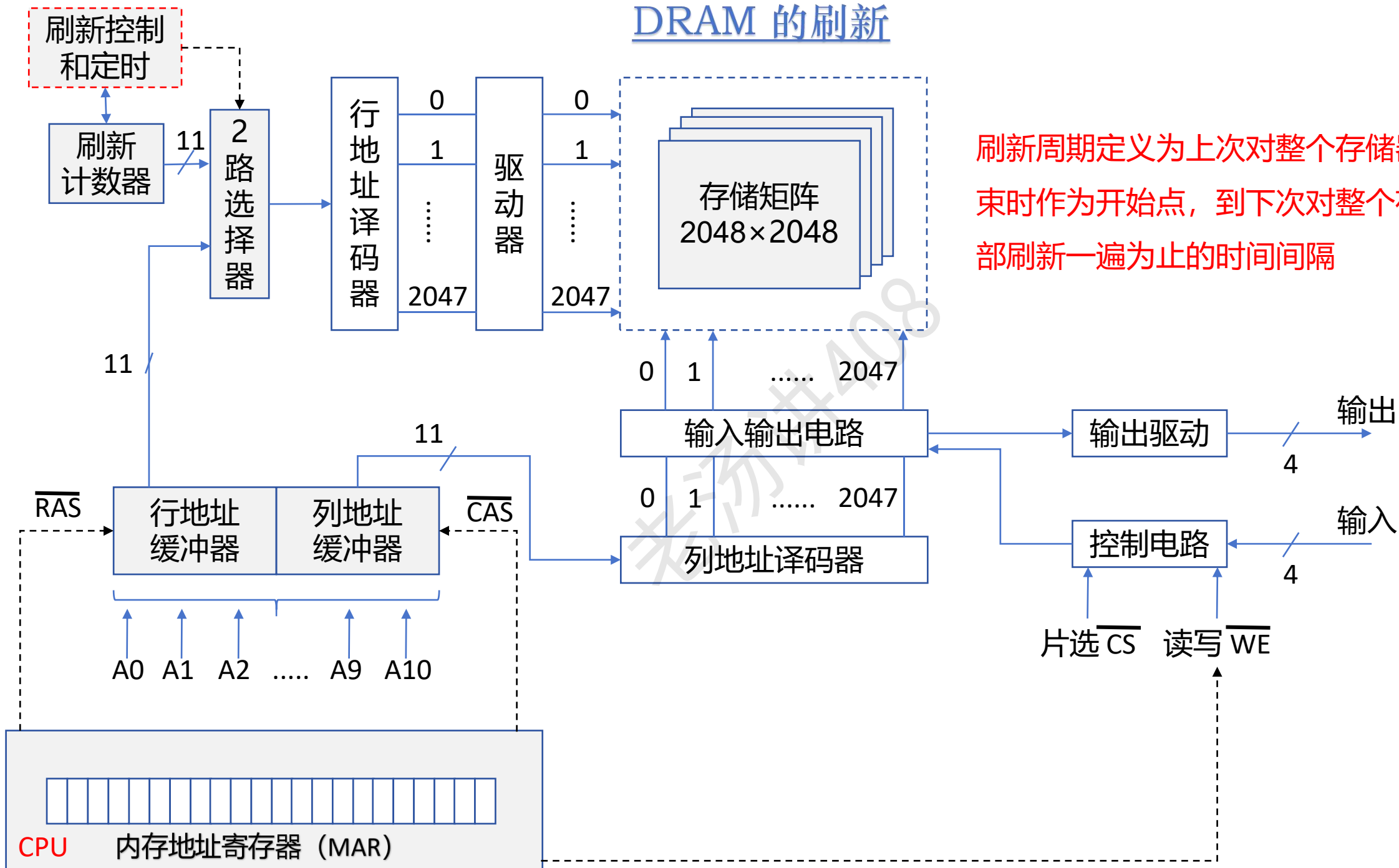
电容 C_S 上存储的电荷会缓慢的放电，超过一定的时间，就会丢失信息，这个是电容的特点，因此必须定时给电容 C_S 充电，这一个过程称为刷新

读后再生

DRAM 的刷新



DRAM 的刷新



刷新周期定义为上次对整个存储器刷新结束时作为开始点，到下次对整个存储器全部刷新一遍为止的时间间隔

DRAM 的刷新策略

① 集中刷新

② 分散刷新

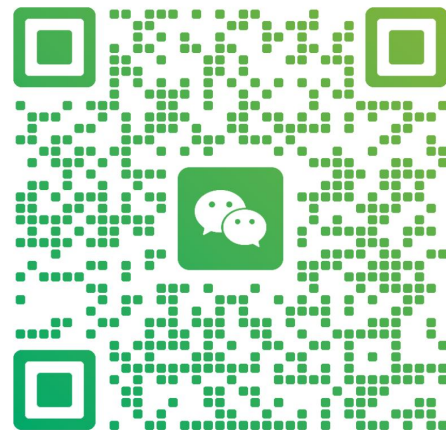
③ 异步刷新

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州

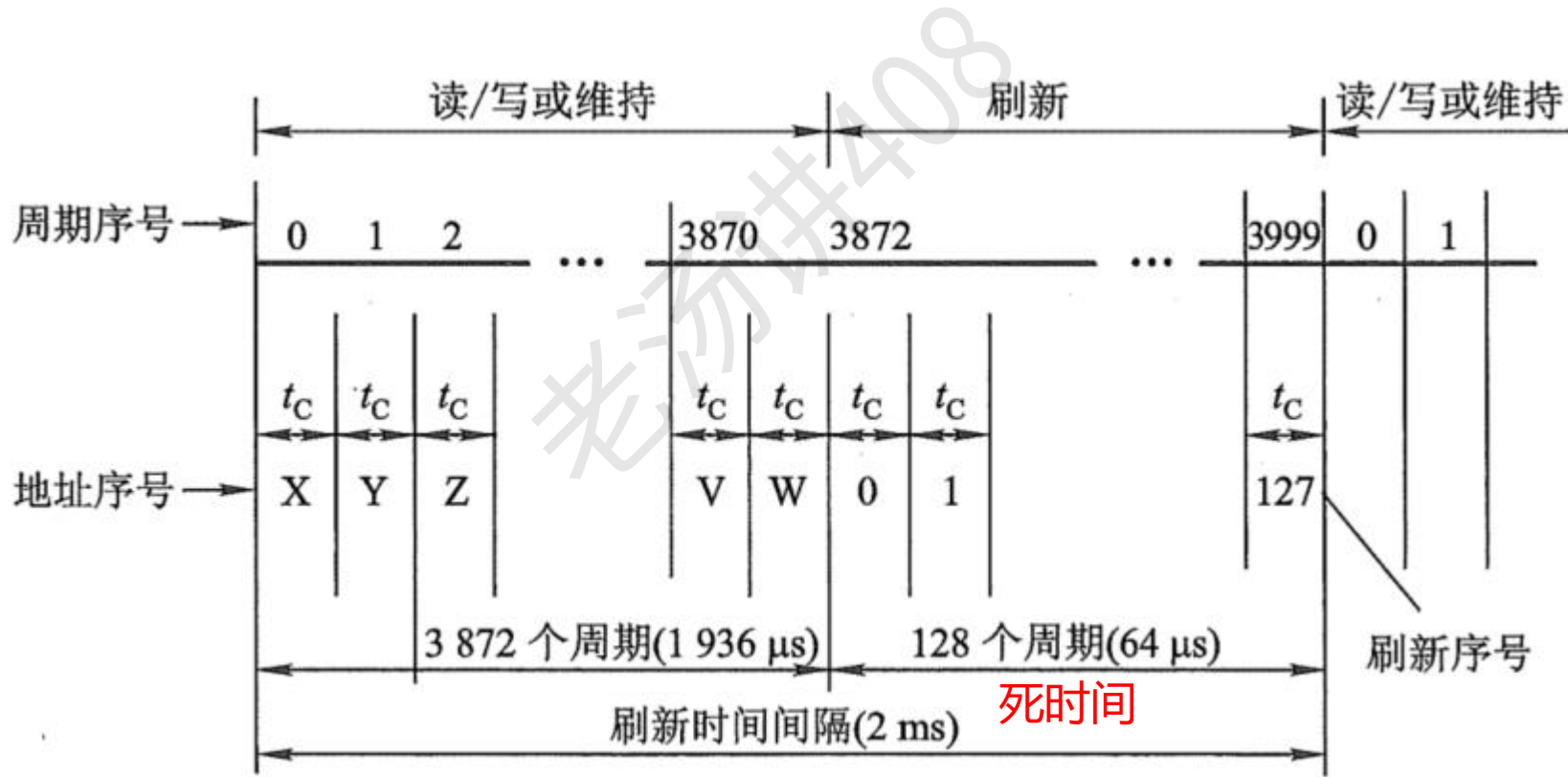


扫一扫上面的二维码图案，加我为朋友。

DRAM 的刷新策略：集中刷新

集中刷新是在规定的一个刷新周期内，对全部存储元集中一段时间逐行进行刷新

存储芯片的存储阵列是 128×128 ，刷新周期是 2ms ，存储周期是 $0.5\mu\text{s}$ $\frac{2\text{ms}}{0.5\mu\text{s}} = 4000\text{个}$ $128 * 0.5\mu\text{s} = 64\mu\text{s}$

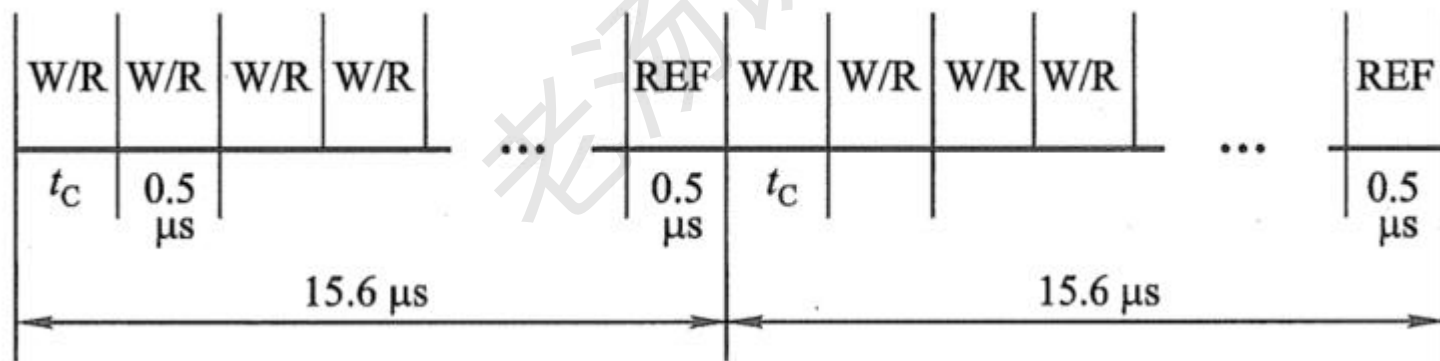


DRAM 的刷新策略：异步刷新

异步刷新是前两种方式的结合，它既可以缩短死时间，又充分利用最大刷新间隔为 2ms 的特点

存储芯片的存储矩阵是 128×128 ，刷新周期是 2ms，读存储周期是 $0.5\mu\text{s}$

$$2\text{ms} \div 128 \approx 15.6\mu\text{s}$$

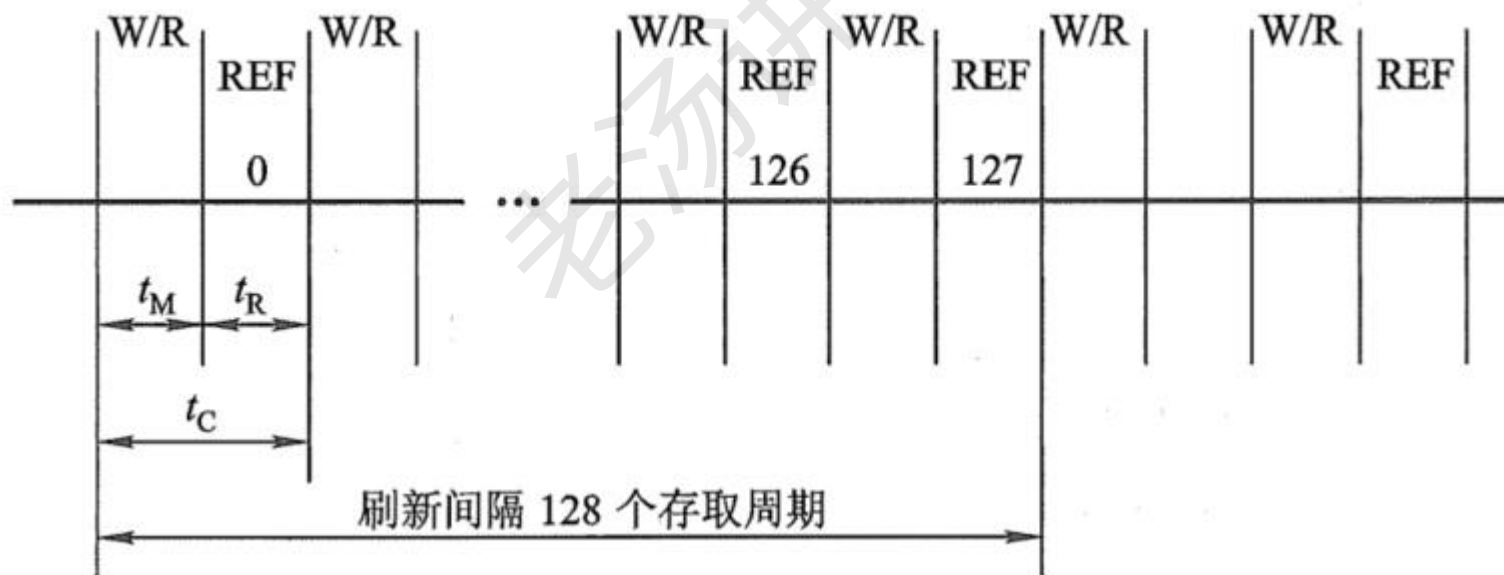


异步刷新即克服了分散刷新独占 $0.5\mu\text{s}$ 用于刷新，使得存储周期加长且降低系统访问速度的缺点，又不会出现集中刷新的访问死区问题

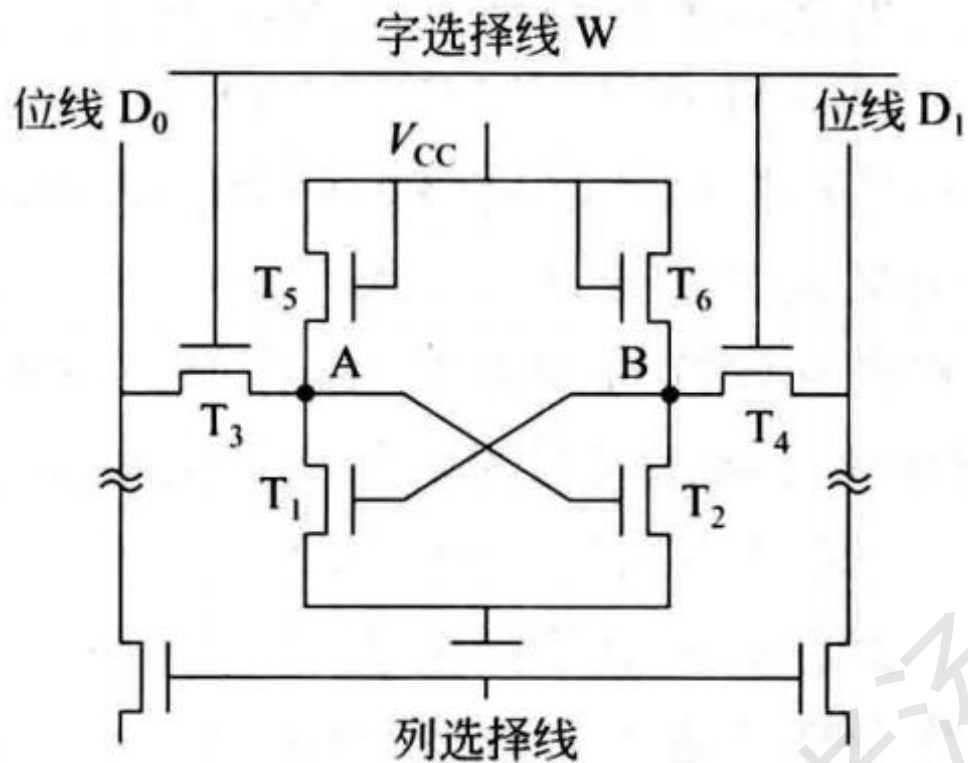
DRAM 的刷新策略：分散刷新

分散刷新是指对每行存储单元的刷新分散到每个存储周期内完成

存储芯片的存储阵列是 128×128 ，刷新周期是 2ms ，读/写周期是 $0.5\mu\text{s}$

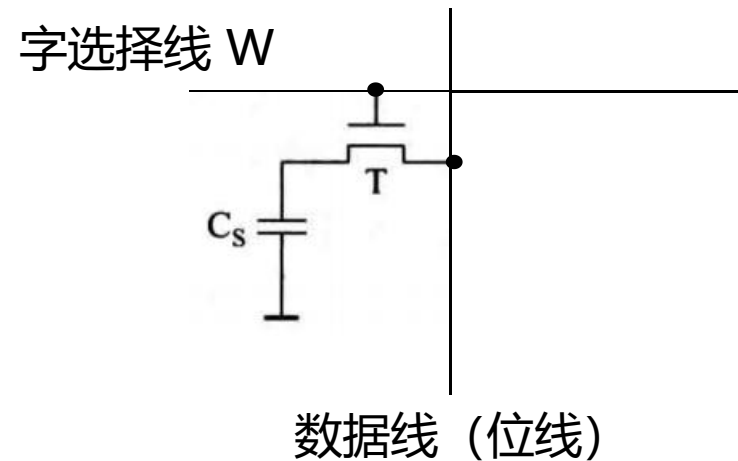


SRAM vs DRAM



SRAM 存储元电路结构

- ① 所用 MOS 管多，占硅片面积大，因此集成度低
- ② 即使存储元不工作，也有电流流过，功耗大
- ③ 只要有电源，可保持状态不变，无需刷新，也不会因为读操作而使状态发生改变，读写速度非常快
- ④ 价格比较昂贵，适合做高速小容量存储器 (Cache)



DRAM 存储元电路结构

- ① 所用 MOS 管少，占硅片面积小，因此集成度高
- ② 存储元不工作，就没有电流流过，功耗小
- ③ 会发生漏电，必须定时刷新，读操作后会使得状态变化，需要读后再生，读写速度相对 SRAM 要慢
- ④ 价格比较便宜，适合做慢速大容量存储器 (主存)